Managing by Feedback

Jinci Liu *

August 3, 2026

Latest Version Available Here

Abstract

How do managers affect worker productivity and retention through feedback? Using data from GitHub and LinkedIn, I analyze over 230 million feedback messages from code reviews across 1.7 million software teams. I use large language models to classify feedback tone (toxicity, positivity) and information (constructiveness). Exploiting the random assignment of code reviewers, I find that toxic feedback reduces workers' subsequent code quantity and quality, whereas nontoxic criticism has no comparable effect. Positive feedback raises productivity. These effects extend beyond code output: positive feedback also raises firm retention within a year, and effects spill over to coworkers. Information content shifts workers toward revising existing tasks and away from new development. Linking these effects back to managers, I find that feedback explains 22% of the variation in manager quality, measured as value added to worker productivity. This paper shows that manager feedback is a factor that affects worker productivity and a measurable component of manager quality.

*IIES, Stockholm University. Email: jinci.liu@iies.su.se.

I am grateful to Arash Nekoei, Mitch Downey, and Jósef Sigurdsson for their encouragement and feedback—always positive and never toxic. I thank Jakob Beuschlein, Kirill Borusyak, Konrad Burchardi, Zoë Cullen, Stefano DellaVigna, Florian Englmaier, Ingrid Haegele, Oskar Nordström Skans, Ricardo Perez-Truglia, Nancy Qian, David Schönholzer, Mateusz Stalinski, David Strömberg, and Horng Chern Wong as well as seminar participants at IIES, UPF, LSE, CUHK, HKUST, CityUHK, UNSW, UIUC, NUS, SMU, the HK Labor Symposium, CSEF-CEPR Labor & Finance, and CEPR IMO & ESF. I am also thankful to Peishan Ju and Ruipu Peng for research assistance in validating the LLM classifications. I acknowledge financial support from Google Cloud Research, the Mannerfelt Foundation, and the Institute for Evaluation of Labour Market and Education Policy (IFAU).

1 Introduction

What determines worker productivity? Economists have long studied drivers such as technology, human capital, and incentives (Solow, 1957; Becker, 1964; Holmström, 1979). A growing literature shows that managers matter, but much less is known about *how* they affect worker productivity.[1]

This paper studies feedback, one of the most common ways managers interact with workers. Managers use feedback to guide. Over 90% of managerial occupations require giving feedback (O*NET, 2025), and leading consultancies call feedback "the fuel of development" (McKinsey, 2024). Yet whether it actually raises productivity is unclear. Existing evidence comes mainly from surveys and reaches mixed conclusions.[2]

The effect of feedback is ex ante ambiguous because feedback carries both information and tone. Information tells the worker how to revise the work, while tone can affect the worker's motivation and effort. Consider a manager who asks a worker to revise their work. A toxic tone may undermine the worker's confidence and lower effort (Compte and Postlewaite, 2004), or it may raise the perceived threat of punishment and induce greater effort (Mirrlees, 1974, 1976). A positive tone may raise effort by improving how workers view their own ability (Bénabou and Tirole, 2003), or it may reduce effort if workers interpret praise as a signal that they have already met the required standard (Kluger and DeNisi, 1996).

Three challenges have limited progress in studying how feedback affects workers. First, in most settings, feedback is verbal and rarely recorded. Second, even when feedback is written, it is multidimensional, which makes it conceptually difficult to decide which dimensions to study and technically demanding to classify them at scale. Third, workers who receive different feedback may differ in unobserved ability, so the correlation between feedback and productivity may reflect selection or reverse causality rather than a causal effect of feedback on worker productivity.

I address these challenges using code review on GitHub, the world's largest online coding platform, used by more than 90% of Fortune 100 companies (GitHub, 2025). Software development is an economically important setting: the digital economy accounted for about 10% of U.S. GDP and 8.9 million jobs in 2022 (U.S. Bureau of Economic Analysis, 2025). In code review, *developers* submit code for integration into the team's codebase, and *reviewers*, typically senior members, provide feedback on submitted code and decide whether to approve it or request revisions. Because developers cannot integrate their code without addressing it, reviewer feedback carries authority that peer feedback does not.

This setting addresses the three challenges. First, code review leaves a complete written record of feedback, rather than the unrecorded verbal exchanges of most workplaces. Second, feedback is abundant and text-based, which lets me use large language models (LLMs) to classify feedback types. Third, some teams adopt random code reviewer assignments, generating variation in the

[1] This literature documents that individual managers influence worker and organizational outcomes; see, e.g., Ichniowski et al. (1997); Bertrand and Schoar (2003); Bandiera et al. (2007); Bloom and Van Reenen (2007); Lazear et al. (2015); Frederiksen et al. (2020); Adhvaryu et al. (2022, 2023); Metcalfe et al. (2023); Weidmann et al. (2026).

[2] In one survey, 80% of workers felt more engaged after receiving "meaningful" feedback (Gallup, 2022). In another, only 14% strongly agreed that the feedback they received was helpful (Sutton and Wigert, 2019).

feedback a developer receives, independent of their unobserved ability.

I construct the dataset from GitHub Archive, which records public code-review activity. I measure weekly developer productivity as code quantity (lines of code) and quality (share of code submissions accepted), and I link developers to LinkedIn career histories from Revelio Labs to measure retention and job switches. The final dataset covers over 1.7 million teams and 230 million feedback messages from 2017 to 2023.

I classify feedback using LLMs. Guided by the seminal work in psychology (Ilgen et al., 1979), I focus on the tone and information content of feedback. I measure two separate dimensions of feedback tone. Toxicity captures intentional, person-directed harm, while positivity captures encouragement or acknowledgment. This distinction is central to the analysis because a manager can criticize the work without attacking the worker, and only the latter is toxic. I measure information content by whether feedback provides specific, actionable guidance, which I label constructiveness. Separating tone from information allows me to compare feedback that differs in tone while holding its information content fixed.[3]

To identify causal effects of feedback, I exploit bot-based reviewer assignment on GitHub. Some teams delegate reviewer selection to an automated workflow that rotates reviewers within an eligible pool on the basis of recent review workload, so that the assigned reviewer is independent of developer and case characteristics. Software teams can activate this feature by adding a script to their codebases, and I identify teams using this feature by scanning their codebases for these scripts. Within these teams, the bot assigns who reviews each submission, creating exogenous variation in the feedback developers receive. I then leverage the variation in reviewer feedback styles for identification. For each feedback type, I construct a measure of reviewer style, defined as the reviewer's average tendency to give that type of feedback in other developers' code reviews. This serves as an instrument for the feedback a developer receives, as it is highly predictive of the reviewer's feedback style in the current case but, as I document, uncorrelated with observable developer and case characteristics. The strategy is analogous to judge-leniency designs, where case assignments are random but outcomes vary because some judges are systematically harsher than others. (Dobbie and Song, 2015; Bhuller et al., 2020; Agan et al., 2023)

I find that feedback tone affects worker productivity and retention, whereas information content shifts effort between revising existing code and writing new code.

Toxic feedback significantly decreases developer productivity. In the month after receiving toxic feedback, developers produce 42.9% less new code for their teams.[4] This decline is not a reallocation of effort because developers do not shift to non-coding tasks or contribute to other teams to escape toxic reviewers, so their total output quantity falls. Nor do they trade quantity for quality, since the acceptance rate of their new code also declines. Toxic feedback thus lowers

[3]Each feedback message receives three binary labels: toxic or non-toxic, positive or negative, and constructive or non-constructive. For example, *What's wrong with you to write such bad code?"* is classified as toxic, whereas *This is bad code"* is classified as non-toxic but negative. See Section **??**.

[4]I focus on future code rather than the reviewed code to ensure that the effect is not mechanically driven by the review process. For example, toxic feedback is more likely to accompany a rejected submission, which would mechanically lower the reviewed code's measured quality.

developer productivity on both the quantity and the quality margin.

Feedback is multidimensional, and most toxic feedback is also negative. Is the decline in productivity driven by toxicity or by negativity? The distinction matters for management practice. After respectful criticism (negative but non-toxic), developers' code quantity does not change significantly, and the decline in code quality is far smaller than after toxic feedback. The harm arises from toxicity, not from expressing dissatisfaction with the work: managers need not avoid criticizing work, only avoid toxicity.

Tone affects productivity in both directions. Whereas toxic feedback lowers it, positive feedback raises it. Developers who receive positive feedback improve the quality of their new code by 6.4%, while code quantity does not change significantly. A possible mechanism is that positive feedback raises effort by improving how workers view their own ability. Models of confidence-enhanced performance and self-esteem imply that workers exert more effort when they hold a higher view of themselves (Compte and Postlewaite, 2004; Bénabou and Tirole, 2002; Kőszegi et al., 2022). The effect of positive feedback also reaches beyond the developer's own work. Developers who receive positive feedback go on to send more positive messages to their coworkers, and they become less likely to leave the firm over the following year.

Why does toxic feedback persist despite its high cost? Managers do not observe the relevant counterfactual. In the raw correlations that managers observe, toxic feedback appears to raise output, and OLS estimates are positive. This correlation partly reflects selection, as developers who submit more code receive more reviews and are therefore more likely to receive toxic feedback. The relationship turns negative once I isolate the exogenous variation generated by random reviewer assignment. The IV estimates then reveal the productivity loss hidden by the raw correlation. Managers who rely on observed outcomes may therefore mistake toxicity for effective management.

Information, by contrast, changes what workers do rather than how much they produce. Constructive feedback, defined as detailed and specific information about submitted code, does not significantly change the amount of new code produced, but it lowers the quality of new code. One explanation is that detailed feedback reallocates effort toward the current code case. Reviewers not only provide feedback but also decide whether submitted code is accepted, so developers must address detailed feedback. Consistent with this mechanism, developers who receive constructive feedback rewrite 49.2% more lines of submitted code. This reallocation leaves less time and attention for new development.

Feedback carries tone and information at once. To study how they combine, I focus on the positive dimension of tone and cross it with information content.[5] Holding tone fixed, adding constructive information does not increase the gains from positive feedback: the largest improvements in productivity come from feedback that is positive but not constructive. Information still matters, but on a different margin: it reallocates effort toward revision rather than raising output.

In the final part of the paper, I examine whether feedback helps identify good managers. Previ-

[5]Toxic feedback is too rare to cross with information content.

ous research shows that managers matter for firm performance and that there are large, persistent differences in manager quality (Weidmann et al., 2026). Yet manager quality is difficult to observe directly. Motivated by these facts, I ask how much variation in manager quality can be explained by the way managers give feedback.

I measure manager quality as their value-added (VA) to worker productivity, following the forecast-based approach of Chetty, Friedman, and Rockoff (2014). In my setting, reviewer VA captures how a developer's productivity changes after being assigned to a reviewer, while holding fixed developer and code-case characteristics. [6] I find that feedback explains a sizable share of the variation in reviewer value-added. A gradient boosting model (XGBoost) shows that the feedback style explains about 22% of the variation in reviewer VA.[7] A simple model using only feedback quantity and the three feedback dimensions, toxicity, positivity, and constructiveness, captures 84% of the incremental explanatory power that the full text embeddings add over feedback quantity alone. These results indicate that the three feedback dimensions studied in this paper summarize much of the relevant variation.

This paper shows that feedback does more than transfer information. Managers can use a feedback tone to affect worker productivity and retention, and feedback explains a substantial share of manager quality in a large, high-skilled setting that is central to the modern economy.

My findings make four main contributions to the literature. First, I contribute to the interdisciplinary literature on feedback across economics, education, and psychology. A widely used definition in education describes feedback as information provided by teachers, peers, parents, or other agents about aspects of one's performance (Banihashem et al., 2022; Hattie and Timperley, 2007). In psychology, feedback is viewed as communication in which a sender conveys information about the receiver, and the receiver's response depends on both the content of the message and how the message is interpreted (Ashford and Cummings, 1983; Ilgen et al., 1979; Kluger and DeNisi, 1996). Recent work in economics studies feedback as a channel for human capital accumulation (Emanuel et al., 2025). Related studies also document gender differences in how feedback is given and received (Shastry et al., 2020; Saygin et al., 2025; Freimane, 2024). I extend this literature by providing large-scale causal evidence on how feedback information and tone affect worker productivity and retention, and vary across workers with different characteristics.

Second, this paper contributes to research on non-wage job amenities and toxic workplace conditions. Much of the economics literature on amenities emphasizes worker sorting, retention, or willingness to accept lower wages for better job conditions. I study a workplace amenity that is embedded in daily production: the tone of managerial feedback. Economic studies have ex-

[6] I estimate reviewer VA separately from changes in developers' code quantity and quality and find that the two measures are positively correlated. I first estimate the reviewer VA in the random reviewer sample. Before extending the analysis to the full sample, I test the validity. I focus on reviewers who work across multiple teams and appear in both random and nonrandom reviewer samples. Their VA rankings closely align, supporting the idea that the forecast-based estimator remains valid beyond random assignment.

[7] I train a gradient boosting model (XGBoost) using three groups of feedback measures. The first is feedback quantity, measured by the number of feedback messages a reviewer sends per week. The second is broad linguistic information from the feedback text, summarized by high-dimensional text-based embeddings. The third is the set of feedback dimensions studied in this paper: toxicity, positivity, and constructiveness.

amined toxic speech outside the workplace, such as on social media and online forums (Smirnov et al., 2023; Ederer et al., 2024), and adverse conditions within firms, including workplace hostility (Collis and Van Effenterre, 2025), sexual harassment (Folke and Rickne, 2022), and violence (Adams-Prassl et al., 2024). Related work outside economics, largely based on surveys and qualitative evidence, shows that toxic environments link to depression, burnout, and absenteeism (Nielsen and Einarsen, 2012; Fattori et al., 2015; McTernan et al., 2013). I connect to this literature by studying toxic feedback inside firms as a specific and measurable non-wage amenity.

Third, the paper relates to work that opens the black box of what managers do, including Hoffman and Tadelis (2021) on people management skills in a large high-tech firm, Haegele (2022) on talent hoarding in a large manufacturing firm, and Minni (2024) on how managers in a consumer goods multinational affect workers through task allocation. I contribute to this literature in two ways. First, I show that feedback is an important yet previously overlooked mechanism through which managers affect workers. Second, I use data from globally distributed firms, moving beyond the single-firm focus of most existing studies. Relatedly, Bandiera et al. (2020) extend the analysis from a single firm to a random sample of manufacturing firms drawn from *Orbis*, surveying CEOs through detailed diary data and showing that those who focus on high-level agendas outperform those who focus on functional management. In contrast, I use large-scale observational data to measure managerial activities directly.

Fourth, the paper relates to research on how emotions and self-confidence shape motivation and effort. Bénabou and Tirole (2002) model self-confidence as a motivational asset that individuals sustain through self-serving beliefs and selective recall. Compte and Postlewaite (2004) show that such biases can raise performance when emotions feed back into outcomes, so that higher confidence becomes self-sustaining. Kőszegi et al. (2022) show that self-image can be fragile and distort behavior when success is central to a person's identity. These models imply that when workers hold a higher view of their own ability, they exert more effort. This literature is largely theoretical, and I provide causal field evidence consistent with the mechanism, showing that the productivity gains from feedback come from its tone.

The rest of the paper is organized as follows. Section 2 describes the data and setting. Section 3 defines and validates the three feedback measures and documents two facts that motivate the empirical designs: feedback style varies mainly across reviewers, and reviews flow hierarchically within teams. Section 4 presents the identification strategy based on random reviewer assignment, and Section 5 reports the effects of feedback on productivity and retention. Section 6 asks how much reviewer quality, measured by value-added feedback, predicts, and Section 7 concludes.

2 Data and Setting

I create a panel dataset by merging three sources: GH Archive (timelines, activities, feedback, and team structure), GitHub profiles (names, images, locations), and Revelio Labs (employment histories). I use the data to (i) characterize reviewer and developer demographics; (ii) identify

reviewer-to-developer feedback messages; and (iii) measure feedback types, developer productivity, and retention. The panel spans 2017 to 2023.

2.1 GitHub and the Code Review Process

GitHub is the world's largest online coding platform, where developers store code, share projects, and collaborate. Technology firms such as Google and Microsoft maintain thousands of public projects and increasingly rely on GitHub for innovation and productivity (Nagle, 2019; Hoffmann et al., 2024).[8] The GH Archive captures all public GitHub activity in near real-time and preserves event-level traces, even after code and feedback are modified or deleted.

Below I define *team*, *code case*, *team member*, *reviewer*, and *developer*, and Section 2.5 states the final sample restrictions used in the analysis.

2.1.1 Team and Code Case

A repository is a workspace where developers store code and track its version history. I define a *team* as a repository owned by an organization, much like a group of researchers co-authoring a paper.[9] In these teams, developers contribute code, review, and discuss. I restrict attention to organization-affiliated teams because they use structured workflows, align with firm objectives such as product development (Andersen-Gott et al., 2012), and better represent workplace settings than individual projects.

A pull request on GitHub is a formal submission of code changes for integration into a team's codebase. I refer to each pull request as a *case*. Submitting a case initiates a structured review process that helps identify bugs, enforce conventions, and maintain quality.[10] Figure 1 shows an example of a code review in a team from Microsoft.

2.1.2 Team Members

Teams include internal members and outside volunteers. Anyone can submit code changes, but only internal members hold project-level permissions. I identify internal members using GitHub's *author association* field, available starting in 2017. This field records the relationship between a user and the repository in which the interaction occurs. I define *team members* as users labeled "owner," "member," or "collaborator". These roles imply write-level access and active involvement in development, including pushing code, reviewing pull requests, and coordinating work.[11]

[8] As of July 2026, GitHub hosts over 331 million public repositories and more than 217 million users (see repository and user, accessed 2026-07-02). Appendix A describes the platform and profile data in more detail.

[9] Economists have defined teams in various ways, from common interests (Marschak and Radner, 1958) to related productive inputs (Holmström, 1982) and observed co-authorship (Jones, 2021). Appendix A discusses how these definitions map to a GitHub repository.

[10] Emanuel et al. (2025) discuss how reviews raise quality and facilitate learning; job postings at Microsoft and Google explicitly require code review.

[11] Because *author association* is observed only when a user acts, I impute continuous membership between the first and last months in which a user is observed as a team member. Appendix A provides details.

Demographics. I infer developers' gender, race, and location from public information in their GitHub profiles. GitHub profiles are often used as professional profiles in software labor markets, giving developers incentives to provide information about their skills, identity, and location (El-Komboz and Goldbeck, 2024). Gender is predicted from profile images, race is classified from names, and location is standardized from self-reported profile entries, as described in Appendix A. These variables are inferred rather than directly observed and should therefore be interpreted as approximate demographic measures. I use them mainly to describe the sample and examine heterogeneity. Among developers with classified gender, about 88% are classified as male. Figure A1 maps the geographic distribution of team members. The United States has the largest concentration, followed by India, China, and Brazil.

2.1.3 Reviewers and Developers

Reviewers are team members formally listed in the "reviewer" field of a code case and are responsible for evaluating the submitted code and providing feedback. *Developers* are the team members who submitted the code case. Reviewer feedback messages come from GH Archive, which records the original text of public GitHub events at the time they occur, including messages that may later be deleted or edited on GitHub.

I do not observe formal managerial relationships, and reviewers can also write and submit code. I nevertheless treat reviewer feedback as managerial feedback rather than ordinary peer feedback because reviewers perform two managerial functions within the code review process: evaluation and approval. Reviewers assess submitted code and decide whether it can be integrated into the team codebase. Developers must address reviewer feedback before integration. This makes reviewer feedback different from ordinary peer feedback, which developers can often ignore without directly affecting the outcome of the case.

This interpretation is supported by the data. In the matched GitHub–LinkedIn sample, reviewers often hold senior titles such as "Principal Engineer" or 'Senior Software Engineer." Appendix A.7 compares reviewers with the developers they review and shows that reviewers differ systematically along several observable characteristics. Many teams also display a hierarchical review structure in which reviewers provide feedback to others but rarely receive feedback from them. The main results are nearly identical when I restrict the sample to teams with highly hierarchical review structures.

2.2 Reviewer Assignment

When a developer submits a case that requires review, GitHub recommends reviewers based on prior contributions to the affected code, using *git blame* to track who last modified each line. The developer selects a reviewer by clicking "Request" next to a suggested name or by typing a team member's username.

2.2.1 Random Assignment

In teams that handle many cases, review workloads often become uneven, which can reduce efficiency and slow collaboration. To address this, some teams adopt random reviewer assignment tools that balance workloads by rotating reviewers based on recent activity.[12]

Under bot-based assignment, developers do not choose their reviewers. Instead, the workflow system assigns reviewers by rotating among eligible reviewers based on recent review workload. Appendix B describes the assignment algorithm in detail. Consistent with this design, reviewer-developer matches exhibit little persistence. Conditional on a developer being reviewed by a given reviewer, the probability that the same reviewer evaluates the developer's next case is only 7.32%. Section 4.1 further shows that reviewer assignment is uncorrelated with observable case and developer characteristics.

2.3 Developer Productivity Measures

The main productivity-related outcomes are code quantity and code quality, measured at the developer-team-week level. I also measure non-code activity quantity to capture potential reallocation of effort across tasks in response to different types of feedback.

Code Quantity. I measure code quantity using the number of lines of code a developer submits to a team in a week. I also count rewritten lines to capture revisions to previously submitted code, as described in Appendix A.8. Line counts are a standard measure of code output in my setting, where the sample consists of professional developers. They are also widely used as a proxy for code output in both computer science and economic literature (Vasilescu et al., 2015; Wagner and Ruhe, 2018; Emanuel et al., 2025).

Code Quality. I measure output quality with two indicators. **Code quality** is the share of code lines that are accepted and retained in the team codebase without revision. A higher value implies the submission was accepted as written; lower values indicate more extensive post-submission revision (McIntosh et al., 2016; Emanuel et al., 2025). Both components are assigned to the week in which the case was opened. **Case quality** is the share of cases whose initial submission is merged into the team's main codebase without major revisions. Appendix A.9 provides definitions and examples. Figure A4 shows that both measures correlate positively with developers' number of GitHub followers, a signal of professional reputation, supporting their validity.

Non-Code Quantity. I measure a developer's non-code quantity each week as the number of non-code GitHub activities, such as participating in issue threads, answering questions, or providing troubleshooting support.

[12] Teams implement this using GitHub Actions and configuration files such as `reviewer-lottery.yml` or `CODEOWNERS`. Section B details the assignment rules, lists the configuration files, and compares adopting and non-adopting teams.

2.4 Developer Career Histories

To measure retention, I link developers to LinkedIn-based career histories from Revelio Labs. Revelio Labs compiles information from LinkedIn and other public sources, producing a panel of 1.25 billion professional profiles worldwide. The data record job histories with start and end dates for each employer–position spell as of August 2024.[13] I use these data to construct two measures of career mobility. I define an *firm switch* as a move to a new LinkedIn employer within one year, and an *occupational switch* as a move to a position in a different occupation within one year, where occupation is the SOC major group (the first two digits of the O*NET-SOC code). Occupation is measured at the worker's role, unlike the employer's industry. I also construct a *industry switch* and report it in Appendix F.

2.5 Sample Restrictions and Summary Statistics

I apply three restrictions to ensure the sample captures structured workplace collaboration rather than ad hoc activity. First, I restrict attention to teams under organizations, excluding projects operated by individual users. Second, I retain only code submissions made by formal team members, who contribute more regularly. Third, I focus on cases with at least one reviewer.

I construct two samples. The *full sample* includes all teams that satisfy these restrictions between 2017 and 2023. It covers 1,774,184 teams and 56,637,966 cases, of which 17.11% have at least one assigned reviewer. These reviewed cases generated over 230 million feedback messages.

The *random reviewer sample* includes 3,457 teams and 4,228,687 cases. Although these teams represent only a small fraction of the full sample, they are highly active. Table 1 shows that teams adopting random reviewer assignment are systematically larger and handle far greater review volumes than other teams. On average, they have nearly 20 members, process 1,223 cases, and generate around 7,070 feedback messages per team. Although they are a small share of all teams, they account for 7 % of all code cases and 11% of all feedback messages in the full sample. This high case volume is a plausible reason for adoption, since assignment lowers the coordination cost of allocating reviewers and distributes reviews more evenly. These teams also tend to belong to leading technology firms (Table B2).

I then turn to developers' outcomes, summarized in Table 2. Productivity is measured along three dimensions: code output, code quality, and non-code activities. Because output distributions are heavy-tailed, I winsorize all continuous variables at the 95th percentile while keeping bounded outcomes such as case correctness and code acceptance rates unchanged. Table 2 further distinguishes the "Focal Team," where the developer received reviewer feedback on a specific case in a given week, from "Other Teams," which aggregates the developer's activity across all other teams in the same week.

[13] A random sample of 1,000 developers shows that 71.2% have LinkedIn profiles, and 97% of these are also in Revelio Labs. Appendix F details the GitHub–LinkedIn matching procedure.

3 Measuring Feedback: Tone and Information

This section classifies reviewer feedback and documents how it varies in the data. Manual classification cannot scale to hundreds of millions of feedback messages and is inconsistent across raters, so I use large language models (BERT-based models and GPT) and validate the results against human annotation. I first describe the overall structure of feedback in the data. I then define and measure three dimensions of feedback: toxicity, positivity, and constructiveness; report the classification results; and close with two descriptive facts that motivate the empirical strategy.

Feedback is multidimensional and can be interpreted from many perspectives. Before focusing on tone and information, I summarize the main patterns in the data using principal component analysis (PCA) on the full corpus of reviewer feedback. Unlike supervised methods that depend on predefined labels, PCA recovers the main axes of variation directly from the text. Two dimensions dominate: positivity (for example, *good*, *thank*) and constructiveness (for example, *add*, *test*), as shown in Figure C1. Toxicity does not emerge as a leading component because toxic feedback is rare on public platforms (Beknazar-Yuzbashev et al., 2022); I nonetheless study it because it carries strong policy implications and is linked to lower well-being (Nielsen and Einarsen, 2012).

Before classification, I clean and standardize the feedback text. I remove quoted blocks and code snippets, expand software acronyms, and adjust for programming terms, such as "kill" or "dump," that are benign in code but may be flagged by general-purpose language models (Sarker et al., 2020). Appendix C describes these preprocessing steps.

3.1 The Tone of Feedback

3.1.1 Toxicity

The first dimension is toxicity, which captures whether feedback inflicts intentional harm. The Cambridge Dictionary defines "toxic" as "very unpleasant or unacceptable, causing you a lot of harm and unhappiness over a long period." I classify toxicity using the ToxiGen-RoBERTa model (Hartvigsen et al., 2022), which, unlike keyword-based classifiers, detects both explicit and implicit toxicity (offensive humor, insults, personal attacks, profanity, aggression, sexual harassment, and discrimination). This definition follows the ToxiGen human-annotation guidelines (Figure C2). The model, a fine-tuned checkpoint of RoBERTa (Liu et al., 2019), achieves 94.5% accuracy and has been used in economics (Ederer et al., 2024) and in workplace settings, for example by Microsoft to flag harmful communication.[14]

Crucially, ToxiGen distinguishes blunt criticism from personal attacks. For example, it classifies "This is bad code" as non-toxic with a probability of 0.918 and "Your paper is not of general interest to publish at the AER" as non-toxic with a probability of 0.999. By contrast, feedback directed at the recipient is more likely to be classified as toxic. Examples include "What's wrong with you to write such bad code?" (0.712), "You'll never survive the job market with work like

[14] See https://www.microsoft.com/en-us/research/publication/toxigen, accessed 2025-10-14.

this" (0.986), and discriminatory statements such as "Women are just not good at coding" (0.988). Figure C6 presents examples of toxic feedback and developer replies, while Appendix C reports additional scored examples.

3.1.2 Positivity

The second dimension is positivity, which captures whether feedback conveys a supportive or discouraging tone. I measure it with SiEBERT, a sentiment model trained on 12 million labeled documents (Hartmann et al., 2023) that is substantially more accurate than dictionary-based methods across domains and has been used in economics (Brynjolfsson et al., 2025). The model classifies "This is bad code" as negative (0.999) and "Look good to me" as positive (0.996).

3.2 The Information of Feedback

3.2.1 Constructiveness

The third dimension captures the information of feedback. Information is difficult to measure because feedback can contain many kinds of information. I focus on a narrow and relatively objective feature: whether the feedback provides specific and actionable information about the submitted code. I refer to this dimension as constructiveness.[15] Under this definition, feedback is constructive when it identifies a concrete issue or suggests a concrete action that the developer can take. By contrast, feedback such as "You can do it in a better way" contains little actionable information and is classified as unconstructive, even if it may be helpful in some situations.

Measuring this dimension requires understanding both the feedback message and the underlying code. GPT models, trained on large and diverse corpora, can integrate this broader context, which makes them well-suited to the task. I therefore pair each feedback message with the corresponding code submission and ask the model to score whether the feedback provides specific and actionable information about the code, from 0 (least) to 10 (most). To select prompts and models, I benchmark four OpenAI models, GPT-4.1-nano, GPT-4o-mini, GPT-4o, and o3, against human labels. Appendix C.3 reports the full procedure, and Table C2 provides examples. For instance, "Agree" scores 0, "This is not clear" scores 1, and a specific suggestion to handle severe error cases scores 4.

[15] The Cambridge Dictionary defines "constructive" as "advice, criticism, or actions that are useful and intended to help or improve something." Prior work labels constructive comments as "high-quality comments that contribute to the conversation" (Kolhatkar et al., 2020). But this evidence comes from news discussions rather than technical platforms like GitHub, where feedback is shorter and highly specialized. This paper focuses on specific and actionable information about the submitted code.

3.3 Classification Results

Table 1, Panel C, compares classification results across the full sample and the random reviewer sample.[16] The two samples yield similar patterns: at the message level, about 0.21% of feedback is toxic, 39.40% is positive, and under 31.36% is constructive. Figure 2a illustrates these shares.

To show how the labels are assigned, Figure C3 plots the distributions of predicted scores. Toxicity probabilities cluster between 0.8 and 1.0, indicating that ToxiGen flags toxic feedback with high confidence. For constructiveness, research assistants labeled a subsample and reached 90% agreement with the model's binary classification; the results are robust to using a cutoff of 3 rather than 4.

Table 3 cross-classifies feedback along all three dimensions. The largest group is non-toxic, negative, and non-constructive (38.5%), for example "This is not clear." Toxic feedback is overwhelmingly negative and non-constructive (85%), such as "You are talking nonsense." Positive feedback is often non-constructive (76%), typically short affirmations like "Looks good to me." Most constructive feedback is non-toxic and negative (70%). For example, "Still feels redundant. An angle should just be a union of float and FreeParameter. Once the parameter is bound, you have a new circuit with no free parameters."

3.4 Descriptive Facts: Reviewer Styles and Team Hierarchy

The classified feedback reveals two facts about code review that motivate the empirical strategy. First, feedback style varies substantially across reviewers. Second, feedback flows hierarchically within teams, with some members regularly reviewing others but rarely being reviewed by them.

Fact 1: Reviewers explain most of the variation in feedback. To identify the sources of variation in feedback tone and information, I conduct the decomposition at the reviewer message level, where each observation is linked to a unique reviewer. I regress the predicted toxicity, positivity, and constructiveness of each message on team, reviewer, and developer fixed effects, together with case level controls. The unit of observation is a developer–team–week–reviewer cell. When a developer receives feedback from multiple reviewers in the same week, each reviewer contributes a separate observation, weighted by the number of messages. Figure 3 shows that reviewer fixed effects explain the largest share of variation in all three dimensions. Team fixed effects explain much less, while developer characteristics and case controls add little. Feedback therefore appears to reflect persistent reviewer styles rather than team culture, developer traits, or case context.

Fact 2: Most teams follow a hierarchical feedback structure. I measure team hierarchy from the direction of feedback exchanges. A fully hierarchical team has feedback flowing in one direction, as when managers review juniors but not vice versa; reciprocal feedback, where two members

[16] In the random reviewer sample, all feedback messages are classified. In the full sample, toxicity and positivity are measured on all messages, while constructiveness is evaluated on a subsample due to budget constraints. Unless otherwise noted, classification results refer to the random reviewer sample.

review each other's code, indicates a more egalitarian structure. For each team-year, I build a directed network in which an edge from A to B indicates that A reviewed B's code at least once, and define the hierarchy score as

$$\text{Hierarchy Score} = 1 - \frac{2R}{E},$$

where E is the number of directed edges and R is the number of pairs that review each other in both directions. Each reciprocated pair accounts for two directed edges, so $2R/E$ is the share of review relationships that are reciprocated. A score of 1 indicates purely one-way review, while 0 means every relationship is reciprocated.Figure 4 shows the distribution of hierarchy scores for teams of size five, together with three example feedback networks at low, medium, and high hierarchy. Most teams are hierarchical: code reviews tend to flow in one direction rather than between peers, and Figure C5 shows the pattern holds across all teams, with reviewers specializing in reviewing rather than contributing code.

Taken together, these facts guide the empirical strategy. Feedback tone and information vary primarily across reviewers, not teams or developers, and provision is concentrated among a small group of reviewers. This combination means that reviewer styles generate meaningful, plausibly exogenous variation in the feedback developers receive, which I exploit to identify their effects on worker productivity.

4 Empirical Strategy

I estimate how different types of reviewer feedback affect developers' subsequent productivity in the same team during the four weeks after review. The unit of observation is a developer by team by week.[17] The treatment is whether the developer receives a given feedback type in week t, and outcomes are measured over weeks $t+1$ to $t+4$. To illustrate the empirical strategy, I begin with toxic feedback as an example; the same framework applies to the other types.

Consider the following structural equation, where t indexes calendar weeks when feedback is given:

$$Y_{im,[t+1,t+4]} = \beta I^{\text{Toxic}}_{im,t} + \Gamma_t X_{i,t} + \gamma_{m,\text{year}(t)} + \varepsilon_{im,t}, \tag{1}$$

where $Y_{im,[t+1,t+4]}$ is the productivity of developer i in team m over weeks $t+1$ to $t+4$ (e.g., code quantity or quality, defined in Section 2.3). The indicator $I^{\text{Toxic}}_{im,t}$ equals 1 if developer i received toxic feedback in team m during week t, and 0 if the feedback was non-toxic. The vector $X_{i,t}$ controls for feedback volume and activity level (measured in deciles) in week $t-1$. I also include team-

[17] More precisely, an observation corresponds to a week in which developer i received reviewer feedback on at least one case in team m. When a developer has multiple reviewed cases in the same week, I retain each case as a separate observation. Because outcomes are measured at the developer by team by week level, cases reviewed within the same week share the same outcome. This structure induces correlated errors and gives greater weight to developers who submit more cases. Restricting the sample to each developer's first reviewed case in a given week yields similar estimates.

by-year fixed effects $\gamma_{m,\text{year}(t)}$.[18] The coefficient β captures the effect of receiving toxic feedback in week t.

The main challenge is that OLS estimates of (1) may be biased if the receipt of toxic feedback correlates with unobserved determinants of productivity. The bias can run in either direction. On the one hand, less able developers may be both less productive and more likely to attract toxic feedback, so OLS overstates the harm of toxicity. On the other hand, reviewers may direct toxic feedback at developers they expect to respond with greater effort, for instance, those with weak outside options who fear dismissal, so OLS understates it. These unobserved factors complicate identification and can distort OLS estimates in either direction.

The as-if random assignment of reviewers to developer cases, conditional on team-by-year fixed effects, is independent of developer and case characteristics. Reviewers differ systematically in their feedback styles: some are more toxic than others. This difference in feedback style generates variation in developers' exposure to toxic feedback. The assigned reviewer, therefore, shifts a developer's exposure to toxicity for reasons unrelated to the developer or the case. [19]

To instrument for receiving toxic feedback, I construct a leave-developer-out measure of reviewer toxicity. This follows the judge or officer designs in Dobbie and Song (2015); Dobbie et al. (2017, 2018); Bhuller et al. (2020). Reviewers and developers may interact repeatedly, so a simple leave-one-case-out measure risks bias if the same pair appears elsewhere. I therefore exclude all cases involving the focal developer when computing a reviewer's toxicity rate, not only the focal case. Concretely, the instrument is the average share of toxic feedback the reviewer gives in cases with other developers. This procedure ensures that the instrument is not mechanically correlated with the focal developer's outcomes.

$Z^{\text{Toxic}}_{j(i)}$ is the leave-developer-out mean measure of toxic feedback for each reviewer j:

$$Z^{\text{Toxic}}_{j(i)} = \frac{1}{\sum_{h \neq i} n_{hj}} \sum_{h \neq i} D^{\text{Toxic}}_{hj}$$

where D^{Toxic}_{hj} is the number of cases in which reviewer j gives toxic feedback to developer h, and n_{hj} is the total number of cases reviewer j reviews for developer h.

The main analysis will be based on 2SLS estimates of the second-stage equation (1) and the first-stage for developer i and reviewer j at event time t is given by:

$$I^{\text{Toxic}}_{im,t} = \alpha Z^{\text{Toxic}}_{j(i)} + \delta_t \mathbf{X}_{i,t} + \gamma_{m,\text{year}(t)} + \varepsilon_{im,t}, \tag{2}$$

[18] Following the judge leniency literature, I condition on the block where assignment is as good as random and include block fixed effects. Identification comes from within block comparisons (Dobbie and Song, 2015; Dobbie et al., 2017; Bhuller et al., 2020; Beuschlein, 2024). In the data, reviewer assignment primarily occurs within team-by-year cells. The average reviewer tenure on a team is 1.12 years. I therefore include team-by-year fixed effects. Results are robust to team-by-month fixed effects.

[19] The same logic applies to positive and constructive feedback. Some reviewers are systematically more positive (Figure C4b) or more constructive (Figure C4c). Consequently, the probability that a developer receives a given feedback type depends on the tendencies of the assigned reviewer. I exploit this reviewer-driven variation as the basis for an instrumental variables strategy to identify the causal effects of feedback types.

where the scalar variable $Z^{\text{Toxic}}_{j(i)}$ denotes the toxicity of reviewer j assigned to developer i, and α is the effect of reviewer toxicity on the likelihood of receiving toxic feedback. Robust standard errors are clustered at the reviewer level in both stages.[20] I report the robust first-stage F-statistic, which is large in this setting (Staiger and Stock, 1994; Lee et al., 2022).[21]

I interpret the 2SLS estimates within the Local Average Treatment Effect (LATE) framework (Angrist and Imbens, 1994). Under conditional independence, exclusion, and monotonicity, the instrument recovers the local causal effect of receiving toxic feedback among the subgroup of developers whose likelihood of exposure varies with the assigned reviewer. Conditional independence follows from as-good-as-random reviewer assignment within team-by-year cells and is assessed in Figure 6. Exclusion requires that the assigned reviewer's style affects developer outcomes only through the targeted feedback dimension. I discuss this assumption below and assess it in Section 5.6. Monotonicity requires the instrument to shift exposure in the same direction for all developers. Section 4.1 presents evidence consistent with this condition.

4.1 Instrument Validity

Instrument Relevance. Figure 5 provides a graphical representation of the first stage. It shows reviewer-level feedback propensities after residualizing team-by-year fixed effects. To reduce measurement error, I only include teams with at least 2 reviewers and reviewers who have handled at least 10 cases. This selection leaves 480,697 cases and 10,873 reviewers, with an average of 44.2 cases per reviewer. All three panels reveal substantial cross-reviewer variation in the likelihood of sending toxic, positive, or constructive feedback.

Panel (a) in Figure 5 shows residualized reviewer toxicity: moving from the 10th to the 90th percentile raises the probability of receiving toxic feedback by 0.2 pp, a 26.7% increase relative to the mean rate of 0.76%.[22] For positivity in Panel (b), moving from the 10th percentile to the 90th raises the probability of positive feedback by 31 pp, a 43.9% increase over the mean of 70.6%. For constructiveness in Panel (c), moving from the 10th percentile to the 90th raises the likelihood of constructive feedback by 41 pp, a 79.2% increase over the mean of 51.8%.

Table 4 presents first-stage estimates from regressions of indicator variables for receiving each feedback type on the corresponding reviewer instrument, defined as the reviewer's residualized rate of providing that feedback type. Column (1) includes team fixed effects, Column (2) replaces

[20] I cluster standard errors at the reviewer level because misclassification in feedback detection is likely correlated within reviewers. Each reviewer evaluates many pull requests, which induces within-reviewer correlation in the errors. Clustering at the reviewer level addresses this dependence and yields a valid inference. As a robustness check, Figure D1 reports results with team-level clustering; the estimates are similar.

[21] Assessing the risk of many-weak-instrument bias in judge-style designs remains an open question (Hull, 2017; Frandsen et al., 2023; Bhuller et al., 2020; Agan et al., 2023). In Table D1, I evaluate the robustness of the leave-developer-out reviewer leniency instrument using alternative IV estimators. These include: (i) limited information maximum likelihood (LIML) with all reviewer dummies as instruments; (ii) the modified bias-corrected two-stage least squares (MBTSLS) estimator of Kolesár et al. (2015); and (iii) the unbiased jackknife instrumental variable estimator (UJIVE) of Kolesár (2013). The estimates are quite stable.

[22] The message-level toxicity rate is 0.21 % as shown in Table 1, while the case-level treatment rate in the IV sample is 0.76 percent because a case can contain multiple reviewer messages and is classified as toxic if any reviewer feedback in the case is toxic.

these with team-by-year fixed effects, and Column (3) adds controls for developer gender, race, GitHub activity in the year before submission, the first year of GitHub activity, case characteristics, and project stage, defined based on the release version of the team product (See Appendix A.10). Standard errors are clustered at the reviewer level. The coefficients are large and statistically significant across all panels. In Panel (a), a 10 pp increase in a reviewer's residualized toxicity rate raises the probability of receiving toxic feedback by roughly 7 pp. Panel (b) shows a comparable effect for positive feedback (8.2 pp), while Panel (c) reports a slightly larger effect for constructive feedback (8.6 pp).

Conditional Independence. A valid instrument must be uncorrelated with developer and case characteristics that affect outcomes. Column (3) of Table 4 adds controls for these predetermined variables to the first-stage regressions. If reviewer assignment is random, the coefficients should remain stable, which is exactly what I find.

As a second test, Figure 6 assesses the randomness of case assignment after controlling for team-by-year fixed effects. In Panel (a), the blue lines with red markers plot coefficients from regressions of reviewer toxicity on standardized developer and case characteristics. None of the coefficients is statistically significant. Similar patterns hold for positive and constructive feedback in Panels (b) and (c). Taken together, these findings provide strong evidence that reviewer assignment is conditionally random.

Exclusion Restriction. Conditional random assignment is sufficient to interpret the reduced form effect of assignment to a given reviewer as causal. Interpreting the IV estimates as causal effects of a specific feedback type requires a restriction on the instrument: conditional on team-by-year fixed effects and controls, the assigned reviewer's style in that dimension must affect developer outcomes only through the included feedback treatment. Three things could break this.

First, feedback dimensions are correlated. Most toxic feedback is also negative (Table 3), so assignment to a more toxic reviewer may also expose the developer to more negative feedback. The estimated effect may therefore capture both toxicity and negativity. Second, reviewer styles are correlated. A reviewer who is more toxic may also provide less praise, so the toxicity instrument may partly capture the absence of encouragement. Third, reviewers differ in ways other than their language. For example, if senior reviewers are more toxic, the estimate may reflect the weight developers place on a senior evaluation rather than the language used. I return to this issue after presenting the main results in Section 5.6.

Monotonicity Condition. If the effect of reviewer feedback were homogeneous across developers, the instrument would need to satisfy only conditional independence and the exclusion restriction. With heterogeneous effects, recovering the LATE also requires monotonicity: the instrument must shift the probability of receiving a given feedback type in one direction across developers. Intuitively, a developer who would not receive toxic feedback from a more toxic reviewer would

also not receive it from a less toxic one. I cannot directly test this assumption, but I can use several indirect tests common in the literature. Following Bhuller et al. (2020) and Beuschlein (2024), I confirm that first-stage estimates are nonnegative across subsamples split by gender, race, activity level, GitHub experience, and follower count. Across all splits, the first stage is positive and statistically different from zero (Appendix Table D2), consistent with the monotonicity assumption. A negative first stage in any observed subgroup would have been direct evidence against monotonicity, and none appears.

5 Effects of Feedback on Developer Productivity and Retention

This section presents the main results on developer productivity and retention. I measure productivity on new code written after feedback, which avoids mechanically linking feedback to the code under review. I report effects on code quantity and quality, and on non-code task activity. Finally, I link feedback to developers' career trajectories to examine its effect on retention. The main results estimate the effect of each feedback dimension separately; I also report a joint specification that includes all three dimensions in Figure D2.

5.1 The Tone of Feedback

Panel A of Table 5 presents 2SLS estimates of the effect of feedback tone on developer productivity within a team over the four weeks following a review.

Toxic feedback reduces code quantity and quality. Toxic feedback lowers subsequent productivity on every margin. Developers become 7% less likely to contribute any code, and when they do code, they write 43% fewer lines (0.56 log points).

Three natural alternatives do not explain the decline. First, developers do not reallocate effort toward other work: toxic feedback does not affect non-code activity such as answering users' questions about the team product, so they produce less of everything rather than reallocating. Second, this is not a quantity-for-quality trade: quality does not rise to offset the lost output. Both quality measures fall. The share of accepted lines drops 9.1 pp (13%), and the share of merged cases also declines, though less precisely. This suggests that toxic feedback not only reduces how much developers produce but also discourages careful and accurate coding. Third, it is not reviewers withholding future work. The number of cases developers opened is unchanged (Figure G11b). What remains is the developers' own response. After toxic feedback, they do less, and what they do, they do less carefully.

Toxicity, not criticism. Most toxic feedback is also negative, which raises the question of whether the harm comes from toxic tone or from negative (non-positive) tone. Figure 7 shows the effect on developer productivity of receiving respectful criticism (non-toxic & negative). While toxic feedback significantly decreases code quantity and quality, respectful criticism does not: code

quantity holds up, with only modest quality costs. Toxic feedback harms not because it criticizes work, but because it attacks the worker.

Positive feedback increases code quality. Positive feedback works in the opposite direction on quality (Panel A of > Table 5). The share of accepted lines rises 4.4 pp (6.35%). Quantity does not rise with it: conditional on coding, the change in code lines is insignificant. Positive feedback thus raises the quality of what developers write rather than the amount.

These findings on feedback tone align with theoretical models of confidence and performance. Compte and Postlewaite (2004) formalize how positive affect and selective recall of successes sustain effort and productivity. Similarly, Bénabou and Tirole (2002) and Kőszegi et al. (2022) emphasize the central role of self-confidence: positive feedback reinforces beliefs about ability and raises effort, while toxic feedback can undermine these beliefs and reduce productivity.

5.2 The Information of Feedback

Constructive feedback shifts effort toward revision. Constructive feedback, defined as specific and actionable information about submitted code, has different effects from feedback tone. Panel B of Table 5 shows that conditional on writing code, developers produce a similar amount after receiving constructive feedback, but the quality of the new code they do write declines: the share of accepted code falls by 10.39%.[23]

This pattern is consistent with a reallocation of effort toward revision. Constructive feedback may require developers to spend more time addressing the current code case before moving on to new work. Consistent with this interpretation, Figure 9a shows that developers rewrite 49.18% more lines after receiving constructive feedback. Developers therefore shift effort and attention toward revising previously submitted code, leaving less time for careful work on new code.

These results highlight an opportunity cost of detailed feedback. Reviewers may focus on improving the current submission without fully internalizing the developer time required to address their feedback. Specific information can induce revisions that satisfy the reviewer, but these revisions may come at the cost of new code production and quality. The result is a tradeoff between improving the current code case and developing new work.

Tone versus information. The previous results study feedback tone and information content separately. I next examine feedback that combines these dimensions to ask whether productivity gains come from what reviewers say or how they say it. Figure 8 shows that the largest improvements in both code quantity and code quality come from positive tone without the specific and actionable information.

[23] Figure D5 reports results using an alternative threshold, ≥ 3, to define constructive feedback. The findings are similar, indicating that the results are not driven by the choice of threshold.

5.3 Why Toxic Feedback Persists: OLS versus IV

Table 5 contrasts the OLS and IV estimates, and in OLS, toxic feedback appears beneficial: developers who receive it produce 6.72% more code and 8.33% more non-code activity, even as code quality declines. The IV estimates reverse this conclusion and find that toxic feedback reduces code quantity by 42.88% and accepted code by 13.13%.

This contrast helps explain why toxic feedback can exist in organizations. If firms observe only correlations, toxic reviewers may appear effective, or at least harmless. Developers who receive toxic feedback may be more able, more visible, or assigned to cases with higher expected output. Such selection makes toxicity look positively associated with productivity even when its causal effect is negative. Toxic feedback may therefore persist not because it improves performance, but because observational data obscure its costs. A further obstacle is detection: firms rarely measure how feedback is delivered, and reviewers may not perceive their own feedback as toxic, so the behavior can go unaddressed even when harmful.

5.4 Retention and Spillovers

Retention. The effects of feedback extend beyond the coding week. Linking developers' GitHub activity to their LinkedIn careers (Appendix F), I find that positive feedback lowers the probability of switching firms within one year by 8 pp, while toxic feedback has no significant effect on retention (Figure 9b). The effect is specific to firm attachment: no feedback type significantly changes the probability of switching occupations or leaving the industry (Figure F1). These results indicate that feedback is a micro-level channel through which managerial practice shapes retention in knowledge-intensive firms. Because employee knowledge is a core asset and turnover is costly, high-tech firms invest heavily in predicting and reducing attrition.[24] The findings suggest that improving feedback practices, and in particular fostering a positive feedback environment, may help strengthen retention and preserve firm-specific human capital.

Spillover. Feedback also affects developers' interactions with coworkers and their work in other teams (Figure 9c). Developers who receive positive feedback subsequently send a higher share of positive messages to coworkers, suggesting that positive tone improves the communication environment within the team. They also increase the quality of code they submit to *other* teams by 5.8%. These patterns are consistent with positive feedback raising workers' confidence in their ability and their motivation to contribute beyond the original code tasks. Constructive feedback, by contrast, lowers code quality in other teams, consistent with the rewrite tradeoff crowding out attention to work elsewhere. Since code quantity in other teams does not change (Figure G10), these effects reflect spillovers in code quality rather than a relabeling of activity across teams.

[24] Hoffman and Tadelis (2021) measure management skill with six survey items, two of which concern feedback directly: communicates clear expectations" and provides continuous coaching."

5.5 Heterogeneous Effects

The average effects mask meaningful heterogeneity. I examine which developers and teams are most affected by splitting the sample by demographics, ability, prior exposure to feedback, tenure, team structure, and the reviewer–developer match. Two patterns are most informative. First, feedback effects are stronger in more hierarchical teams, where reviewer feedback may carry greater evaluative weight. Second, positive feedback has larger effects on code quality after repeated exposure, consistent with appreciation accumulating over time. Because these analyses involve many subgroup comparisons, I interpret the patterns as suggestive rather than definitive.

By Race. Figure G1 compares Asian and White developers, the two largest racial groups in the sample (Appendix A.5). The estimated effects of most feedback types are similar across the two groups, but responses to toxic feedback differ. Asian developers increase subsequent code output after receiving toxic feedback, whereas White developers reduce output. One possible interpretation is that developers differ in whether they view person-directed toxicity, which may vary with norms surrounding directness, criticism, and authority in workplace and educational settings (Kim et al., 2013; Gelfand et al., 2011; Pinquart and Kauser, 2018).

By Gender. Figure G2 reports feedback effects by developer gender (Appendix A.4). Male and female developers respond similarly to positive and constructive feedback, but differ on toxic feedback: males reduce code quantity, while females slightly increase it, though this is not statistically significant. With women only 12% of the sample, this subgroup is likely selected.

By First vs. Repeat Exposure. Figure G3 compares developers' responses to their first exposure to each feedback type with later exposures. Most patterns are similar, suggesting developers do not quickly adapt. The exception is positive feedback: quality gains appear mainly after repeated exposures, consistent with developers internalizing appreciation over time.

By Developer Ability. Figure G4 splits developers by a proxy for ability: whether their pre-feedback code quality is above or below the median. Toxic feedback reduces code quality for both groups, suggesting that its negative effect is not concentrated among lower-ability developers. Positive feedback has larger benefits for developers with above-median prior quality. This pattern is consistent with positive feedback reinforcing developers' confidence when they have already performed well, thereby increasing their motivation to maintain high-quality work. Constructive feedback, by contrast, tends to reduce output for developers with below-median prior quality, suggesting that detailed feedback may impose larger adjustment costs on developers who are less prepared to incorporate it.

By New Developers vs. Incumbents. Figure G5 compares developers in their first two months on a team with longer-tenured incumbents. While incumbents are a selected group who have re-

mained on the team, the key distinction is between developers who are still learning the team's norms and those who are already integrated. Toxic feedback is especially damaging for incumbents, whereas positive feedback most strongly increases their code quality. This contrast suggests that feedback becomes more consequential once developers are integrated into the team.

By Team Hierarchy. Figure G6 examines effects by team hierarchy score. In more hierarchical teams, toxic feedback reduces future code quality more and positive feedback improves it more, perhaps because appreciation from high-status reviewers carries stronger motivational weight.

By Demographic Matching. Figures G7 and G8 show how the effects vary with reviewer–developer demographic similarity. The effect of toxic feedback is estimated most precisely and is significantly negative for male–male and White–White pairs. The point estimate for Asian–Asian pairs is similar in magnitude but less precise because the cell is much smaller. Estimates for cross-group pairs are also imprecise. These patterns are consistent with demographic similarity strengthening the response to negative feedback, but the estimates do not establish this mechanism.

5.6 Threats to the Exclusion Restriction

Interpreting the IV estimates as the effect of a specific feedback type requires that each type affect outcomes only through its own channel. Three concerns threaten this exclusion restriction. First, feedback is multidimensional, and its dimensions are correlated at the message level (for example, toxic feedback is rarely positive), so an instrument for one dimension may partly capture the effect of another. Second, correlation also arises at the reviewer level: a reviewer who gives more toxic feedback may also give less positive or less constructive feedback, so the toxicity instrument may reflect the reviewer's style in the other dimensions rather than toxicity itself. Third, reviewer characteristics such as gender, seniority, engagement, or strictness may correlate with the feedback they give, so the estimates could reflect these traits rather than the feedback. I address each concern in turn.

5.6.1 Jointly Instrumenting Other Feedback Dimensions

A single feedback message can be toxic, non-positive, and non-constructive at once, so a reviewer's toxicity instrument may also shift the chance of receiving positive or constructive feedback. If it does, the estimated effect of toxicity absorbs the influence of encouragement or information. To separate the three channels, I instrument them jointly, extending the baseline model in equations (1) and (2):

$$Y_{im,[t+1,t+4]} = \beta^{\text{Toxic}} I^{\text{Toxic}}_{im,t} + \beta^{\text{P}} I^{\text{P}}_{im,t} + \beta^{\text{C}} I^{\text{C}}_{im,t} + \Gamma_t X_{i,t} + \gamma_{m,\text{year}(t)} + \varepsilon_{im,t}, \tag{3}$$

$$I^{\text{Toxic}}_{im,t} = \phi^{\text{Toxic}} Z^{\text{Toxic}}_{i,t} + \delta^{\text{Toxic}} X_{i,t} + \gamma^{\text{Toxic}}_{m,\text{year}(t)} + \nu^{\text{Toxic}}_{im,t}, \tag{4}$$

$$I^{\mathrm{P}}_{im,t} = \phi^{\mathrm{P}} Z^{\mathrm{P}}_{i,t} + \delta^{\mathrm{P}} X_{i,t} + \gamma^{\mathrm{P}}_{m,\mathrm{year}(t)} + \nu^{\mathrm{P}}_{im,t}, \tag{5}$$

$$I^{\mathrm{C}}_{im,t} = \phi^{\mathrm{C}} Z^{\mathrm{C}}_{i,t} + \delta^{\mathrm{C}} X_{i,t} + \gamma^{\mathrm{C}}_{m,\mathrm{year}(t)} + \nu^{\mathrm{C}}_{im,t}. \tag{6}$$

Here $I^{\mathrm{Toxic}}_{im,t}$, $I^{\mathrm{P}}_{im,t}$, and $I^{\mathrm{C}}_{im,t}$ equal 1 if developer i received toxic, positive, or constructive feedback in team m during week t, and 0 otherwise; each is instrumented by the corresponding leave-developer-out reviewer style $Z^{\theta}_{i,t}$. With all three channels instrumented simultaneously, β^{Toxic} isolates the effect of toxicity holding positive and constructive feedback fixed. Figure D2 reports the results, which are nearly identical to the baseline in Table 5.

5.6.2 Controlling for Reviewer Feedback Styles in Other Feedback Dimensions

The joint system above addresses correlation among the three feedback *treatments*. The second concern is different: it concerns correlation in the *reviewer's style*. A reviewer who gives more toxic feedback may also give less positive or less constructive feedback, so the toxicity instrument may proxy for the reviewer's tendencies in the other dimensions rather than for toxicity itself.

Rather than instrumenting those other dimensions as additional endogenous treatments, I hold the reviewer's style in them fixed. I keep only the focal feedback type instrumented and add the non-focal reviewer instruments, $Z^{\mathrm{P}} * i, t$ and $Z^{\mathrm{C}} * i, t$, as controls in both stages. For example, the toxicity specification controls for the assigned reviewer's positivity and constructiveness tendencies, so identification comes from variation in reviewer toxicity among reviewers with similar styles in the other dimensions. Figure D3 shows that the estimates are again nearly identical to the baseline.

5.6.3 Controlling for Reviewer Observable Characteristics

A third concern is that feedback style correlates with reviewer traits that independently affect outcomes. For example, if more senior or male reviewers tend to give more toxic feedback, the estimated effect of toxicity could reflect reviewer seniority or gender rather than the feedback itself. I therefore add standardized reviewer controls to the baseline specification: gender, race, seniority, measured by the first year of GitHub activity, number of followers, and recent reviewing activity. As Figure D4 shows, the estimates remain stable. The results therefore do not appear to be driven by observable characteristics of the reviewer who delivers the feedback.

Reviewers may also differ in how they conduct reviews. This channel does not enter the outcome mechanically because I exclude the reviewed case when measuring subsequent productivity. The reviewer's approval decision and any revisions made to that case therefore do not directly affect the outcome measure.

5.7 Unobserved In-Person Feedback

The analysis observes only feedback delivered on GitHub, yet reviewers and developers on the same team may also interact in person, and the direction of the resulting bias is ambiguous. Con-

sider toxic feedback. If reviewers who are toxic online are also toxic in person, the on-platform estimate $\hat{\beta}_t^{\text{online}}$ overstates the online-specific effect, since it absorbs both channels. If instead reviewers temper their online feedback and redirect toxicity to in-person exchanges, the on-platform estimate understates it. The same logic applies to positive and constructive feedback: whether in-person interaction reinforces or substitutes for online feedback determines whether the measured effect is too large or too small.[25]

Co-located vs. Multi-located Teams. In-person feedback requires physical proximity, so offline feedback is more likely when team members share a location. I compare co-located teams, whose members are more likely to interact in person, with multi-located teams, for which platform feedback is more likely to capture most of the feedback. This comparison helps assess whether unobserved offline feedback attenuates the estimated effects. Because co-location is a team-level characteristic, its main effect is absorbed by the team-by-year fixed effects. The interaction coefficient η_t^θ is identified by comparing the relationship between feedback and outcomes across co-located and multi-located teams, using within-team-year variation in feedback. It is not identified by teams changing co-location status. A causal interpretation therefore requires that the two types of teams do not differ in other characteristics that also affect how developers respond to feedback.

$$
\begin{aligned}
Y_{im,[t+1,t+4]} &= \beta_t^\theta \, I_{im,t}^\theta + \eta_t^\theta \left(I_{im,t}^\theta \times C_{im}\right) + \mathbf{X}_{i,t}' \Gamma_t + \gamma_{m,\text{year}(t)} + \varepsilon_{im,t} \\
I_{im,t}^\theta &= \pi_t^\theta \, Z_{i,t}^\theta + \Gamma_t^I \mathbf{X}_{i,t} + \gamma_{m,\text{year}(t)} + u_{im,t}^I \\
\left(I_{im,t}^\theta \times C_{im}\right) &= \lambda_t^\theta \left(Z_{i,t}^\theta \times C_{im}\right) + \Gamma_t^{IM} \mathbf{X}_{i,t} + \gamma_{m,\text{year}(t)} + u_{im,t}^{IC}.
\end{aligned}
$$

I classify a team as co-located if all members are based in the same country (28.17% of teams). This measure overstates true physical proximity because members of the same country may still be geographically distant. The measured co-located group is therefore an upper bound on the set of teams with meaningful in-person interaction. Any resulting measurement error attenuates the estimated interaction toward zero. Consequently, a null interaction should be interpreted as weak rather than conclusive evidence against an offline channel. I report this exercise as a bound, since a sharper same-city classification is not feasible given that GitHub profile locations are typically available only at the country level.

[26] Table G1 reports the interaction estimates η^θ and nearly all are statistically insignificant.

[25] Two features of the setting limit the scope for unobserved offline feedback. Code review is conducted on the platform by design: reviewers record their approval or change requests through GitHub to merge code, so the substantive feedback on a submission is captured on-platform. More broadly, developers and reviewers tend to consolidate work-related communication within a single platform rather than switching channels.

[26] Location is inferred from developers' self-reported profiles; see Appendix A.6. I construct C_{im} from team members with an observed location: a team is co-located if all such members report the same country, and multi-located otherwise. Teams keep all their developer-team-week observations in the regressions. As a more flexible alternative, I identify the country with the largest share of located members and classify teams below the sample median share as multi-located, yielding a more balanced split. Figure G9 reports results from separate regressions in each subsample.

6 Feedback and Manager Quality

The analysis so far has examined how feedback affects developers. This section turns to what feedback reveals about the reviewers who provide it. Some reviewers raise developer productivity far more than others, and what is much less clear is what the better ones do differently. I ask how much of the variation in reviewer quality, measured by value added to developer productivity, can be explained by the feedback they give.

I measure manager quality by reviewer value-added (VA) to developer productivity, following the forecast-based approach of Chetty, Friedman, and Rockoff (2014). Reviewer VA captures whether developers produce more or better code after being assigned to a reviewer, relative to their own baseline and conditional on developer and case characteristics. I first estimate VA in the random reviewer sample, where reviewer assignment is as good as random, then extend the estimator to the full sample and validate it by comparing reviewers who appear in both settings; their VA rankings are strongly correlated (Appendix E.1). The differences are economically meaningful: a one standard deviation increase in reviewer VA corresponds to 0.20 standard deviations higher developer code quantity and 0.10 standard deviations higher code quality.

Because VA captures a reviewer's total effect on developer productivity, and feedback is only one channel through which that effect operates, the next exercise is a decomposition: what share of a reviewer's measured effect is traceable to observable features of their feedback, rather than to the many other things reviewers do? I train an XGBoost prediction model and compare four sets of predictors: reviewer demographics, feedback quantity, high-dimensional text embeddings, and the three feedback dimensions studied in this paper. Figure 10 reports results for VA measured from developer code quantity. Reviewer gender and race explain little, while feedback explains a substantial share. In the random reviewer sample, feedback quantity alone accounts for about 7% of the out-of-sample variation in reviewer VA; adding text embeddings raises this to about 22%, an increment of roughly 15 pp. Replacing the embeddings with the three feedback dimensions recovers 84% of that increment. The same pattern appears in the full sample, and in Appendix E.3 for VA measured from code quality.

These results show that feedback is not only a treatment that affects workers but also a measurable component of manager quality. The three dimensions used throughout the paper capture much of the variation in feedback that predicts reviewer value added.[27] Because firms already record the feedback managers give, these measures offer a practical way to identify the component of managerial quality that operates through feedback.

7 Conclusion

Economists have long known that managers matter for worker productivity. This paper identifies one specific, high-frequency thing they do that matters. Using over 230 million feedback messages

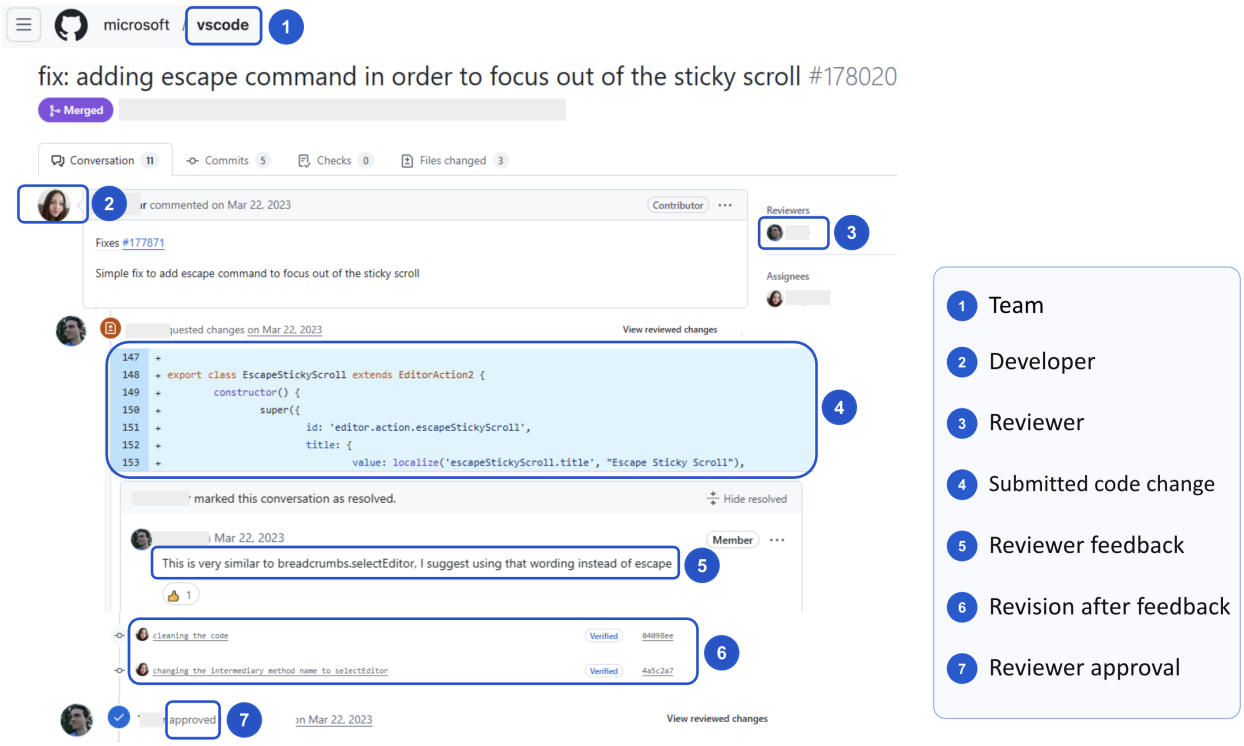
[27] Because reviewer value added is estimated from the same productivity outcomes that feedback affects, this exercise decomposes the reviewer effect rather than providing independent validation.

from software teams on GitHub, I exploit random reviewer assignment to show that the tone of feedback affects workers' productivity. Toxic feedback lowers developers' code quantity and quality in the following month, whereas respectful criticism does not. The productivity cost comes from attacking the person, not from criticizing the work. Positive feedback raises code quality, improving firm retention, and spills over to coworkers and other teams. Information content, by contrast, mainly reallocates effort toward the reviewed task and away from new work.

Two implications follow for firm management. First, tone is a tool managers can use to affect worker productivity. Unlike bonuses or promotions, it is not constrained by budgets or vacant positions. A manager who criticizes work respectfully incurs no apparent cost on the quantity of subsequent output, whereas a manager who crosses into toxicity reduces output. Second, feedback offers a way to evaluate manager quality. The way managers communicate explains a substantial share of the variation in reviewer value-added, suggesting that feedback style captures an important dimension of managerial effectiveness.

Methodologically, the paper shows that LLMs make workplace communication legible to economics at scale. I study three dimensions of feedback in one production setting central to the digital economy (Hoffmann et al., 2024), but the same approach extends to other records of what managers do, such as emails, meetings, and performance reviews, and to other dimensions of communication, such as formality and stylistic variation. This paper takes the feedback managers give as given. It does not ask why some reviewers are more toxic, positive, or constructive, or whether managers target feedback strategically. The large heterogeneity in effects across workers makes adaptive or targeted feedback a natural next question. Future work could also study upward feedback, how workers' responses and evaluations shape managerial behavior, and examine which other forms of workplace communication make teams more productive.

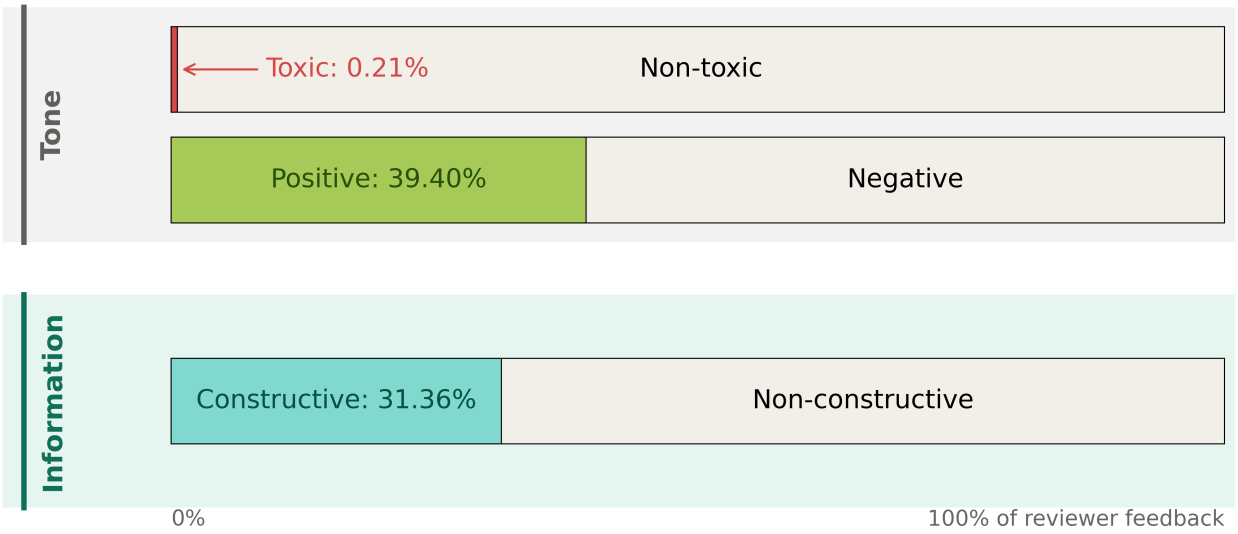
Figure 1: Example of GitHub Code Review



Notes: This figure shows an example of a code review in the Microsoft VS Code team (a "repository," in GitHub terminology). A developer submits a code change (a "pull request") to the team's codebase, and a reviewer checks the code and gives feedback on the submission. The developer then revises the code in response. The reviewer decides whether to approve the change. In this example, the revised change is approved.

Figure 2: Classification of Reviewer Feedback by Tone and Information

(a) Share of Feedback by Tone and Information

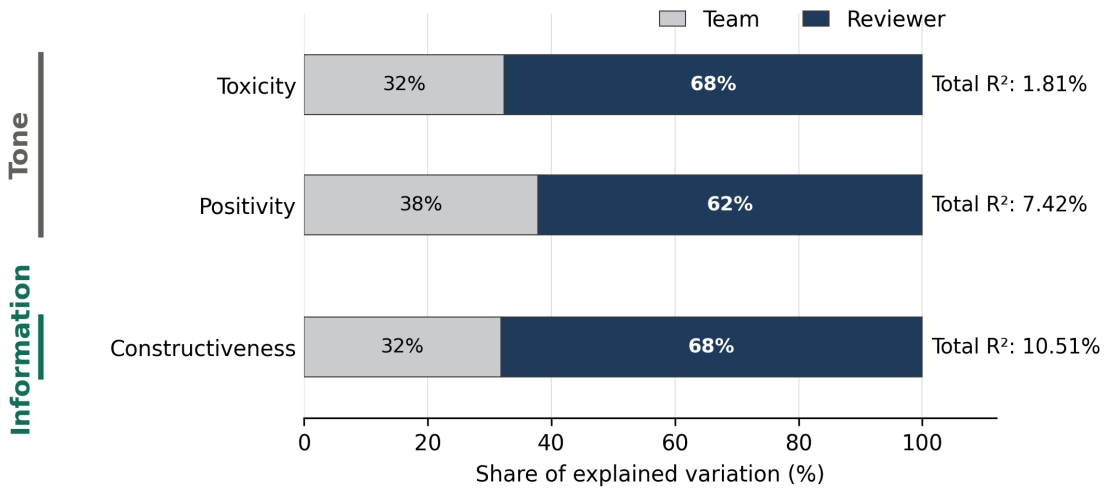


(b) Joint Distribution of Feedback Tone

		Toxicity	
		Toxic	**Non-toxic**
Positivity	**Positive**	Toxic Positive 0.02%	Non-toxic Positive 39.38%
	Negative	Toxic Negative 0.19%	Non-toxic Negative 60.41%

Notes: The figure classifies reviewer feedback along two dimensions: tone, which comprises toxicity and positivity, and information, measured by constructiveness. Toxicity captures intentional harm, positivity a supportive and encouraging tone, and constructiveness, whether feedback provides specific, actionable information about the submitted code. **Panel (a)** reports the share of feedback in each dimension in the random reviewer sample. Each bar represents 100% of reviewer feedback; the shaded portion is the share in the named dimension, and the remainder is its complement. Table 1 reports the corresponding shares in the full sample. **Panel (b)** reports the joint distribution of feedback tone. Columns classify feedback by toxicity (toxic or non-toxic) and rows by positivity (positive or negative). The four cells sum to 100%. Table 3 reports the full joint distribution across all three dimensions, together with examples for each type.

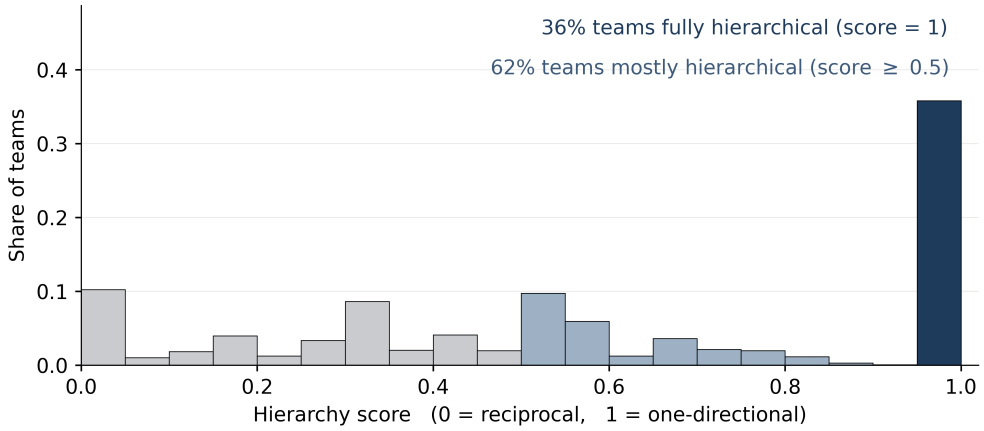
Figure 3: Sources of Feedback Variation



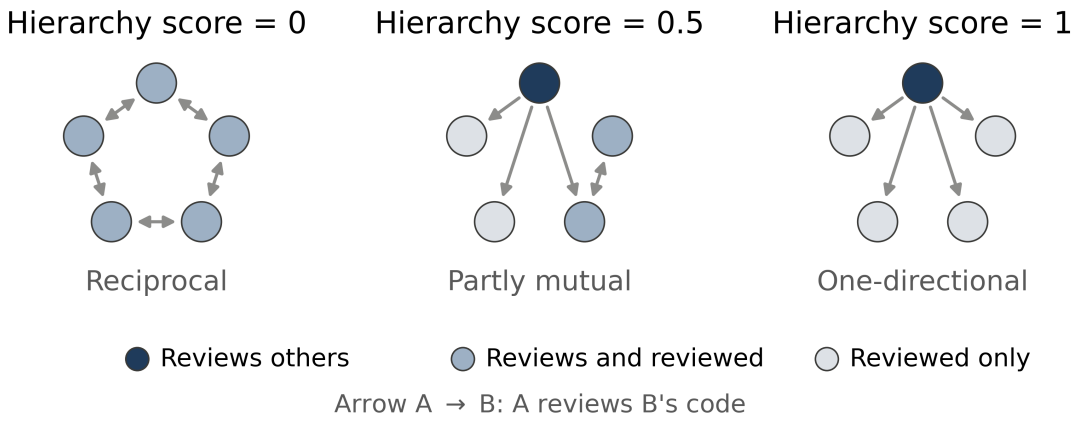
Notes: This figure decomposes the explained variation in feedback tone and information. Tone is measured by toxicity and positivity. Information is measured by constructiveness. The unit of observation is a developer–team–week, and the dependent variable is the share of feedback a developer receives in that team-week that is toxic, positive, or constructive. Bars report the share of explained variation accounted for by team and reviewer fixed effects. Total R^2 values are reported to the right of each bar. Additional factors, including developer fixed effects and case characteristics such as whether the case involves an enhancement or a bug, each add less than 0.01 percentage points and are not shown. The sample is the random reviewer subsample.

Figure 4: Feedback Hierarchy within Team

(a) Distribution of Team Hierarchy Scores



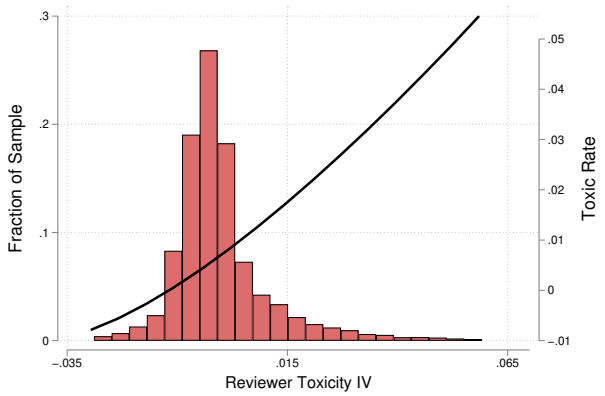
(b) Example Feedback Networks



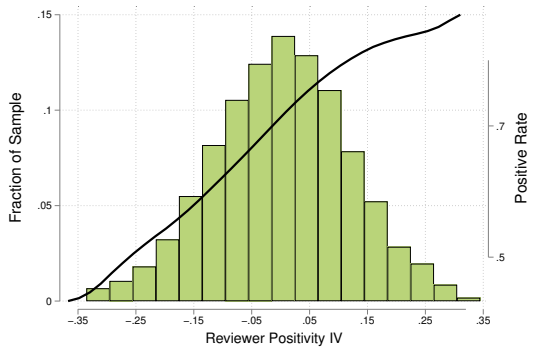
Notes: This figure summarizes how feedback flows within teams. For each team, the hierarchy score is $1 - 2R/E$, where E is the number of directed reviewer–developer pairs and R is the number of reciprocated reviewer-developer pairs. It ranges from 0, when members review each other symmetrically (reciprocal), to 1, when feedback is one-directional, with some members reviewing others but rarely receiving feedback in return. Higher scores indicate a more hierarchical review structure. Teams with a single case or a single member are excluded. **Panel (a)** plots the distribution of hierarchy scores across teams. The navy bar marks fully hierarchical teams (score $= 1$) and the shaded region mostly hierarchical teams (score ≥ 0.5). The concentration near 1 is consistent with feedback that resembles manager-like evaluation rather than symmetric peer review. **Panel (b)** shows three stylized five-member teams, holding the layout fixed and varying only the direction of feedback. Node color denotes a member's role: reviews only, reviews and is reviewed, or reviewed only. A directed edge A $\rightarrow$ B means A reviews B's code. Figure C5 shows the distribution for teams with exactly five members and for the random reviewer sample.

Figure 5: Distribution of Reviewer Instruments and First Stage

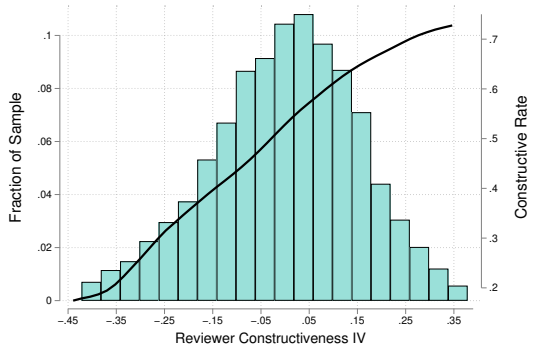
(a) Toxicity



(b) Positivity



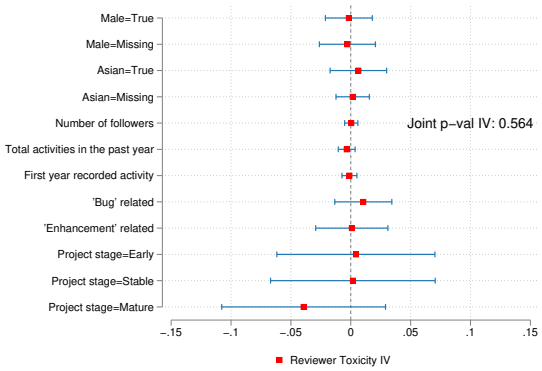
(c) Constructiveness



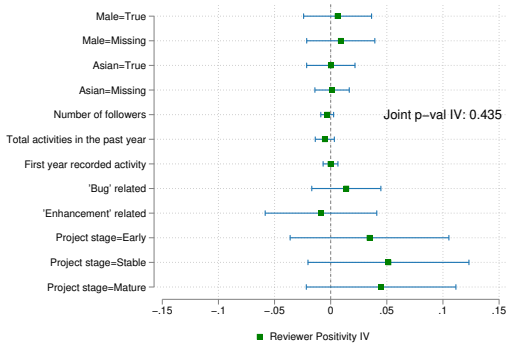
Notes: Each panel plots the distribution of reviewer instruments and the nonparametric first-stage relationship between the instrument and the probability that the focal case receives the corresponding feedback type. The instrument is the reviewer's leave-developer-out average feedback share, residualized by team-by-year fixed effects. I include teams with at least 2 reviewers and reviewers who have handled at least 10 cases, leaving 480,697 cases and 10,873 reviewers, with an average of 44.2 cases per reviewer. The bars show the distribution of reviewer instrument values, and the solid line shows the kernel-weighted local polynomial fit of the first stage. Panels are ordered by the feedback classification: toxicity and positivity measure tone, and constructiveness measures information content. The sample is the random reviewer sample, excluding the top and bottom 1% of instrument values.

Figure 6: Balance of Reviewer Instruments

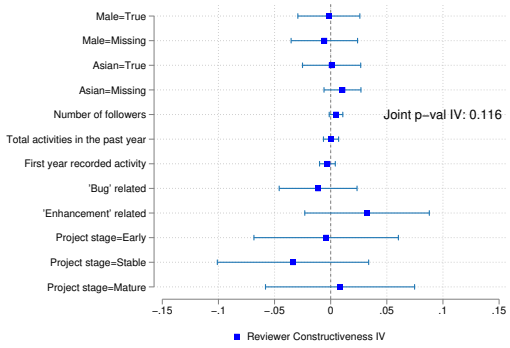
(a) Toxicity



(b) Positivity

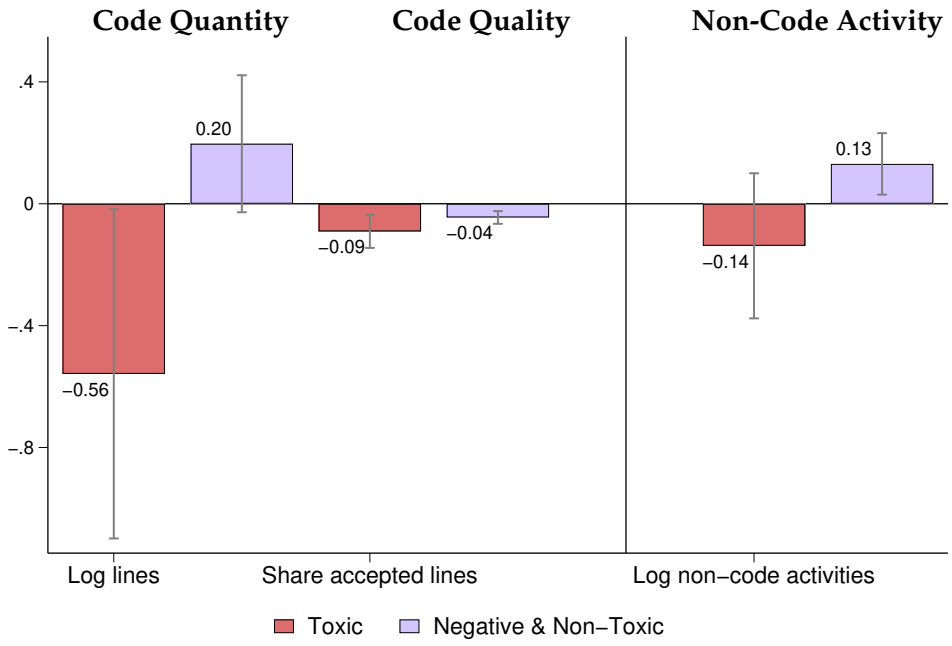


(c) Constructiveness



Notes: Each panel reports OLS coefficients from regressions of the reviewer instrument on standardized developer and case characteristics. The instrument is the reviewer's leave-developer-out average feedback share, residualized by team-by-year fixed effects. Coefficients are shown with 95 percent confidence intervals. The p-values shown in the panels are from joint significance tests of all characteristics. Standard errors are clustered at the reviewer level. The sample is the random reviewer sample. Appendix A.4 and Appendix A.10 define the variables.

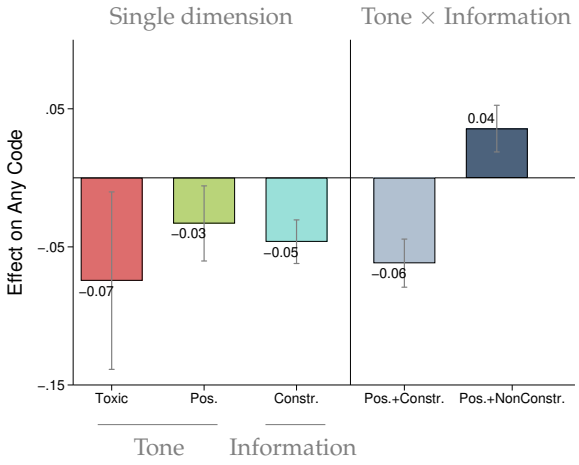
Figure 7: Effects of Toxic versus Respectful Critical Feedback on Developer



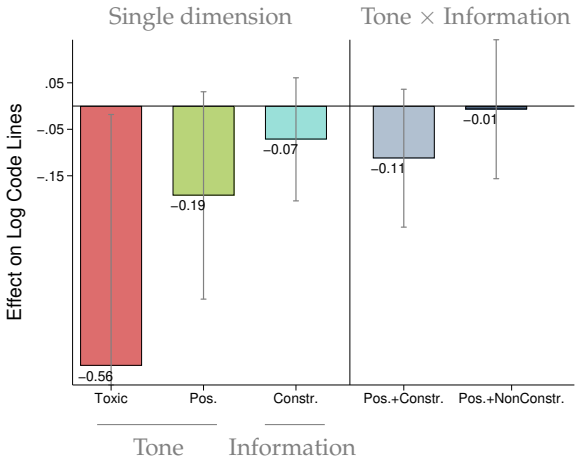
Notes: This figure plots 2SLS estimates of the effect of toxic feedback (red) and respectful criticism (purple) on developer productivity, where respectful criticism is feedback coded as negative but non-toxic. Each bar is from a separate regression of the outcome on an indicator for receiving the relevant type of feedback. Outcomes are measured on activity within the team during weeks 1–4 after a review: log code lines (conditional on coding), the share of new lines accepted, and log non-code activity such as answering user questions. All regressions include team-by-year fixed effects and controls for the developer's lagged activity and message deciles. Lines indicate 90% confidence intervals. Standard errors are clustered at the reviewer level.

Figure 8: Joint Effects of Feedback Tone and Information on Developer Productivity

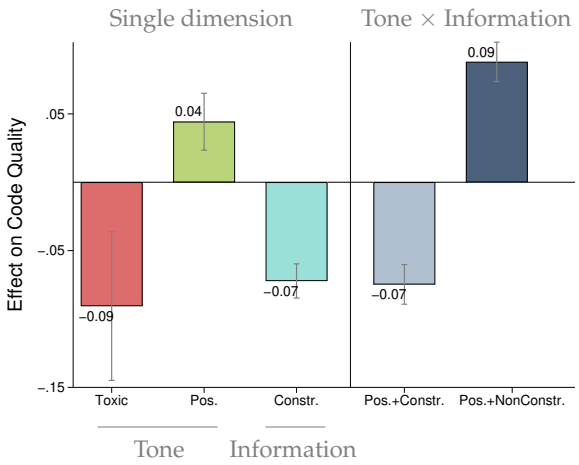
(a) Code Quantity (Extensive)



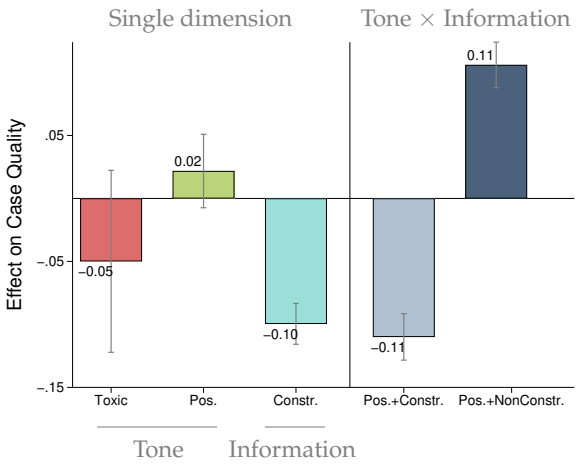
(b) Code Quantity (Intensive)



(c) Code Quality



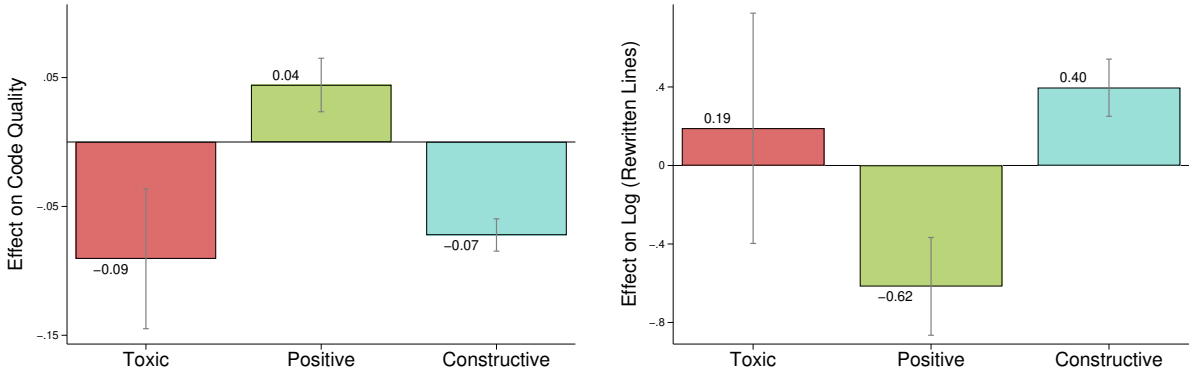
(d) Case Quality



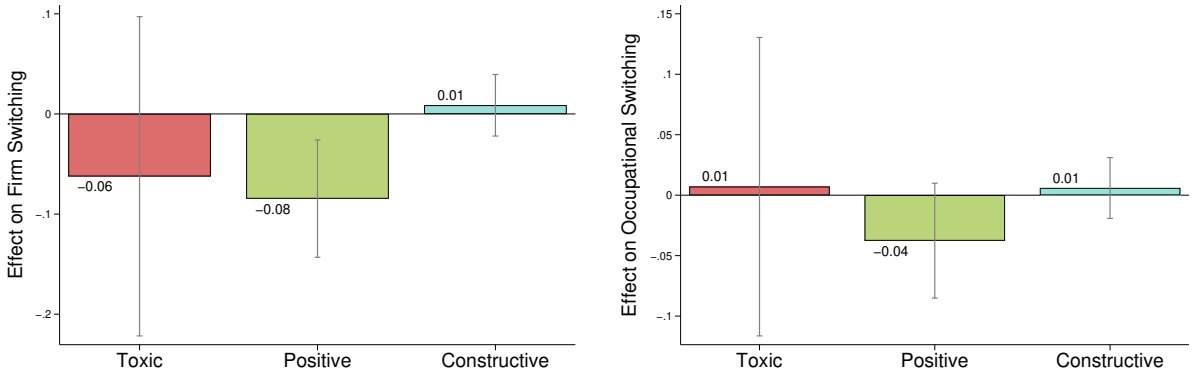
Notes: This figure reports 2SLS estimates of the effects of single and joint feedback types on developer productivity within a team in the next 1–4 weeks following feedback. The first three bars correspond to single feedback types: toxic, positive, and constructive; the fourth is non-toxic & negative; the fifth, positive & constructive; and the sixth, positive & non-constructive. Joint feedback types with very small shares are omitted, based on the share reported in Table 3. **Panels (a)** and **(b)** report *Code Quantity*, the extensive margin (Ever) and the intensive margin (Log). **Panel (c)** reports *Code Quality*, the share of lines of new code accepted, and **Panel (d)** reports *Case Quality*, the share of new cases merged into the team's codebase. Feedback exposure is instrumented using reviewer-level feedback tendencies within the random assignment sample. Lines indicate 90% confidence intervals, which are trimmed at –0.6 and 0.3 in the graph to focus on the main range of estimated effects. Standard errors are clustered at the reviewer level.

Figure 9: Effects of Feedback on Developer Outcomes

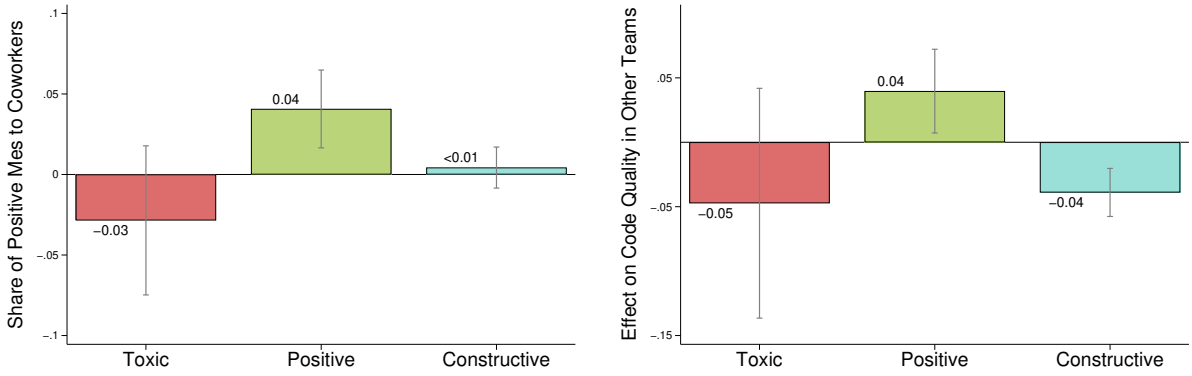
(a) Tradeoff (GitHub): New Code Quality and Old Code Rewritten



(b) Retention (LinkedIn): Firm and Occupational Switch

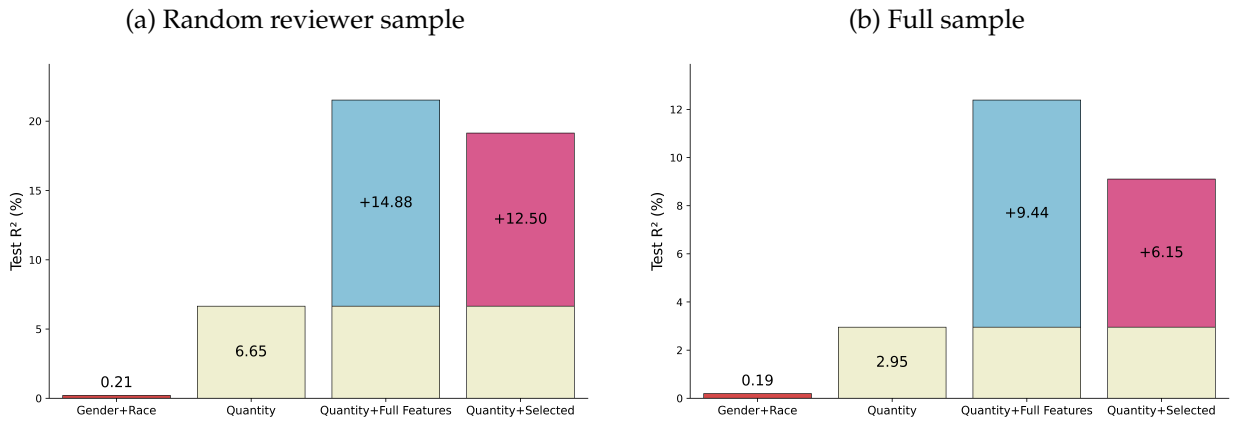


(c) Spillover Effects (GitHub): Coworkers and Other Teams



Notes: This figure reports 2SLS estimates of the effects of different feedback types on developer outcomes after receiving feedback. Panel (a) shows the tradeoff between new code quality and old code rewritten. New code quality is measured by the code-level acceptance outcome, reported in column (3) under "Code Quality" in Table 5. Panel (a2) reports the effect of rewritten code measures the log of code revisions made to reviewed code. Panel (b) reports the effects of feedback on developer career mobility over the next year using LinkedIn data, measured by firm switches and occupational switches. Panel (c) presents spillover effects of feedback on other developers within and across teams in the next 1-4 weeks on GitHub. Panel (c1) shows how receiving feedback in the focal team affects the share of positive messages that developers send to coworkers. Panel (c2) reports effects on the quality of code developers submit to other teams. Feedback exposure is instrumented using reviewer-level feedback tendencies within the random assignment sample. Lines indicate 90% confidence intervals. Standard errors are clustered at the reviewer level.

Figure 10: Predicting Reviewer Value-Added from Feedback: Developer Code Quantity



Notes: This figure reports out-of-sample R^2 from models predicting reviewer value-added (VA), measured by the reviewer's effect on developer code quantity. **Panel (a)** uses the random reviewer sample; **Panel (b)** uses the full sample. Reviewer VA is estimated using the forecast-based estimator of Chetty, Friedman, and Rockoff (2014). "Demographics" includes reviewer gender and race. "Quantity" is the number of feedback messages a reviewer sends. "Quantity + Full Text" adds 768-dimensional text embeddings. "Quantity + Selected Feedback" adds the three feedback dimensions used in the paper: toxicity, positivity, and constructiveness. Appendix E.3 reports model details and analogous results using developer code quality as the VA outcome.

Table 1: Descriptive Statistics

Description	Full Sample	Random Reviewer Sample
Panel A: Teams and Cases		
Team (#)	1,774,184	3,457
Case (#)	56,637,966	4,228,687
Case with reviewers (%)	17.11	33.42
Reviewers (#)	364,889	35,072
Avg. case per team (#)	22	1223
Avg. team size (#)	1.03	19.42
Panel B: All Messages		
Message (#)	231,293,881	24,453,479
Avg. message per case (#)	4.08	5.78
Avg. message per team (#)	130	7,074
Panel C: Reviewer Feedback		
Message sent by reviewers (%)	36.33	32.60
Toxic (%)	0.21	0.21
Positive (%)	41.87	39.40
Constructive (%)	34.65	31.36
Panel D: Team Feedback Structure		
P(B reviews A | A reviews B) (%)	52.17	23.67
Same reviewer on developer's next case (%)	10.91	7.32

Notes: This table reports summary statistics for all teams (Full Sample) and for teams that adopted random reviewer assignment (Random Reviewer Sample) between 2017 and 2023. Team size is measured as the average number of members from 2017 to 2023. Panel A reports summary statistics about team characteristics. Panel B includes messages sent by both team members and external users. Panel C focuses only on feedback messages sent by reviewers, which are the focus of this paper. For the Random Reviewer Sample, all reviewer feedback messages are used. For the Full Sample, toxic and positive feedback are based on all reviewer feedback messages, while constructive feedback is based on a random subsample due to budget constraints.

Table 2: Developer–Team Weekly Productivity Summary Statistics

	N	Mean	Std. Dev	P75	P95	Max
Focal Team						
Code Lines	12,216,340	1,119.21	54,898.64	3	537	41,436,660
Rewritten Lines	12,216,340	507.81	39,439.85	0	114	29,285,783
Non-Code Activities	12,216,340	6.36	39.89	5	26	62,784
Cases	12,216,340	3.18	11.83	3	13	16,722
Case Quality	3,607,865	0.63	0.42	1	1	1
Code Quality	4,095,714	0.74	0.38	1	1	1
Other Teams						
Code Lines	12,216,340	2,812.08	199,277.21	0	617	78,347,840
Rewritten Lines	12,216,340	861.94	56,482.51	0	97	29,285,786
Non-Code Activities	12,216,340	549.94	2,938.83	43	1,819	62,801
Cases	12,216,340	85.41	457.56	16	97	16,741
Case Quality	2,054,565	0.67	0.34	1	1	1
Code Quality	2,467,433	0.70	0.40	1	1	1

Notes: This table shows the summary statistics of productivity measures for all developers in the random reviewer sample from 2017 to 2023. Each observation is a developer-team-week. "Focal Team" refers to the team in the current row; "Other Teams" aggregates activity from all remaining teams during the same week. Winsorization at the 95th percentile is applied to all continuous variables to reduce the influence of extreme outliers, which are common in developer productivity data (e.g., unusually large code commits or bursts of activity). Statistics are computed on the raw data. In the regression analysis, all continuous outcomes are winsorized at the 95th percentile to limit the influence of extreme values, which are common in developer productivity data (for example, unusually large commits). Bounded outcomes — case quality and code quality are unaffected by winsorization. See Appendix A.8 and Appendix A.9 for variable definitions.

Table 3: Distribution and Examples of Reviewer Feedback Types

Panel A: Distribution of Feedback Types

Constructive

	Positive	**Negative**
Toxic	0.000	0.01
Non-Toxic	9.43	21.91

Non-Constructive

	Positive	**Negative**
Toxic	0.02	0.17
Non-Toxic	29.95	38.50

Panel B: Examples of Feedback Types

Toxic	**Positive**	**Constructive**	**Feedback Message**	**Share (%)**
1	1	0	Sounds good, can't wait to restaple my ass on.	0.02
1	0	1	You should just keep the file name, not the full file path. This is Pantherinternal and doesn't provide value...	0.01
1	0	0	You will have to answer for it and you are talking nonsense.	0.17
0	1	1	Docs look good overall. I think it would also be helpful to add a few more screenshots showing the overall page...	9.43
0	1	0	Looks good to me.	29.95
0	0	1	Still feels redundant. An angle should just be a union of float and FreeParameter. Once the parameter is bound...	21.91
0	0	0	We need a way to detect the cluster domain.	38.50

Notes: This table shows the distribution of reviewer feedback by toxicity, positivity, and constructiveness in the random reviewer sample. Panel A reports the share of feedback in each category. Entries are percentages and sum to 100. Panel B reports representative examples from each feedback type. The share column reports the percentage of feedback in the corresponding category.

Table 4: First-Stage Estimates by Feedback Type

	(1)	(2)	(3)
Panel A: Toxic Feedback			
Reviewer Toxicity	0.708***	0.703***	0.703***
	(0.013)	(0.013)	(0.013)
F-Stat.	2931.913	2866.244	2868.936
Dependent mean	0.008	0.008	0.008
Panel B: Positive Feedback			
Reviewer Positivity	0.843***	0.820***	0.820***
	(0.005)	(0.005)	(0.005)
F-Stat.	34335.891	28057.043	27450.902
Dependent mean	0.706	0.706	0.706
Panel C: Constructive Feedback			
Reviewer Constructiveness	0.875***	0.864***	0.863***
	(0.004)	(0.004)	(0.004)
F-Stat.	61420.629	60183.796	60733.265
Dependent mean	0.518	0.518	0.518
Team FE	✓		
Team × Year FE		✓	✓
Baseline controls			✓
N	1,179,525	1,179,525	1,179,525

Notes: Each panel reports a first-stage regression where the dependent variable is an indicator for receiving a specific type of feedback. Baseline controls include developer gender, race, GitHub activity in the year before submission, and the first year of GitHub activity. Case characteristics account for issue labels such as Bug and Enhancement, and project stage is defined based on the release version of the team product. Appendix A.4 provides details on the gender and race variables, and Appendix A.10 describes the construction of project stage. Standard errors are clustered at the reviewer level. *$p < .10$, ** $p < .05$, *** $p < .01$.

Table 5: Effect of Reviewer Feedback on Developer Productivity Within a Team

	Code Quantity		Code Quality		Non-Code Quantity	
	Ever	Log	Code	Case	Ever	Log
Panel A: Tone						
Toxic Feedback						
OLS	-0.013***	0.065***	0.001	-0.007***	0.004***	0.080***
	(0.003)	(0.015)	(0.002)	(0.002)	(0.001)	(0.008)
RF	-0.057*	-0.421*	-0.068***	-0.038	-0.014	-0.106
	(0.030)	(0.250)	(0.025)	(0.033)	(0.014)	(0.111)
2SLS	-0.074*	-0.558*	-0.091***	-0.050	-0.019	-0.138
	(0.039)	(0.328)	(0.033)	(0.044)	(0.019)	(0.145)
Positive Feedback						
OLS	0.015***	0.178***	-0.003**	-0.005***	0.010***	0.137***
	(0.002)	(0.008)	(0.001)	(0.001)	(0.001)	(0.004)
RF	-0.009**	-0.050	0.012***	0.006	-0.003	-0.035**
	(0.005)	(0.035)	(0.003)	(0.005)	(0.002)	(0.016)
2SLS	-0.033**	-0.192	0.044***	0.022	-0.011	-0.130**
	(0.017)	(0.136)	(0.013)	(0.018)	(0.008)	(0.061)
Panel B: Information Content						
Constructive Feedback						
OLS	0.017***	0.184***	-0.015***	-0.019***	0.009***	0.122***
	(0.001)	(0.007)	(0.001)	(0.001)	(0.001)	(0.003)
RF	-0.017***	-0.025	-0.026***	-0.035***	-0.003*	-0.008
	(0.004)	(0.028)	(0.003)	(0.003)	(0.002)	(0.013)
2SLS	-0.046***	-0.072	-0.072***	-0.100***	-0.009**	-0.023
	(0.010)	(0.080)	(0.008)	(0.010)	(0.005)	(0.035)
Dependent mean	0.846	5.018	0.693	0.545	0.957	2.450
N	1,176,459	995,139	1,028,793	998,141	1,176,459	1,125,874
Team × Year FE	✓	✓	✓	✓	✓	✓
Decile t–1 activity	✓	✓	✓	✓	✓	✓
Decile t–1 message	✓	✓	✓	✓	✓	✓

Notes: This table reports OLS, reduced-form (RF), and two-stage least squares (2SLS) estimates of receiving each feedback type on developer outcomes during weeks $t+1$ to $t+4$. Panel A reports the two tone dimensions, toxic and positive feedback; Panel B reports the information dimension, constructive feedback. Figure D2 reports estimates from the augmented IV system in equations (3)–(6), which includes all three feedback dimensions jointly and yields similar estimates. Outcomes are measured on new code written by the developer within the team during weeks 1–4 after receiving feedback. Within each outcome group, *Ever* is the extensive margin (whether the developer is active at all) and *Log* is the intensive margin (log code lines); for Code Quality, *Code* is the share of new lines accepted and *Case* is the share of cases merged. All statistics are reported for the sample used in the 2SLS estimation. All models include team-by-year fixed effects and controls for the developer's lagged activity and message deciles. The unit of observation is a developer-team-week feedback event in the random reviewer sample. Standard errors, in parentheses, are clustered at the reviewer level. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

References

Adams-Prassl, A., K. Huttunen, E. Nix, and N. Zhang (2024). Violence Against Women at Work. *The Quarterly Journal of Economics 139*(2), 937–991.

Adhvaryu, A., N. Kala, and A. Nyshadham (2022). Management and Shocks to Worker Productivity. *Journal of Political Economy 130*(1), 1–47.

Adhvaryu, A., A. Nyshadham, and J. Tamayo (2023). Managerial Quality and Productivity Dynamics. *The Review of Economic Studies 90*(4), 1569–1607.

Agan, A., J. L. Doleac, and A. Harvey (2023). Misdemeanor Prosecution. *The Quarterly Journal of Economics 138*(3), 1453–1505.

Andersen-Gott, M., G. Ghinea, and B. Bygstad (2012). Why Do Commercial Companies Contribute to Open Source Software? *International Journal of Information Management 32*(2), 106–117.

Angrist, J. and G. Imbens (1994). Identification and Estimation of Local Average Treatment Effects.

Angrist, J. D. and B. Frandsen (2022). Machine Labor. *Journal of Labor Economics 40*(S1), S97–S140.

Ashford, S. J. and L. L. Cummings (1983). Feedback as an Individual Resource: Personal Strategies of Creating Information. *Organizational Behavior and Human Performance 32*(3), 370–398.

Bandiera, O., I. Barankay, and I. Rasul (2007). Incentives for Managers and Inequality Among Workers: Evidence from a Firm-level Experiment. *The Quarterly Journal of Economics 122*(2), 729–773.

Bandiera, O., A. Prat, S. Hansen, and R. Sadun (2020). CEO Behavior and Firm Performance. *Journal of Political Economy 128*(4), 1325–1369.

Banihashem, S. K., O. Noroozi, S. Van Ginkel, L. P. Macfadyen, and H. J. Biemans (2022). A Systematic Review of the Role of Learning Analytics in Enhancing Feedback Practices in Higher Education. *Educational Research Review 37*, 100489.

Becker, G. S. (1964). Human Capital.

Bekker, P. A. (1994). Alternative Approximations to the Distributions of Instrumental Variable Estimators. *Econometrica*, 657–681.

Beknazar-Yuzbashev, G., R. Jiménez Durán, J. McCrosky, and M. Stalinski (2022). Toxic Content and User Engagement on Social Media: Evidence from a Field Experiment. *Available at SSRN 4307346*.

Bénabou, R. and J. Tirole (2002). Self-confidence and Personal Motivation. *The Quarterly Journal of Economics 117*(3), 871–915.

Bénabou, R. and J. Tirole (2003). Intrinsic and extrinsic motivation. *The review of economic studies 70*(3), 489–520.

Bertrand, M. and A. Schoar (2003). Managing with Style: The Effect of Managers on Firm Policies. *The Quarterly Journal of Economics 118*(4), 1169–1208.

Beuschlein, J. (2024). Designing Debt Restructuring: The Adverse Effects on Labor Market Outcomes.

Bhuller, M., G. B. Dahl, K. V. Løken, and M. Mogstad (2020). Incarceration, Recidivism, and

Employment. *Journal of Political Economy 128*(4), 1269–1324.

Bloom, N. and J. Van Reenen (2007). Measuring and Explaining Management Practices Across Firms and Countries. *The Quarterly Journal of Economics 122*(4), 1351–1408.

Brynjolfsson, E., D. Li, and L. Raymond (2025). Generative AI at Work. *The Quarterly Journal of Economics*, qjae044.

Chetty, R., J. N. Friedman, and J. E. Rockoff (2014). Measuring the Impacts of Teachers I: Evaluating Bias in Teacher Value-added Estimates. *American Economic Review 104*(9), 2593–2632.

Collis, M. R. and C. Van Effenterre (2025). Workplace Hostility.

Compte, O. and A. Postlewaite (2004). Confidence-enhanced Performance. *American Economic Review 94*(5), 1536–1557.

Dobbie, W., J. Goldin, and C. S. Yang (2018). The Effects of Pre-trial Detention on Conviction, Future Crime, and Employment: Evidence from Randomly Assigned Judges. *American Economic Review 108*(2), 201–240.

Dobbie, W., P. Goldsmith-Pinkham, and C. S. Yang (2017). Consumer Bankruptcy and Financial Health. *Review of Economics and Statistics 99*(5), 853–869.

Dobbie, W. and J. Song (2015). Debt Relief and Debtor Outcomes: Measuring the Effects of Consumer Bankruptcy Protection. *American Economic Review 105*(3), 1272–1311.

Ederer, F., P. Goldsmith-Pinkham, and K. Jensen (2024). Anonymity and Identity Online. *arXiv Preprint arXiv:2409.15948*.

El-Komboz, L. A. and M. Goldbeck (2024). Career Concerns as Public Good the Role of Signaling for Open Source Software Development.

Emanuel, N., E. Harrington, and A. Pallais (2025). The Power of Proximity to Coworkers.

Fattori, A., L. Neri, E. Aguglia, A. Bellomo, A. Bisogno, D. Camerino, B. Carpiniello, A. Cassin, G. Costa, P. De Fazio, et al. (2015). Estimating the Impact of Workplace Bullying: Humanistic and Economic Burden Among Workers with Chronic Medical Conditions. *BioMed Research International 2015*(1), 708908.

Folke, O. and J. Rickne (2022). Sexual Harassment and Gender Inequality in the Labor Market. *The Quarterly Journal of Economics 137*(4), 2163–2212.

Frandsen, B., L. Lefgren, and E. Leslie (2023). Judging Judge Fixed Effects. *American Economic Review 113*(1), 253–277.

Frederiksen, A., L. B. Kahn, and F. Lange (2020). Supervisors and Performance Management Systems. *Journal of Political Economy 128*(6), 2123–2187.

Freimane, M. (2024). Gender Bias, Feedback, and Productivity.

Gallup (2022). How Effective Feedback Fuels Performance. Retrieved April 19, 2025, from https://www.gallup.com/workplace/357764/fast-feedback-fuels-performance.aspx.

Gelfand, M. J., J. L. Raver, L. Nishii, L. M. Leslie, J. Lun, B. C. Lim, L. Duan, A. Almaliach, S. Ang, J. Arnadottir, et al. (2011). Differences Between Tight and Loose Cultures: A 33-nation Study. *Science 332*(6033), 1100–1104.

GitHub (2025). GitHub Enterprise. Accessed: August 27, 2025.

Haegele, I. (2022). Talent Hoarding in Organizations. *arXiv Preprint arXiv:2206.15098*.

Hartmann, J., M. Heitmann, C. Siebert, and C. Schamp (2023). More Than a Feeling: Accuracy and Application of Sentiment Analysis. *International Journal of Research in Marketing 40*(1), 75–87.

Hartvigsen, T., S. Gabriel, H. Palangi, M. Sap, D. Ray, and E. Kamar (2022). ToxiGen: A Large-Scale Machine-Generated Dataset for Implicit and Adversarial Hate Speech Detection. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*.

Hattie, J. and H. Timperley (2007). The Power of Feedback. *Review of Educational Research 77*(1), 81–112.

Heckman, J. J. and E. Vytlacil (2005). Structural Equations, Treatment Effects, and Econometric Policy Evaluation. *Econometrica 73*(3), 669–738.

Hoffman, M. and S. Tadelis (2021). People Management Skills, Employee Attrition, and Manager Rewards: An Empirical Analysis. *Journal of Political Economy 129*(1), 243–285.

Hoffmann, M., F. Nagle, and Y. Zhou (2024). The Value of Open Source Software. *Harvard Business School Strategy Unit Working Paper* (24-038).

Holmström, B. (1979). Moral Hazard and Observability. *The Bell Journal of Economics*, 74–91.

Holmström, B. (1982). Moral Hazard in Teams. *The Bell Journal of Economics*, 324–340.

Hull, P. (2017). Examiner Designs and First-stage F Statistics: A Caution. *Brown University Working Paper 5*.

Ichniowski, C., K. L. Shaw, and G. Prennushi (1997). The Effects of Human Resource Management Practices on Productivity.

Ilgen, D. R., C. D. Fisher, and M. S. Taylor (1979). Consequences of Individual Feedback on Behavior in Organizations. *Journal of Applied Psychology 64*(4), 349.

Jones, B. F. (2021). The Rise of Research Teams: Benefits and Costs in Economics. *Journal of Economic Perspectives 35*(2), 191–216.

Kim, S. Y., Y. Wang, D. Orozco-Lapray, Y. Shen, and M. Murtuza (2013). Does "Tiger Parenting" Exist? Parenting Profiles of Chinese Americans and Adolescent Developmental Outcomes. *Asian American Journal of Psychology 4*(1), 7.

Kluger, A. N. and A. DeNisi (1996). The Effects of Feedback Interventions on Performance: A Historical Review, a Meta-analysis, and a Preliminary Feedback Intervention Theory. *Psychological Bulletin 119*(2), 254.

Kolesár, M. (2013). Estimation in an Instrumental Variables Model with Treatment Effect Heterogeneity.

Kolesár, M., R. Chetty, J. Friedman, E. Glaeser, and G. W. Imbens (2015). Identification and Inference with Many Invalid Instruments. *Journal of Business & Economic Statistics 33*(4), 474–484.

Kolhatkar, V., N. Thain, J. Sorensen, L. Dixon, and M. Taboada (2020). Classifying Constructive Comments. *arXiv Preprint arXiv:2004.05476*.

Kőszegi, B., G. Loewenstein, and T. Murooka (2022). Fragile Self-esteem. *The Review of Economic Studies 89*(4), 2026–2060.

Laohaprapanon, S., G. Sood, and B. Naji (2017). Ethnicolr Algorithm.

Lazear, E. P., K. L. Shaw, and C. T. Stanton (2015). The Value of Bosses. *Journal of Labor Economics 33*(4), 823–861.

Lee, D. S., J. McCrary, M. J. Moreira, and J. Porter (2022). Valid T-ratio Inference for IV. *American Economic Review 112*(10), 3260–3290.

Liu, Y., M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov (2019). Roberta: A Robustly Optimized Bert Pretraining Approach. *arXiv Preprint arXiv:1907.11692*.

Marschak, J. and R. Radner (1958). Economic Theory of Teams. Chapter 5.

McIntosh, S., Y. Kamei, B. Adams, and A. E. Hassan (2016). An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering 21*, 2146–2189.

McKinsey (2024). Feedback Culture: Great Learning Design as a Bridge to Culture Building. Accessed: August 27, 2025.

McTernan, W. P., M. F. Dollard, and A. D. LaMontagne (2013). Depression in the Workplace: An Economic Cost Analysis of Depression-related Productivity Loss Attributable to Job Strain and Bullying. *Work & Stress 27*(4), 321–338.

Metcalfe, R. D., A. B. Sollaci, and C. Syverson (2023). Managers and Productivity in Retail.

Minni, V. M. L. (2024). Making the Invisible Hand Visible: Managers and the Allocation of Workers to Jobs.

Mirrlees, J. (1974). Notes on Welfare Economics, Information and Uncertainty. *Essays on Economic Behavior Under Uncertainty*, 243–261.

Mirrlees, J. A. (1976). The Optimal Structure of Incentives and Authority Within an Organization. *The Bell Journal of Economics*, 105–131.

Nagle, F. (2019). Open Source Software and Firm Productivity. *Management Science 65*(3), 1191–1215.

Nielsen, M. B. and S. Einarsen (2012). Outcomes of Exposure to Workplace Bullying: A Meta-analytic Review. *Work & Stress 26*(4), 309–332.

O*NET (2025). Management Occupations. https://www.onetonline.org/find/family?f=11. Retrieved August 19, 2025.

Pinquart, M. and R. Kauser (2018). Do the Associations of Parenting Styles with Behavior Problems and Academic Achievement Vary by Culture? Results from a Meta-analysis. *Cultural Diversity & Ethnic Minority Psychology 24*(1), 75.

Rothstein, J. (2010). Teacher Quality in Educational Production: Tracking, Decay, and Student Achievement. *The Quarterly Journal of Economics 125*(1), 175–214.

Rothstein, J. (2017). Measuring the Impacts of Teachers: Comment. *American Economic Review 107*(6), 1656–1684.

Sarker, J., A. K. Turzo, and A. Bosu (2020). A Benchmark Study of the Contemporary Toxicity Detectors on Software Engineering Interactions. In *2020 27th Asia-Pacific Software Engineering*

Conference (APSEC), pp. 218–227. IEEE.

Saygin, P. O., G. Pollard, T. Knight, and M. Rush (2025). Gender Biased Resistance to Harsh Feedback.

Shastry, G. K., O. Shurchkov, and L. L. Xia (2020). Luck or Skill: How Women and Men React to Noisy Feedback. *Journal of Behavioral and Experimental Economics 88*, 101592.

Smirnov, I., C. Oprea, and M. Strohmaier (2023). Toxic Comments Are Associated with Reduced Activity of Volunteer Editors on Wikipedia. *PNAS Nexus 2*(12), pgad385.

Solow, R. M. (1957). Technical Change and the Aggregate Production Function. *The Review of Economics and Statistics 39*(3), 312–320.

Staiger, D. O. and J. H. Stock (1994). Instrumental Variables Regression with Weak Instruments.

Sutton, R. and B. Wigert (2019). More Harm Than Good: The Truth About Performance Reviews. *GALLUP: Workplace*.

U.S. Bureau of Economic Analysis (2025). Digital Economy. https://www.bea.gov/data/special-topics/digital-economy. Retrieved September 8, 2025, from https://www.bea.gov/data/special-topics/digital-economy.

Vasilescu, B., Y. Yu, H. Wang, P. Devanbu, and V. Filkov (2015). Quality and Productivity Outcomes Relating to Continuous Integration in GitHub. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pp. 805–816.

Wagner, S. and M. Ruhe (2018). A Systematic Review of Productivity Factors in Software Development. *arXiv Preprint arXiv:1801.06475*.

Weidmann, B., J. Vecci, F. Said, S. Bhalotra, A. Adhvaryu, A. Nyshadham, J. Tamayo, and D. Deming (2026). How do you identify a good manager? *The Quarterly Journal of Economics*, qjag004.

Appendix

Managing by Feedback

Jinci Liu

August 3, 2026

A Setting, Data, and Variable Definitions

A.1 GitHub as a Workplace

GitHub is the largest online coding platform, where developers store code, share projects, and collaborate. As of July 2026, GitHub hosts over 331 million public repositories and more than 217 million users. Hoffmann et al. (2024) estimate that the open-source software ecosystem these repositories support is worth roughly $8.8 trillion in demand-side value to the firms that use it. GitHub records the full version history of each file and supports asynchronous collaboration across time zones and teams.

To join GitHub, each user must create a unique username and can publicly display personal information on a profile, which usually lists name, profile image, location, firm, bio, and email. Many developers highlight programming skills to attract potential employers, often providing accurate details and links to LinkedIn profiles or personal websites to signal credibility and professional identity (El-Komboz and Goldbeck, 2024). These profiles are the source of the demographic measures described below.

Studying workplaces on GitHub is challenging because millions of volunteers contribute and many activities are sporadic. To focus on professional settings, I include only teams owned by organizations, where developers work in teams with defined roles, and I exclude individual projects outside organizational teams so that the sample reflects structured workplace collaboration rather than ad hoc or personal coding.

A.2 Defining a Team

Economists have defined the concept of a "team" in various ways. Marschak and Radner (1958) defines a team as "an organization the members of which have only common interests," highlighting the alignment of incentives. Holmström (1982) defines a team "rather loosely as a group of individuals who are organized so that their productive inputs are related," emphasizing joint production. In more recent work, such as Jones (2021), teams are observed through patterns of co-authorship in science. Following this last definition, I treat each organization-owned GitHub repository as a team: a group of developers jointly contributing code toward a shared project, much like a group of researchers co-authoring a paper. I restrict the analysis to organization-

affiliated teams for three reasons. First, they use structured workflows. Second, they align with firm objectives such as product development or commercialization (Andersen-Gott et al., 2012). Third, they better represent workplace settings than individual projects, which often reflect personal work.

A.3 Identifying Team Members

GitHub's *author association* field classifies each user, when they act, as *owner*, *member*, *collaborator*, *contributor*, *none*, or *mannequin*. I define a team member as a user labeled *owner*, *member*, or *collaborator*. *Owner* and *member* are formally affiliated with the organization; *collaborator* is an invited contributor with similar technical access. Users labeled *contributor* or *none* are external or occasional participants without sustained permissions, and *mannequin* is a placeholder account that I exclude. This measure is preferred to GitHub's `MemberEvent`, which captures only a subset of membership changes.

Because *author association* is observed only when a user acts, months with no activity are missing, making it difficult to track continuous membership directly. To address this, I impute continuous membership between the first and last months in which a user is observed as a team member.

A.4 Predicting Gender

To predict the gender of developers based on their profile page avatars, I select a deep learning model that closely matches the images in the training sample in terms of resolution and cropping methods. The chosen model trains vision transformers on male and female portraits and produces, for each image, predicted probabilities for both genders. To determine the optimal cutoff for gender classification, I use a sample labeled by human annotators and evaluate accuracy across different thresholds. The highest accuracy is achieved with a cutoff of 0.85. For each image, I first identify the gender with the higher predicted probability and then assign that gender only if this maximum probability exceeds 0.85; otherwise, the observation is coded as missing. Classification accuracy is computed on the labeled subset where a gender is assigned, excluding cases left unclassified.

The resulting variable, *Predicted Gender*, has three categories: male, female, and missing. In the regressions, I enter these categories as separate indicators, with one category omitted as the reference group. Thus, unclassified developers form a distinct category and are not treated as lying between male and female.

A.5 Predicting Race

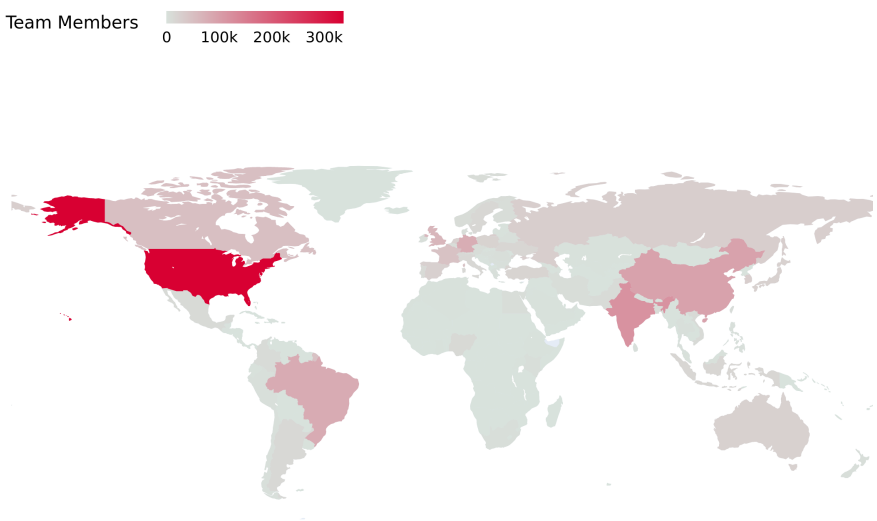
I use Ethnicolr, developed by Laohaprapanon et al. (2017), to predict developers' race and ethnicity from their names. The algorithm uses first and last names to classify individuals into three racial categories: Asian, Black, and White. It is trained on data from the US Census, Florida voter

registration, and Wikipedia. For each developer, I take the race with the highest predicted probability as the assigned category. When a name is unavailable, the observation is coded as "missing." This yields four mutually exclusive outcomes: Asian, Black, White, and Missing. For the heterogeneous analysis, I construct separate indicators for each racial category. For example, *Asian* equals one when the predicted category is Asian and zero otherwise. I also include a separate indicator for developers whose race is missing. I construct the *White* indicator analogously.

A.6 Predicting Location

I built the country variable in three steps. Between October 2023 and March 2024, I scraped each developer's free-text "location" field from their GitHub profile. I then standardized entries—e.g., "Québec" → "Canada," "北京" → "China," "São Paulo" → "Brazil", "München" → "Germany". Finally, I assigned each user to exactly one country or flagged the entry as missing if it failed to match any country. The resulting variable records each developer's self-reported nation of residence.

Figure A1: Geographic Distribution of Team Members



Notes: This world map shows the number of team members by country, based on their reported location. Team member is defined in Section 2.1.2. Darker red shades indicate higher team member counts. The United States has the largest concentration, followed by India, China, and Brazil. Participation is also notable in parts of Europe and Southeast Asia.

A.7 Comparing Reviewers and Developers

Within the random reviewer sample, I compare reviewers and developers along five dimensions: gender, race, platform reputation, experience, and activity. Figure A2 shows that reviewers are more likely to be male (89% vs. 86%) and less likely to be Asian (16% vs. 21%). They have greater platform visibility (133 followers vs. 83) and are more active (1,844 yearly activities vs. 668). LinkedIn job titles confirm this pattern: reviewers cluster in senior roles such as *Senior Software Engineer*, *Staff*, and *Principal*, whereas developers concentrate in junior roles such as *Software Engineer* and *Intern*.

Reviewers and developers also differ in their roles within the team. Reviewers are not always senior in title: in 31.97% of cases in the random reviewer sample (28.04% in the full sample), the assigned reviewer is junior to the developer they review. This is consistent with my functional definition of a reviewer—based on evaluation and approval authority over the case rather than on title—under which a reviewer can hold gatekeeping authority without out-ranking the developer. Reviewing and coding are also not mutually exclusive: 45.91% of reviewers in the random reviewer sample (41.43% in the full sample) also submit code within the same team-year, indicating that many reviewers act as working managers who both write and evaluate code.

A.8 Code Quantity Measures

I construct four weekly code-quantity measures from the case (pull-request) lifecycle. Let t denote a week, i a developer, and k a case.

New Code Lines

$$new\_code\_lines_{i,t} = \sum_{\substack{k: \\ \text{open\_week}(k)=t}} \text{additions}_{i,k,\text{opened}}$$

where $\text{additions}_{i,k,t,\text{opened}}$ is the lines developer i first submits for case k in week t. New code lines capture the total lines first submitted by developer i in week t, before any review.

Rewritten Code Lines

$$rewritten\_code\_lines_{i,t} = \sum_{\substack{k: \\ \text{close\_week}(k)=t}} |\text{additions}_{i,k,\text{closed}} - \text{additions}_{i,k,\text{opened}}|$$

the revision a case undergoes between submission and closing. Revisions are attributed to the week the case closes, when the revised code enters the codebase. A case opened in one week and closed in a later week therefore contributes new lines to the first week and rewritten lines to the second.

Consider a developer i who makes a single code submission in week 2020-01, adding 120 lines of code to a case. By the time the case is closed in week 2020-02, the cumulative number of added

lines has increased to 200. We therefore define

$$new\_code\_lines_{i,\text{2020-01}} = 120,$$

and

$$rewritten\_code\_lines_{i,\text{2020-02}} = 200 - 120 = 80.$$

A.9 Code Quality Measures

Code Quality

$$code\_quality_{i,t} \;=\; \frac{new\_code\_lines_{i,t}}{new\_code\_lines_{i,t} + rewritten\_code\_quantity_{i,t}}.$$

This metric captures the proportion of code submitted by developer i in week t that required no subsequent revision. Both new and rewritten lines are assigned to the week in which the case was opened. Thus, *code_quality_i*, t is computed only from cases opened in week t. A value of 1 indicates that none of the lines submitted at case opening were subsequently rewritten, while lower values indicate more extensive revisions. Higher *code_quality_i*, t therefore reflects greater first-pass accuracy and higher code quality.

Case Quality

$$case\_quality_{i,t} \;=\; \frac{\sum_{\substack{k:\,\text{author}(k)=i,\\ \text{open\_week}(k)=t}} \mathbf{1}\{\text{correct}_k\}}{\sum_{\substack{k:\,\text{author}(k)=i,\\ \text{open\_week}(k)=t}} 1},$$

where

$$\mathbf{1}\{\text{correct}_k\} = \begin{cases} 1, & \text{if the initial code submission for case } k \text{ is merged,} \\ 0, & \text{otherwise.} \end{cases}$$

$case\_quality_{i,t}$ measures the share of cases opened by developer i in week t that meet review standards without requiring substantial changes.

A.10 Classifying Project Stage

A GitHub *release* is the project's official, stable snapshot. Because each release package includes bug fixes, new features, or architectural changes, the sequence of releases offers a natural way to track a project's progress. Each release includes a *Semantic Versioning* (SemVer) tag in the format `MAJOR.MINOR.PATCH`, such as `v1.2.3`. The three components indicate the scale of change: the patch number increases for bug fixes, the minor number for backward-compatible feature additions, and the major number for breaking or incompatible changes.

Most modern projects adopt the SemVer tag, and the release tag provides a reliable and comparable signal of project stages. To classify project maturity, I extract the `MAJOR` and `MINOR` com-

ponents from each release's Semantic Versioning tag. I then group releases into three stages. Stage 1 (Early) includes all tags with `MAJOR` = 0, as well as `1.0.0`, reflecting either pre-release development or an initial stable version. Stage 2 (Stable) covers all remaining releases with `MAJOR` = 1, that is, every `1.x.y` tag other than `1.0.0`, representing backward-compatible feature growth after the first stable release. Stage 3 (Mature) includes all releases with `MAJOR` ≥ 2, indicating that the project has undergone at least one major upgrade. [28] Projects that do not follow SemVer, for example, those using date stamps, commit hashes, or missing tags, are grouped into a separate "non-SemVer" category. This approach yields a simple, interpretable stage variable that can be compared across projects, much like how academic papers move through stages such as data collection, analysis, submission, and revision.

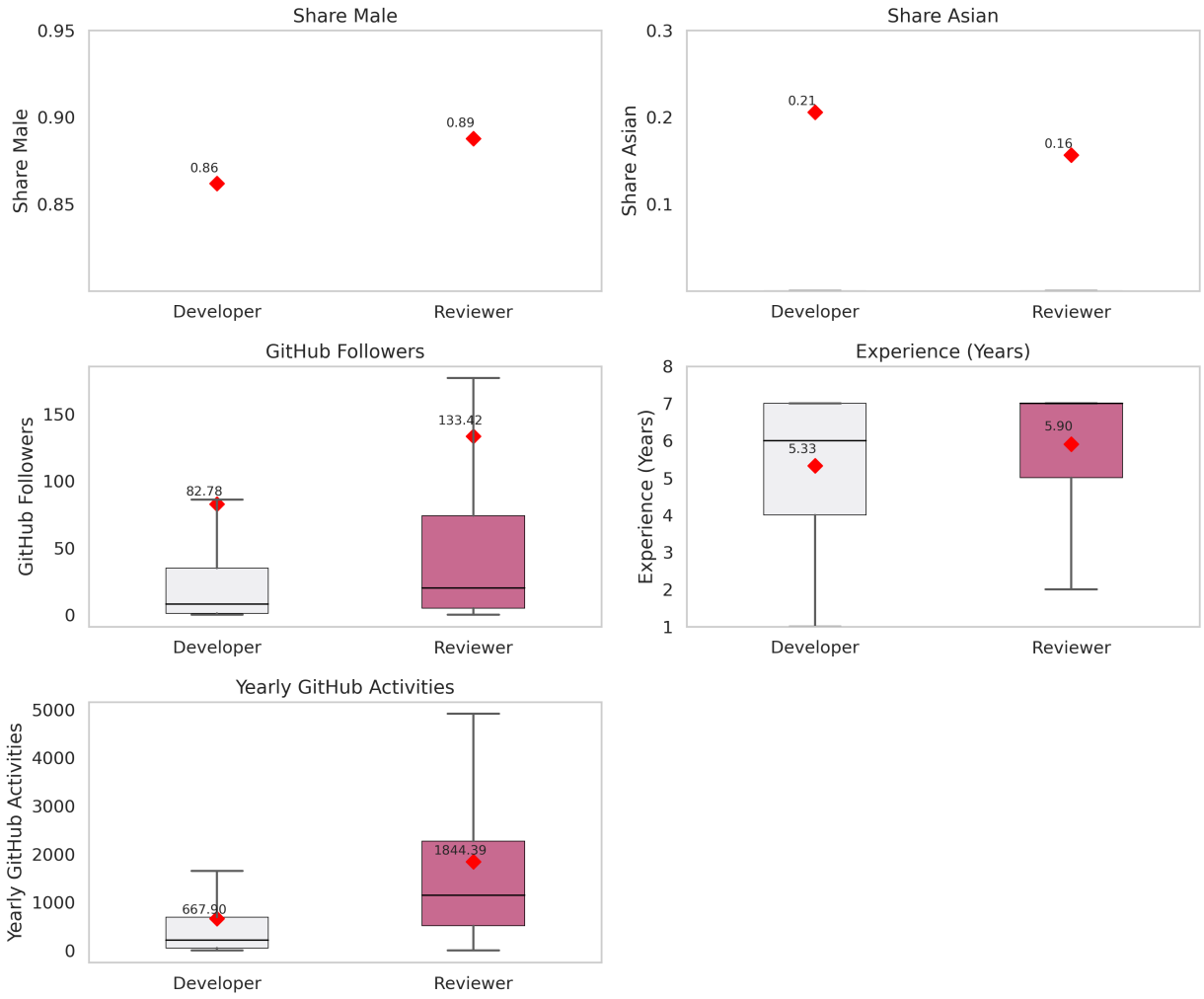
Assigning Cases to Project Stage. Let d_k denote the opening date of case k, and let $\{(s_i, t_i)\}_{i=1}^{N}$ represent the sequence of project releases, where s_i is the stage assigned to release i and t_i is the release date. The project stage at the time of case k is assigned as:

$$\text{stage}_k = s_{i^*} \quad \text{where} \quad i^* = \max\{i : t_i \leq d_k\}.$$

This procedure links each case to the most recently completed release stage available at the time the case was opened. Among cases in the random reviewer sample, 15.45% fall into Stage 1 (early development), 17.40% into Stage 2 (stable), 42.19% into Stage 3 (mature), and 24.96% are associated with a non-SemVer category.

[28] Incrementing the `PATCH` number (e.g. 1.0.1 $\rightarrow$ 1.0.2) records a bug fix; raising the `MINOR` number (1.0.2 $\rightarrow$ 1.1.0) adds new, backward-compatible features; and advancing the `MAJOR` number (1.1.0 $\rightarrow$ 2.0.0) signals an intentional, incompatible redesign. See https://semver.org/ for more information.

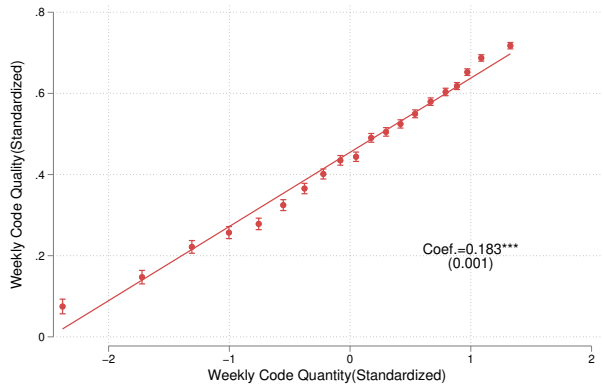
Figure A2: Comparison between Developers and Reviewers



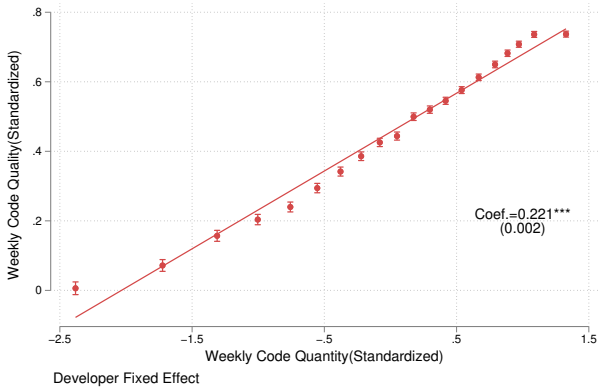
Notes: Each panel compares characteristics between developers and reviewers on GitHub. Developers are defined as those who never serve as reviewers. The sample includes teams that adopt random reviewer assignment. "Share Male" and "Share Asian" report the average share across individuals (red diamonds). "GitHub Followers," "Experience," and "Yearly GitHub Activities" display distributions and mean values (red diamonds) using standard box plots. The black line inside each box represents the median; whiskers extend to 1.5 times the interquartile range. GitHub followers are scraped as of 2023. Experience is calculated as 2024 minus the first year a developer appeared on GitHub. Yearly GitHub activities measure the total number of activities in the most recent calendar year.

Figure A3: Developer Weekly Code Quantity vs. Code Quality

(a) Without Developer Fixed Effect



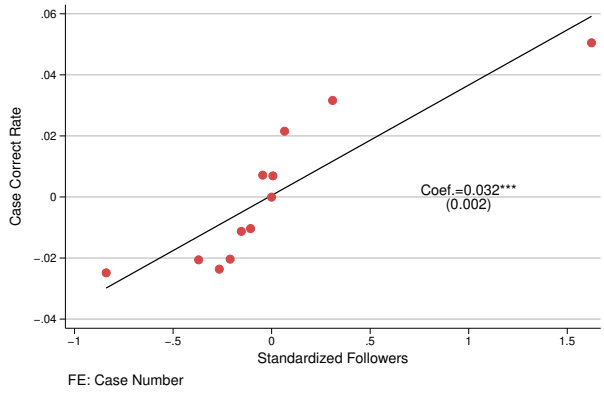
(b) With Developer Fixed Effect



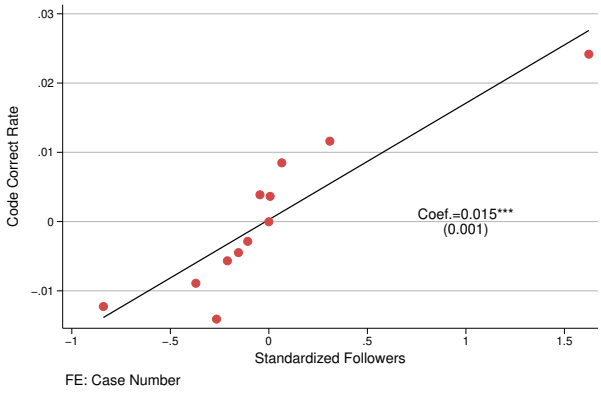
Notes: This figure plots binned relationships between weekly code quantity and weekly code quality. Both variables are standardized. Panel (a) reports the raw relationship without developer fixed effects. Panel (b) reports the within-developer relationship after absorbing developer fixed effects. The fitted line shows the corresponding linear regression coefficient, with standard errors in parentheses. The number of observations is 256,998 in Panel (a) and 253,050 in Panel (b).

Figure A4: Output Quality Validation

(a) Case Quality



(b) Code Quality



Notes: This figure validates output-quality measures. The x-axis displays each developer's standardized follower count (as a proxy for reputation), and the y-axis shows their average Case Quality and Code Quality as defined in Appendix A.9. The binscatter controls for the total number of cases per developer to account for varying activity levels. Developers with more followers exhibit higher quality.

B Random Reviewer Assignment

GH Archive does not record when teams enable random reviewer assignment, since such changes do not appear in the event timeline. Instead, I infer adoption by identifying the earliest pull request that modifies one of the configuration files associated with random assignment. Table B1 lists these files and explains their functions. By noting the first change to any of these files, I determine when each team activated the random reviewer assignment.

Table B1: Configuration Files Indicating Random Reviewer Assignment

Configuration File	Purpose
`CODEOWNERS`	GitHub's native review-routing file. Maps file patterns to an owning reviewer group, so a matched pull request auto-requests review from that group rather than from a reviewer the case submitter picks; where the owner is a team, GitHub rotates the request among its members by recent review load.
`.github/auto_assign.yml` or `.github/auto-assign.yml`	Config for the "auto-assign" Action: lists eligible reviewers and rules (e.g. random selection) for automatic assignment on pull request open.
`.github/reviewer_lottery.yml` or `.github/reviewer-lottery.yml`	Defines reviewer pools and lottery rules for custom bots that rotate or randomly pick reviewers per pull request.
`.github/assign_reviewers.yml` or `.github/assign-reviewers.yml`	Generic config used by GitHub Actions to specify reviewer pools and selection logic (e.g. round-robin, weighted).
`.github/workflows/assign_reviewers.yml` or `.github/workflows/assign-reviewers.yml`	A workflow that reads the above config and auto-assigns reviewers on pull request events.
`.github/workflows/auto_assign.yml` or `.github/workflows/auto-assign.yml`	Workflow file for an Action that implements automated reviewer selection (e.g. based on past workload or random choice).

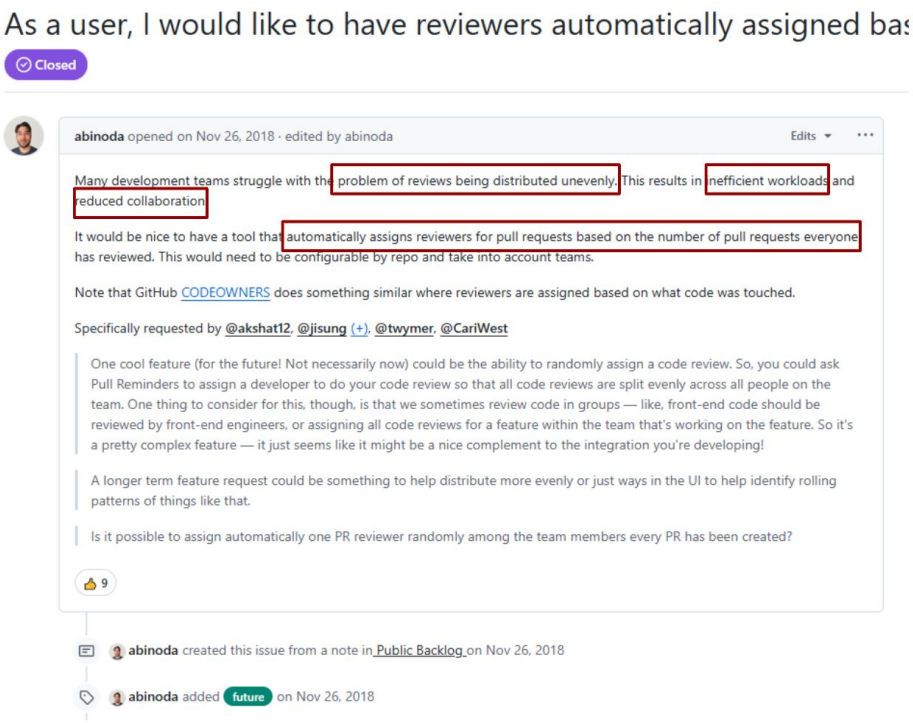
Notes: This table lists the most common files and workflows used for random reviewer assignment. Additional custom or third-party configurations may also exist, but this selection reflects a conservative approach.

The files in Table B1 differ in how they pick a reviewer. Lottery and auto-assign workflows choose among team members by recent review load or at random. CODEOWNERS first narrows the choice to the members who own the files a submission touches, and a rotation then picks one of them. Because the files a developer works on are not random, I do not treat CODEOWNERS routing as random in itself. The design does not require it to be: what it requires is that among the members who could have been assigned, the one who is assigned does not depend on the submission. Two facts in the estimation sample, which includes these teams, are consistent with this. Developers rarely draw the same reviewer twice in a row: conditional on being reviewed by a given reviewer, the chance the same reviewer handles the next case is 7.32% (Table 1). And the assigned reviewer's feedback style is uncorrelated with developer and case characteristics (Figure 6).

Figure B1 illustrates the motivation for random reviewer assignment, drawn from a discussion post highlighting the need to balance review workloads. Figure B2 shows two scenarios: Figure B2a with automatic random assignment, and Figure B2b where the developer selects re-

viewers manually—either via "git blame" suggestions or by typing usernames directly.

Figure B1: Reason for Random Reviewer Assignment

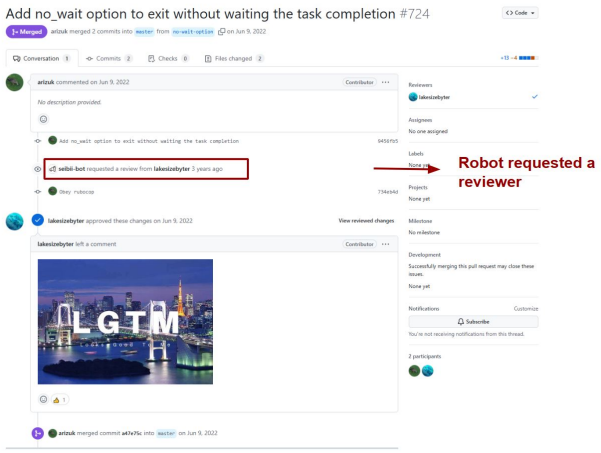


Notes: The figure captures a discussion proposing automatic reviewer assignment using metrics such as each member's recent review workload. These systems are deterministic in design but generate reviewer selection that is independent of case content and developer identity, conditional on workload. Original post, accessed 2024-12-31.

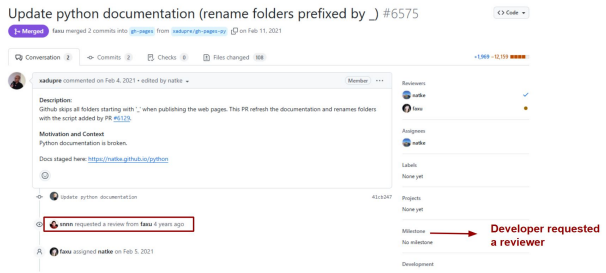
Table B2 reports the share and total number of teams in major tech firms adopting GitHub's random reviewer feature. Overall, 0.2% of teams use the feature (3,457 out of 1,744,184).

Figure B2: Random Reviewer Example

(a) Review requested by Bot



(b) Review requested by Developer



Notes: This figure illustrates two types of review requests. Figure B2a depicts a GitHub bot initiating a review request (random assignment), while Figure B2b shows a developer requesting a review.

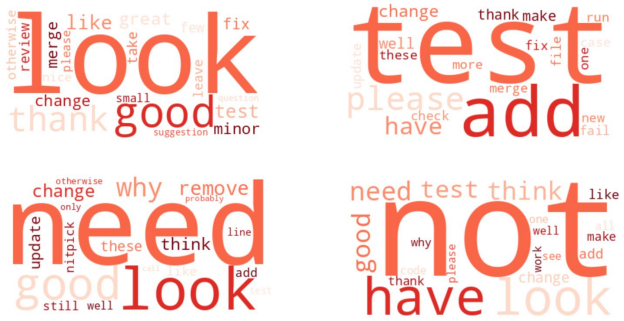
Table B2: Share of Teams in Major Tech Firms adopting Random Reviewer Assignment

Firm	Share(%)	Teams	Firm	Share (%)	Teams
GitHub	3.226	341	Spotify	0.508	197
Pinterest	3.030	66	IBM	0.337	1780
Intel	1.173	597	Adobe	0.140	714
Apple	1.156	173	Uber	0.000	205
Netflix	1.156	173	PayPal	0.000	189
Salesforce	1.145	262	Yahoo	0.000	185
NVIDIA	1.124	267	Amazon	0.000	146
Microsoft	1.119	4645	Twitter	0.000	137
Stripe	0.917	109	Airbnb	0.000	120
Facebook	0.855	234	Dropbox	0.000	120
Oracle	0.800	250	eBay	0.000	100
OpenAI	0.794	126	Reddit	0.000	60
Google	0.723	1797	Zoom	0.000	43
Lyft	0.699	143	Snapchat	0.000	27
Tencent	0.585	171	Tesla	0.000	5

Notes: This table summarizes the share and total number of teams in major tech firms adopting the random reviewer feature. Overall, 0.2% of teams used the feature (3,457 out of 1,774,184). GitHub, Pinterest, and Intel show the highest adoption rates, with GitHub at 3.23% across 341 teams. Microsoft (4,645 teams) and Google (1,797 teams) also have above-average adoption. Firms like Uber, Twitter, and Tesla show no adoption during the sample period.

C Feedback Classification and Descriptive Facts

Figure C1: Word Clouds of Principal Components of Reviewer Feedback



Notes: This figure presents word clouds for the top four principal components extracted from reviewer feedback in the random reviewer sample. PCA reduces millions of feedback messages into a few components that capture most of the meaningful variation in the data. Each word cloud visualizes one component, where larger words appear more frequently within that dimension. The components consistently capture meaningful dimensions of feedback—such as positive (e.g., *good*, *thank*), negative (e.g., *not*), and constructive (e.g., *test*, *add*) language. A single word may load on multiple components if it conveys more than one meaning.

C.1 Pre-processing

To ensure reliable measurement, I clean and standardize the feedback text before classification. First, I remove quoted blocks from other users' messages to prevent misattribution, as well as code snippets. Second, I replace common software-engineering acronyms, such as *"lgtm" (looks good to me)* and *"kiss" (keep it simple)*, with their full forms. Third, I account for software-specific terminology that general-purpose models often misclassify: words like "kill," "trash," and "dump" refer to legitimate coding operations but are frequently flagged as toxic (Sarker et al., 2020). Addressing these ambiguities yields more accurate classification.

Examples of Feedback Types with Continuous Scores

The following examples report ToxiGen-predicted toxicity scores. Blunt critiques of the work are classified as non-toxic, while feedback that targets the person is flagged as toxic:

1. *This is bad code* → non-toxic, probability: 0.918
2. *Your paper is not of general interest to publish at AER* → non-toxic, probability: 0.999
3. *What's wrong with you to write such bad code?* → toxic, probability: 0.712
4. *Women are just not good at coding* → toxic, probability: 0.988

5. *Reject. We can learn nothing from your paper* → toxic, probability: 0.674

6. *You'll never survive the job market with work like this* → toxic, probability: 0.986

The next examples report GPT-based constructiveness scores on the 0–10 scale, where a threshold of 4 defines constructive feedback:

1. *Agree* → non-constructive, score: 0

2. *This is not clear* → non-constructive, score: 1

3. *I don't think all introspector errors are reported in the domain status. We need to make sure that all severe jrf ones are* → constructive, score: 4

C.2 Toxicity

Figure C2: Toxigen Human-Annotation Guidelines

Notes: This is from Figure 6 of Hartvigsen et al. (2022), which shows how human annotators label toxicity for the Toxigen Classification Model.

C.3 Constructiveness

I use large language models (LLMs) to detect constructive feedback on GitHub. I experimented with small changes in prompt wording, aiming to give the LLM a clear and objective instruction. I also tested including and excluding code snippets in the input. These changes did not alter the binary constructive versus non-constructive classification on a manually labeled validation set of 1000 feedback messages, although minor variations appeared in token-level probabilities.

My specific LLM prompt is as follows:

System Content:

You are an experienced principal software engineer reviewing a pull request comment. Rate its constructiveness on a scale from 0 (not constructive) to 10 (highly constructive). Focus on whether the comment provides specific or actionable information. A specific comment identifies a bug, suggests an improvement, or points to a concrete issue. If the comment is vague or lacks detail, treat that as a weakness. Focus on the content, not on the tone. Provide them in JSON format with the following key: info_score.

User Content:

Code: `<INPUT>`

Comment: `<INPUT>`

To identify the best-performing LLM for this task, I compare four OpenAI models (gpt-4.1-nano, gpt-4o-mini, gpt-4o, and o3) to find the one whose judgments most closely match those of software industry professionals, while minimizing cost. The initial human labels are produced by a master's student in computer science and verified by a software engineer.

The annotator rates each feedback item on a 0–10 scale: scores of 0–2 indicate little actionable information, 3 indicates borderline relevance, and 4–10 indicate specific or actionable information. I define feedback as constructive when the score is at least 4. This cutoff matches the paper's definition of constructiveness as concrete, actionable information rather than general helpfulness.

Once I obtain the ground truth and model predictions from different LLMs, I evaluate performance using binary rather than absolute scores to reduce measurement error from minor discrepancies in human ratings. To generate binary labels, I test three decision cutoffs (≥ 3, ≥ 4, and ≥ 5). Each cutoff changes the share of feedback classified as constructive and affects model performance. Table C1 reports results for each cutoff.

Table C1: Accuracy and F1 Scores by Model and Cutoff

Cutoff	**gpt-4.1-nano**		**gpt-4o-mini**		**gpt-4o**		**o3**	
	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
≥ 3	0.87	0.817	0.76	0.760	0.82	0.804	0.80	0.783
≥ 4	0.84	0.579	0.90	0.814	0.88	0.767	0.90	0.828
≥ 5	0.84	0.200	0.96	0.875	0.87	0.723	0.93	0.829

I select the gpt-4o-mini model because it offers the best balance between precision and recall, aligns closely with the human annotator's definition of clear and actionable feedback, and delivers

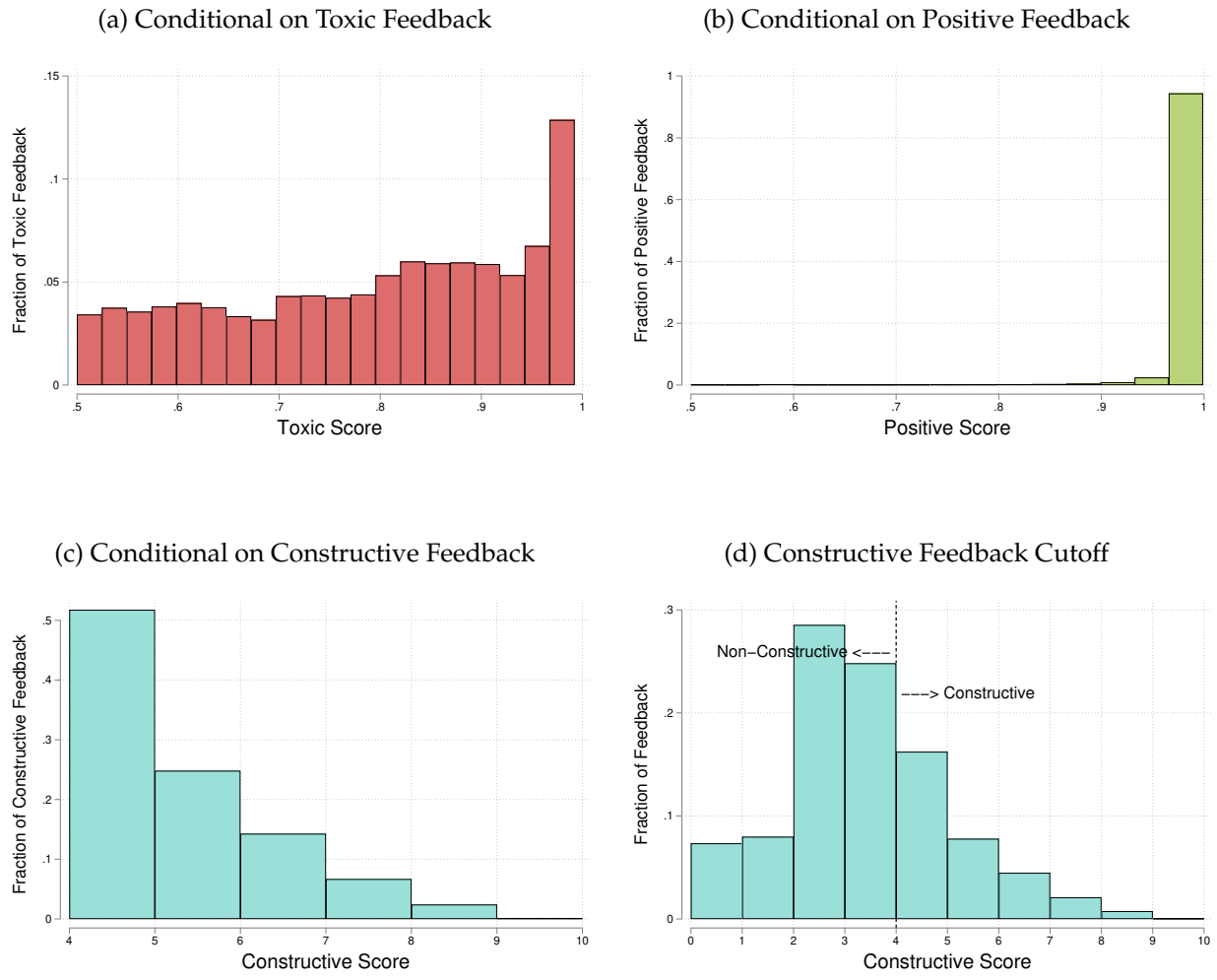
performance comparable to higher-end models at a fraction of the cost. For example, it achieves an F1 score of 0.814 at a cost of about $0.93 to classify 10,000 feedback messages, compared with 0.829 and $23.25 for o3—a saving of more than 95%. I use a decision cutoff of 4 because it maintains a reasonable class balance (about 25% positives) and reflects the annotator's standard for clear and actionable feedback. I also report results for cutoff = 3 as a robustness check. What is stable is the empirical result: Figure D5 re-estimates the main specification with constructiveness defined at the ≥ 3 cutoff, and the estimates are similar. Table C2 presents examples of constructive feedback from the real data.

Table C2: Examples of Reviewer Feedback and Constructiveness Scores

Feedback Message	**Constructive**	**Score**
Agree	0	0
Just checked one more time - now it's okay	0	1
Same advice on the assets point of view	0	2
Explain why there are private fields in this visitor. too multiple of them	0	3
I don't think all introspector errors are reported in the domain status. We need to make sure that all severe jrf ones are	1	4
You only need to define the acronym once, so I'd suggest doing it in the text not the title	1	5
This should be an async function that can use await and, as a result, standard trycatch blocks	1	6
We need to be generating a new nonce used to encrypt here rather than reusing the one that was used previously to encrypt the keyring. Otherwise there's a crypto bug in this that would allow an attacker to combine the old ciphertext with the new ciphertext in a way that could reveal the xor'd plaintext of the two messages.	1	10

Notes: This table reports examples of reviewer feedback messages scored by GPT-4o-mini. Scores range from 0 to 10, with higher scores indicating more specific and actionable information about the submitted code. *Constructive* equals one if the score is at least 4.

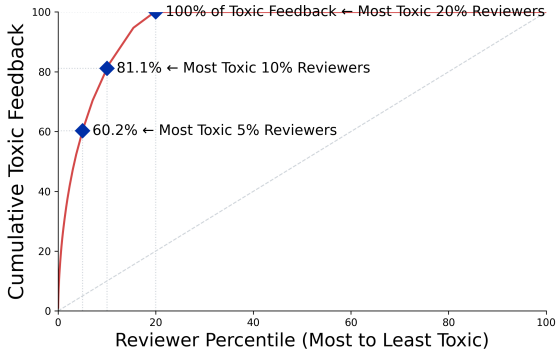
Figure C3: Distribution of Predicted Feedback Probability



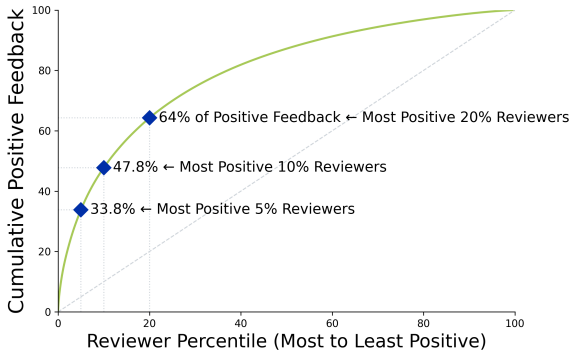
Notes: These figures illustrate the distribution of predicted probability scores for each feedback dimension, based on all feedback messages written by code reviewers in the random reviewer sample. Figures C3a and C3b report score distributions for toxic and positive feedback, respectively, based on binary labels assigned by a pre-trained classifier. Figure C3c shows the distribution of predicted constructive scores generated by GPT-4o-mini. Figure C3d illustrates the classification of feedback as constructive or non-constructive using a cutoff on the GPT-based score: messages above the threshold are labeled constructive, while those below are labeled non-constructive. As a robustness check, I replicate the analysis with an alternative cutoff of 3 (rather than 4); the results remain qualitatively unchanged (see Figure D5).

Figure C4: Reviewer Heterogeneity in Feedback Styles

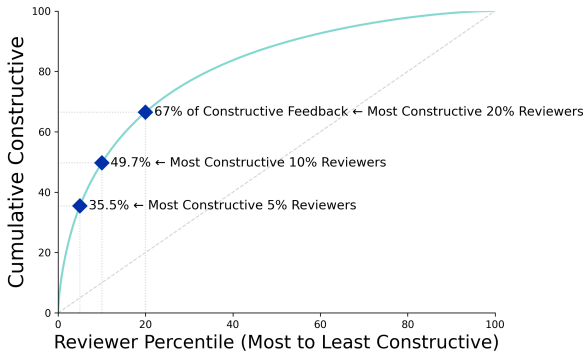
(a) Toxic Feedback



(b) Positive Feedback



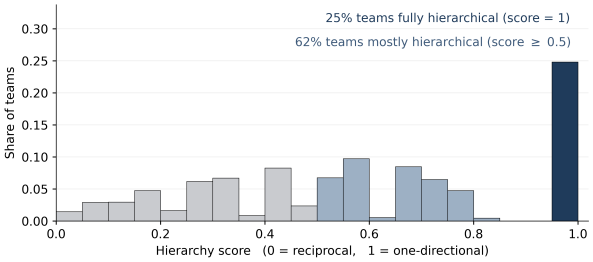
(c) Constructive Feedback



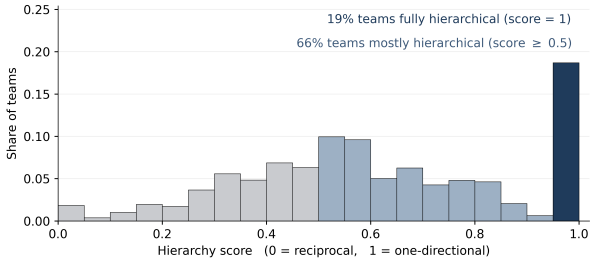
Notes: Each panel plots the cumulative share of feedback messages of a given type accounted for by reviewers, ranked from most to least likely to provide that type of feedback. Panel (a) shows toxic feedback, Panel (b) positive feedback, and Panel (c) constructive feedback. The diagonal line represents an equal distribution across reviewers. A steeper curve indicates greater concentration, meaning that a small share of reviewers accounts for a large share of messages of that type. The sample is restricted to the random reviewer sample.

Figure C5: Team Feedback Structure

(a) Teams with Size 5



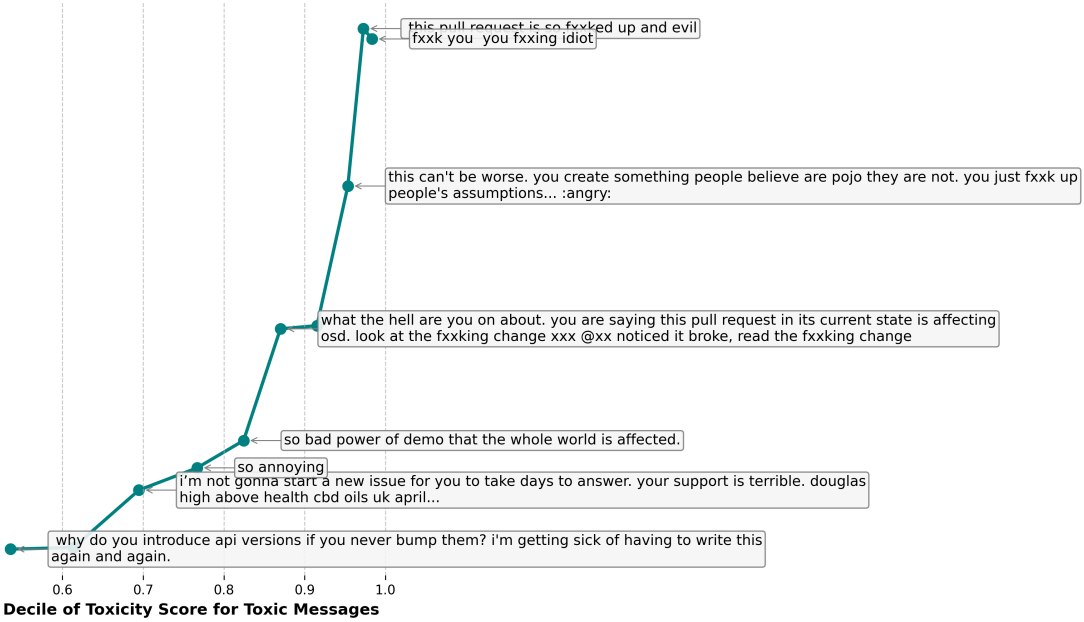
(b) Random Reviewer Sample



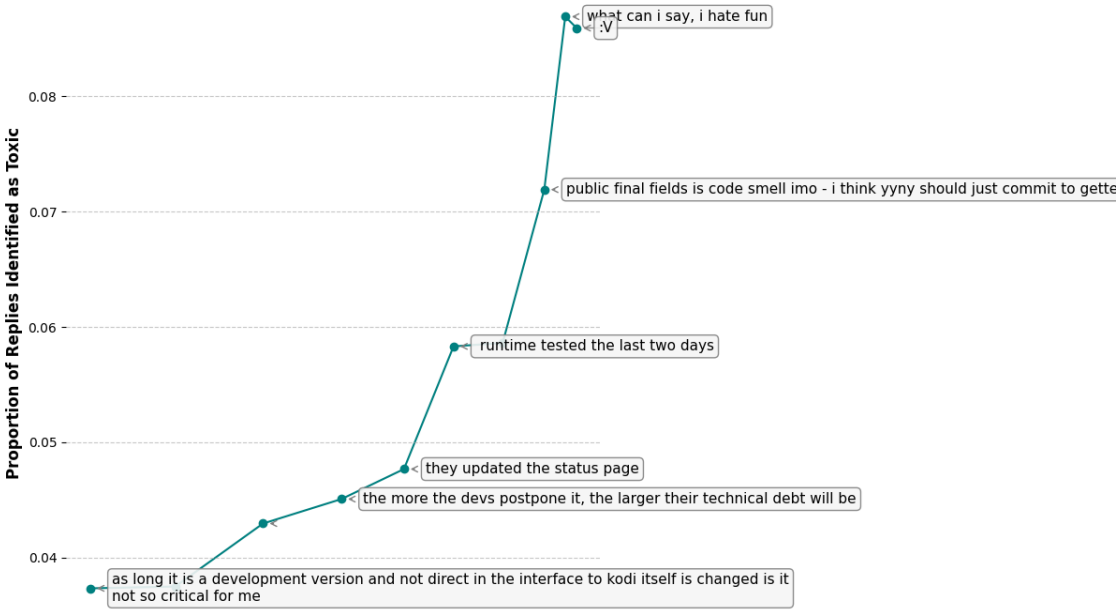
Notes: This figure plots the distribution of team hierarchy scores over 2017–2023. The unit of observation is the team–year. For each team, the hierarchy score is $1 - 2R/E$, where E is the number of directed reviewer–developer pairs and R the number of reciprocated reviewer–developer pairs; it ranges from 0, when members review each other symmetrically, to 1, when review flows in one direction. Higher scores indicate that review is concentrated among a small subset of members; lower scores reflect a more evenly distributed structure. Teams with a single case or a single member are excluded, since one-directional review is mechanical when only one case is reviewed. **Panel (a)** restricts to teams with exactly five members. **Panel (b)** uses the random-reviewer sample, which includes teams that use random reviewer assignment, with at least two reviewers per year and at least ten cases reviewed per reviewer per year. Figure 4 plots the full sample.

Figure C6: Toxic Feedback and Replies

(a) Toxic Feedback by Classification Probability Decile



(b) Corresponding Replies



Notes: This figure displays randomly selected examples of toxic feedback and their replies in the full sample. These interactions often occur in contexts where senior members provide feedback or evaluations to juniors, and developers who receive toxic feedback frequently respond by explaining their actions or decisions.

D Identification and Robustness

D.1 Alternative IV Estimators

The main instrument is a residualized leave-out mean leniency score, based on a reviewer's feedback messages in cases involving other developers. Following standard practice (Dobbie and Song, 2015; Dobbie et al., 2017; Bhuller et al., 2020; Agan et al., 2023), I report F-statistics without correcting for the fact that the instrument is estimated. This approach has attractive properties in applied settings.

As Agan et al. (2023) note, including reviewer fixed effects directly may lead to bias due to the large number of potentially weak instruments. In addition, my specification includes many fixed effects (e.g., team-by-year), which are necessary for identifying cases with as-if random reviewer assignment. Still, it may introduce bias in jackknife IV estimators (Kolesár, 2013). Using a continuous instrument also enables the estimation of marginal treatment effects (Heckman and Vytlacil, 2005).

While convenient, the main estimation strategy does not account for the fact that the instrument is itself estimated. To assess robustness, I explore several alternative approaches commonly used when IV estimates may be biased due to many (potentially weak) instruments. Table D1 reports these results. Column (1) repeats the main 2SLS estimates using the residualized leave-out mean leniency score. Column (2) reports estimates from a limited information maximum likelihood (LIML) model using all reviewer dummies as instruments (Bekker, 1994; Angrist and Frandsen, 2022). Column (3) presents results from the modified bias-corrected two-stage least squares (MBTSLS) estimator of Kolesár et al. (2015), and column (4) shows estimates from the unbiased jackknife IV estimator (UJIVE) of Kolesár (2013). In each case, the estimated coefficients are nearly identical to the main 2SLS results.

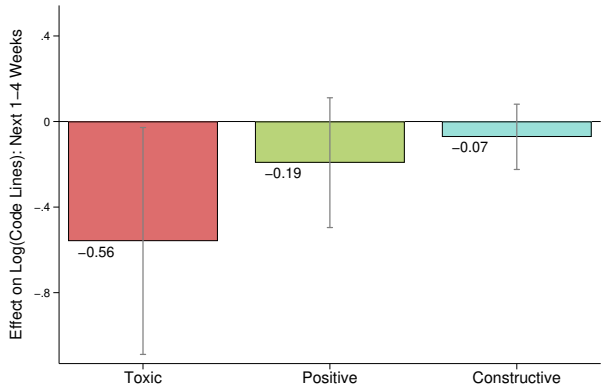
Table D1: Alternative IV Estimators

	(1) Main	(2) LIML	(3) MBTSLS	(4) UJIVE
Panel A: Toxic Feedback				
Code Quantity Ever	-0.074*	-0.074*	-0.074*	-0.077**
	(0.039)	(0.039)	(0.039)	(0.039)
Code Quantity Log	-0.558*	-0.558*	-0.558*	-0.569*
	(0.328)	(0.328)	(0.328)	(0.328)
Non-Code Quantity Ever	-0.019	-0.019	-0.019	-0.020
	(0.019)	(0.019)	(0.019)	(0.019)
Non-Code Quantity Log	-0.138	-0.138	-0.138	-0.143
	(0.145)	(0.145)	(0.145)	(0.145)
Case Quality	-0.050	-0.050	-0.050	-0.049
	(0.044)	(0.044)	(0.044)	(0.044)
Code Quality	-0.091***	-0.091***	-0.091***	-0.091***
	(0.033)	(0.033)	(0.033)	(0.033)
Panel B: Positive Feedback				
Code Quantity Ever	-0.033**	-0.033**	-0.033**	-0.033**
	(0.017)	(0.017)	(0.017)	(0.017)
Code Quantity Log	-0.192	-0.192	-0.192	-0.190
	(0.136)	(0.136)	(0.136)	(0.135)
Code Quality	0.044***	0.044***	0.044***	0.044***
	(0.013)	(0.013)	(0.013)	(0.013)
Case Quality	0.022	0.022	0.022	0.022
	(0.018)	(0.018)	(0.018)	(0.018)
Non-Code Quantity Ever	-0.011	-0.011	-0.011	-0.011
	(0.008)	(0.008)	(0.008)	(0.008)
Non-Code Quantity Log	-0.130**	-0.130**	-0.130**	-0.130**
	(0.061)	(0.061)	(0.061)	(0.061)
Panel C: Constructive Feedback				
Code Quantity Ever	-0.046***	-0.046***	-0.046***	-0.046***
	(0.010)	(0.010)	(0.010)	(0.010)
Code Quantity Log	-0.072	-0.072	-0.072	-0.071
	(0.080)	(0.080)	(0.080)	(0.080)
Code Quality	-0.072***	-0.072***	-0.072***	-0.072***
	(0.008)	(0.008)	(0.008)	(0.008)
Case Quality	-0.100***	-0.100***	-0.100***	-0.099***
	(0.010)	(0.010)	(0.010)	(0.010)
Non-Code Quantity Ever	-0.009*	-0.009*	-0.009*	-0.009*
	(0.005)	(0.005)	(0.005)	(0.005)
Non-Code Quantity Log	-0.023	-0.023	-0.023	-0.023
	(0.035)	(0.035)	(0.035)	(0.035)
Team × Year FE	✓	✓	✓	✓
Decile t–1 activity	✓	✓	✓	✓
Decile t–1 message	✓	✓	✓	✓

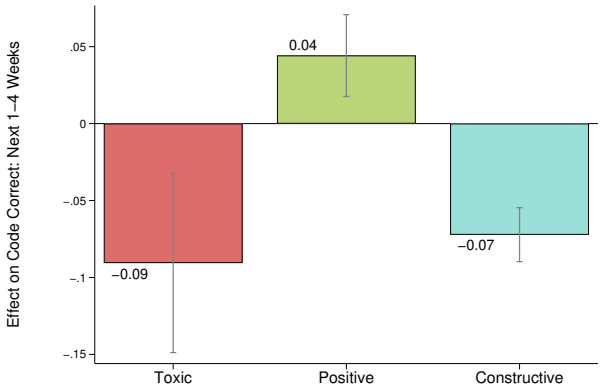
Notes: This table reports 2SLS estimates using alternative estimators, as indicated in the column headers. All specifications include team-by-year fixed effects and developer controls. OLS and reduced-form estimates for the same specification appear in Table 5. Standard errors are clustered at the reviewer level. Column (1) presents the main 2SLS estimates using the residualized leave-out mean leniency score. Column (2) applies limited information maximum likelihood (LIML) with all reviewer dummies as instruments. Column (3) reports modified bias-corrected 2SLS estimates following Kolesár et al. (2015). Column (4) reports UJIVE estimates following Kolesár (2013). *$p < .10$, ** $p < .05$, *** $p < .01$.

Figure D1: Robustness: Clustering at the Team Level

(a) Code Quantity



(b) Code Quality



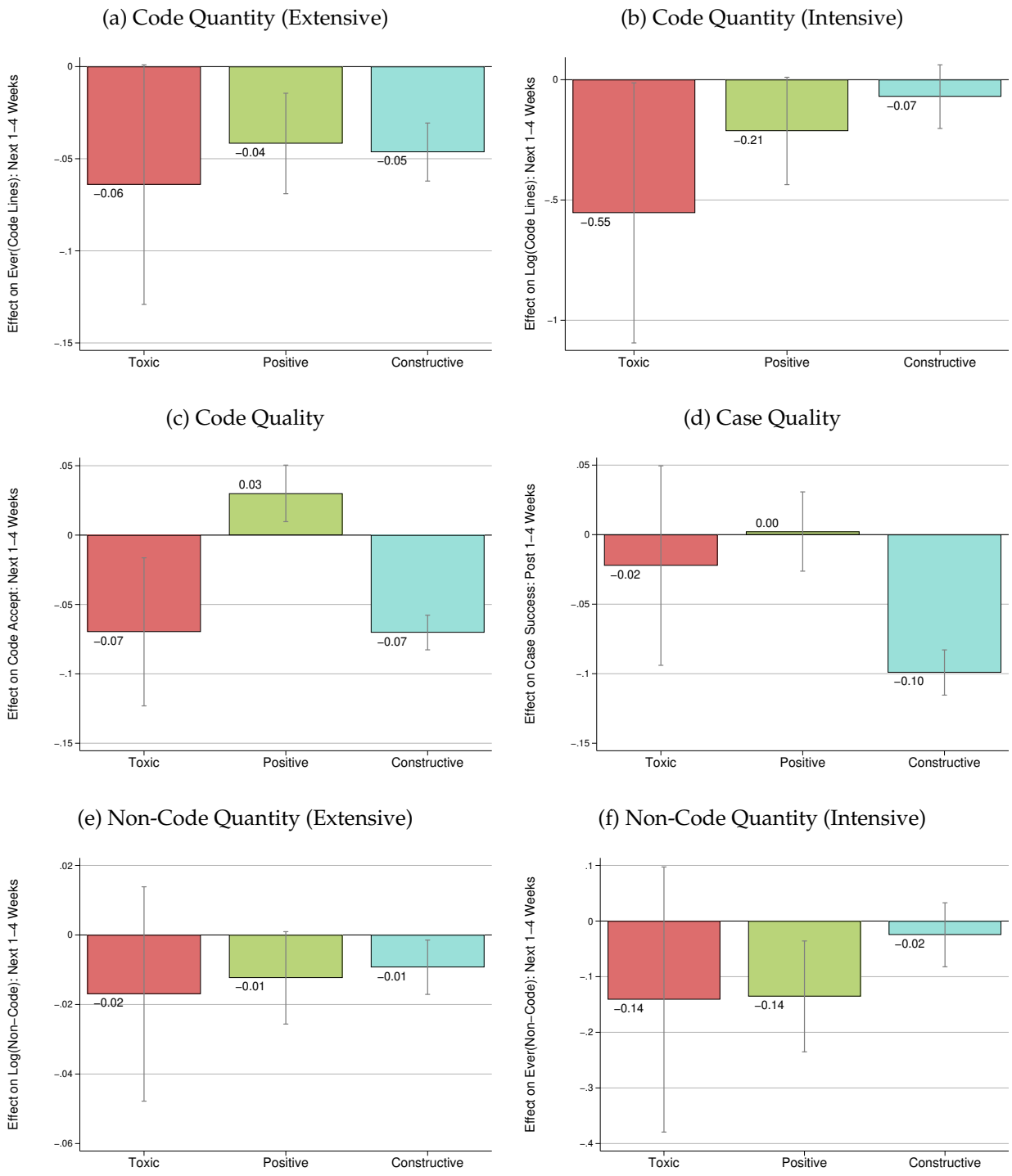
Notes: Panels (a) and (b) show estimates of the effect of feedback on code quantity and code quality for developers in the next 1-4 weeks. Standard errors are clustered at the team level. Lines report 90% confidence intervals.

Table D2: Subgroup First Stages (Monotonicity Diagnostic)

	Split I (Gender)		Split II (Asian)		Split III (Activity)		Split IV (Experience)		Split V (Followers)	
	Male	Others	Asian	Others	Less	More	Less	More	Less	More
Panel A. Toxic										
Reviewer Toxicity	0.701***	0.708***	0.727***	0.714***	0.706***	0.697***	0.702***	0.712***	0.701***	0.725***
	(0.015)	(0.046)	(0.046)	(0.018)	(0.014)	(0.028)	(0.016)	(0.023)	(0.015)	(0.034)
F-Stat.	2,087.50	239.38	251.04	1,623.17	2,721.44	632.20	1,866.46	970.15	2,257.34	451.62
Panel B. Positive										
Reviewer Positivity	0.816***	0.789***	0.832***	0.816***	0.813***	0.846***	0.807***	0.845***	0.824***	0.811***
	(0.006)	(0.017)	(0.015)	(0.007)	(0.005)	(0.009)	(0.006)	(0.006)	(0.005)	(0.013)
F-Stat.	18,476.72	2,225.65	2,964.45	13,847.83	26,252.07	8,830.59	16,173.51	17,913.26	28,733.81	4,078.64
Panel C. Constructive										
Reviewer Constructiveness	0.860***	0.866***	0.885***	0.858***	0.867***	0.859***	0.852***	0.885***	0.867***	0.849***
	(0.004)	(0.011)	(0.010)	(0.005)	(0.004)	(0.007)	(0.005)	(0.005)	(0.004)	(0.010)
F-Stat.	38,760.69	5,984.54	7,343.11	28,196.83	47,478.40	16,968.30	35,778.10	28,685.66	55,841.15	7,488.37
N	789,792	89,921	109,912	557,633	829,262	350,017	719,968	459,309	983,909	195,407

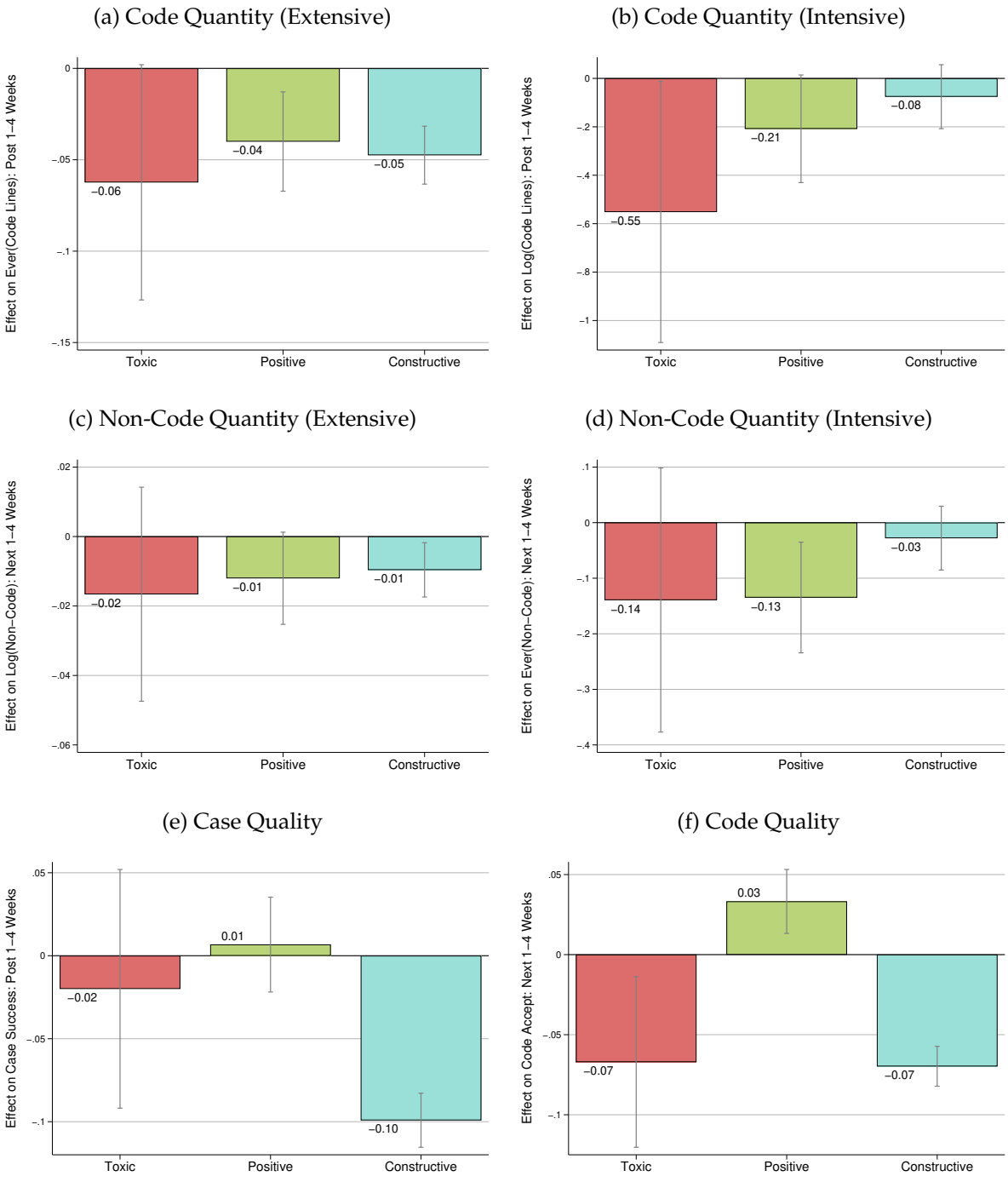
Notes: This table reports the first-stage coefficients and F-statistics from regressions of an indicator for receiving each feedback type on the corresponding reviewer instrument across different sample splits. Columns 1–2 separate the sample by gender, Columns 3–4 by race (Asian vs. others), Columns 5–6 by pre-week activity, Columns 7–8 by experience (based on first active year on GitHub), and Columns 9–10 by number of followers. Standard errors are clustered at the reviewer level. Significance levels: * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Figure D2: Jointly Instrumenting All Three Feedback Dimensions



Notes: The sample is the random reviewer sample from 2017 to 2023. Controls include all variables listed in Table 5. The results for each type of feedback are from the augmented IV models given by equations (3)–(6). Standard errors are clustered at the reviewer level. Lines report 90% confidence intervals.

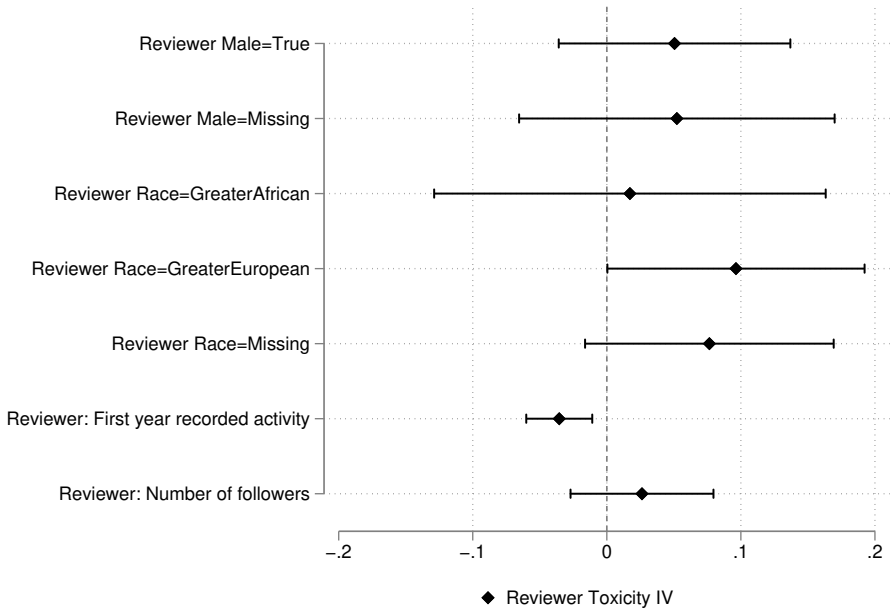
Figure D3: Controlling for Reviewer Feedback Styles in Other Feedback Dimensions



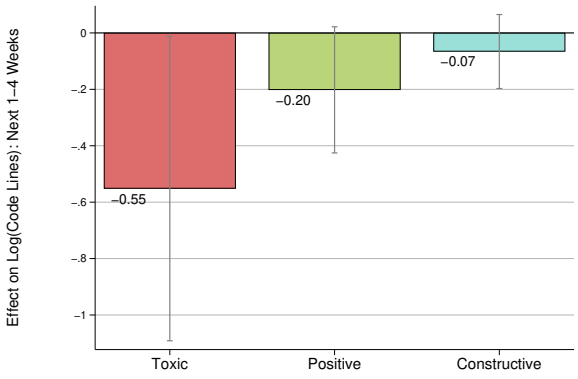
Notes: Baseline sample of cases for random reviewer sample from 2017 to 2023. Controls include all variables listed in Table 5. Each panel reports augmented IV estimates that add reviewer-style instruments for the non-focal feedback dimensions as controls. The toxicity results control for the reviewer positivity and constructiveness instruments; the positivity results control for the reviewer toxicity and constructiveness instruments; and the constructiveness results control for the reviewer toxicity and positivity instruments. Standard errors are clustered at the reviewer level. Lines report 90% confidence intervals.

Figure D4: Controlling for Reviewer Observable Characteristics

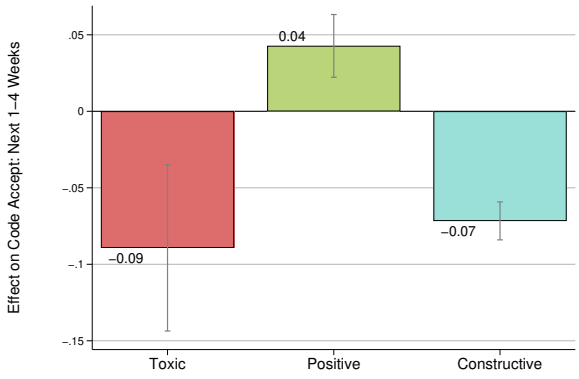
(a) Reviewer Characteristics and Reviewer Toxicity IV



(b) Code Quantity

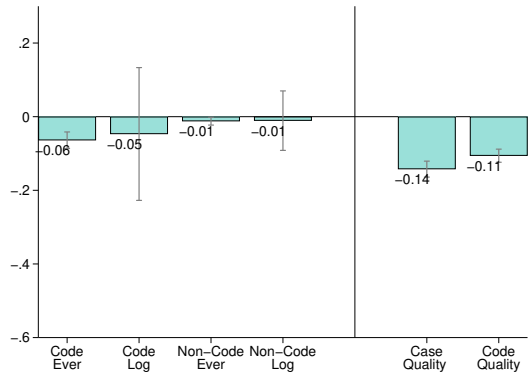


(c) Code Quality



Notes: Panel (a) reports OLS regression coefficients for reviewer toxicity based on standardized reviewer characteristics. The controls include gender, race, the number of followers (in 2023), the first year of GitHub activity (seniority), and reviewer activity in the past year. Standard errors are clustered at the reviewer level. Panels (b) and (c) report the main outcomes of code quantity and code quality when controlling for reviewer characteristics listed in panel (a). The sample and existing controls are the same as in Table 5.

Figure D5: Effect of Constructive Feedback on Productivity: Alternative Constructiveness Cutoff



Notes: This figure reports the effect of constructive feedback on developer productivity using an alternative definition of constructive feedback. The main analysis defines feedback as constructive when the LLM-predicted constructiveness score is at least 4. This figure lowers the cutoff to at least 3 and reestimates the leave-developer-out reviewer constructiveness instrument. Lines report 90% confidence intervals.

E Reviewer Value-Added and Feedback Prediction

E.1 Reviewer Value-Added: Measurement and Validation

I measure reviewer quality by reviewer value-added (VA) to developer productivity. The central challenge is sorting. If more productive developers are systematically matched with particular reviewers, estimated VA may reflect developer ability rather than reviewer quality. This is the same problem faced in the teacher value-added literature, where students of different baseline ability may be assigned to different teachers (Rothstein, 2010, 2017).

The ideal design would randomly assign many developers to each reviewer and measure whether developers systematically produce more after working with that reviewer, relative to their own baseline. I approximate this ideal in two steps. I first estimate reviewer VA in the random reviewer sample, where reviewer assignment is as good as random. I then extend the estimator to the full sample and validate it using reviewers who appear in both random and non-random assignment settings.

I adopt the forecast-based estimator of Chetty, Friedman, and Rockoff (2014). The estimator residualizes developer output on lagged output, developer characteristics, team characteristics, and case characteristics. It then forecasts a reviewer's contribution in a period using the reviewer's residualized performance in other periods. This jackknife procedure reduces mechanical correlation with the outcome being validated and shrinks noisy estimates toward the mean. Section E.2 gives the formal estimator.

Developer weekly code output is the total number of code lines submitted to a team in a week. In the VA regressions, I control for code-case characteristics, team- and firm-level output, and developer demographics. Since reviewers are assigned in week t, but some code produced in week t may predate the review, I compare output in $t+1$ to output in $t-1$, using week t only for assignment. I winsorize output at the 95th percentile, standardize log code lines, and include an indicator for missing output.

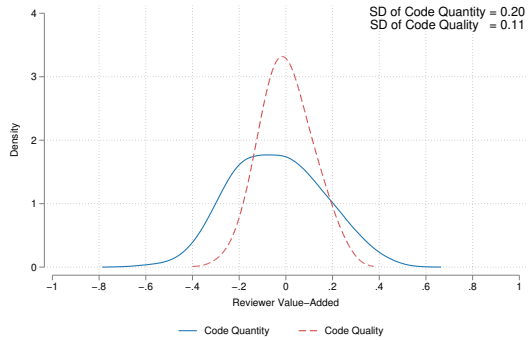
Figure E1 summarizes the VA estimates and their validation. Panel (a) plots the distribution of reviewer VA in the random reviewer sample. A one standard deviation increase in reviewer VA is associated with 0.20 standard deviations higher developer code quantity and 0.10 standard deviations higher developer code quality. Panel (b) shows that VA based on code quantity and VA based on code quality are positively correlated. Panel (c) compares reviewers who appear in both the random and non-random samples; their VA rankings are strongly correlated, supporting the use of the estimator in the full sample.

E.2 The Forecast-Based Value-Added Estimator

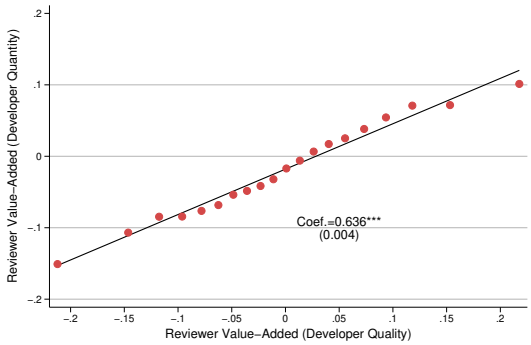
I estimate reviewer value-added (VA) for the full sample using the forecast-based estimator proposed by Chetty et al. (2014). Equation (7) models developer code output Y_{isjt} after receiving feedback from reviewer j, controlling for lagged output $Y_{i,t-1}$, developer characteristics $\mathbf{X}_{it}$, and team characteristics $\mathbf{S}_{st}$. Reviewer effects are captured through a vector of indicators $\mathbf{T_j}$,

Figure E1: Reviewer Value-Added: Distribution, Correlation, and Validation

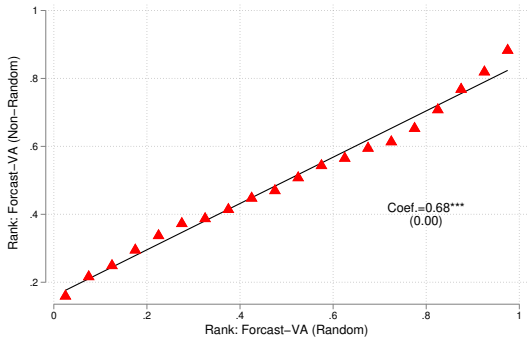
(a) Distribution of VA Estimates



(b) Correlation of VA Measures



(c) Validation of VA Rankings



Notes: Panels (a) and (b) use the random reviewer sample. **Panel (a)** plots the empirical distribution of reviewer value-added (VA) estimated from developer code quantity and code quality. **Panel (b)** plots the relationship between reviewer VA based on code quantity and reviewer VA based on code quality. **Panel (c)** compares VA rankings for reviewers who appear in both the random and non-random reviewer samples. Each dot is a bin of reviewers; the slope and standard error come from a regression of one ranking on the other. The validation sample contains 6,952 reviewers.

with coefficients $\boldsymbol{\theta}$ representing reviewer value-added. From this regression, I obtain residuals Y^*_{isjt} by subtracting the fitted values excluding reviewer fixed effects, as shown in Equation (8). I then compute $\bar{Y}^*_{jt}$, the average residual for reviewer j in week t, defined in Equation (9). Finally,

Equation (10) forecasts reviewer VA γ_{jt} based on the leave-one-week-out residual averages $\bar{\mathbf{Y}}^*_{\mathbf{jt}-}$. Throughout, t indexes the assignment week. The outcome Y_{isjt} is developer output in week $t+1$, the first full week after the review, and the lagged control $Y_{i,t-1}$ is output in the week before assignment. Week t itself is used only for assignment and is excluded from both, since code produced in that week may predate the review.

$$Y_{isjt} = \beta_0 + Y_{i,t-1}\beta_1 + \mathbf{X_{it}}'\boldsymbol{\beta_2} + \mathbf{S_{st}}'\boldsymbol{\beta_3} + \mathbf{T_j}'\boldsymbol{\theta} + \varepsilon_{isjt} \tag{7}$$

$$Y^*_{isjt} = Y_{isjt} - \left[\beta_0 + Y_{i,t-1}\beta_1 + \mathbf{X_{it}}'\boldsymbol{\beta_2} + \mathbf{S_{st}}'\boldsymbol{\beta_3}\right] \tag{8}$$

$$\bar{Y}^*_{jt} = \frac{1}{n_{jt}} \sum_{i \in \{i:j(i,t)=j\}} Y^*_{isjt} \tag{9}$$

$$\gamma_{jt} = \pi_0 + \overline{\mathbf{Y}}^{*\prime}_{j,-t}\boldsymbol{\pi}_1 + u_{jt} \tag{10}$$

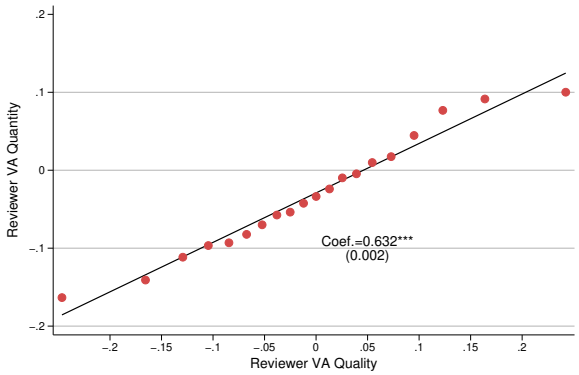
The forecast-based estimator offers several advantages for estimating reviewer value-added. First, instead of including reviewer fixed effects directly in the regression, the model omits them from the fitted values. This procedure yields a modified residual, Y^*_{isjt}, defined in Equation (8), that preserves the full reviewer contribution while removing variation explained by past developer output and other covariates. By residualizing the outcome without partialling out reviewer fixed effects, I avoid the shrinkage toward zero that would arise from estimating $\boldsymbol{\theta}$ directly for reviewers observed in only a few weeks. Second, to avoid overfitting and make use of prior information, the model predicts reviewer value-added in week t using the average residuals from all other weeks (a leave-week-out approach). This produces $\hat{\gamma}_{jt}$, the best linear predictor of reviewer j's impact in week t. Third, Equation (10) allows the reviewer VA to vary over time.

E.3 Predicting Reviewer Value-Added from Feedback

To predict reviewer value-added, I apply the Extreme Gradient Boosting (XGBoost) algorithm. The goal is descriptive: I ask how much independently measured feedback characteristics predict independently estimated reviewer VA. For the continuous outcome, the model uses squared error loss (*reg:squarederror*) and is evaluated using out-of-sample R^2. I randomly split the data into a training set (80%) and a hold-out test set (20%). Model training and tuning are performed on the training set using five-fold cross-validation.

Hyperparameters are selected using a two-stage procedure: a random search explores a wide parameter space, followed by a grid search that refines the best-performing regions. I repeat the tuning process separately for each predictor set. Figure E4 examines how feedback explains variation in reviewer value-added, measured by changes in developer code quality before and after assignment. Panel (a) uses the random-assignment sample, while Panel (b) uses the full sample, including both random and non-random assignments. Reviewer value-added is estimated using

Figure E2: Reviewer Value-Added Based on Developer Code Quantity vs. Code Quality

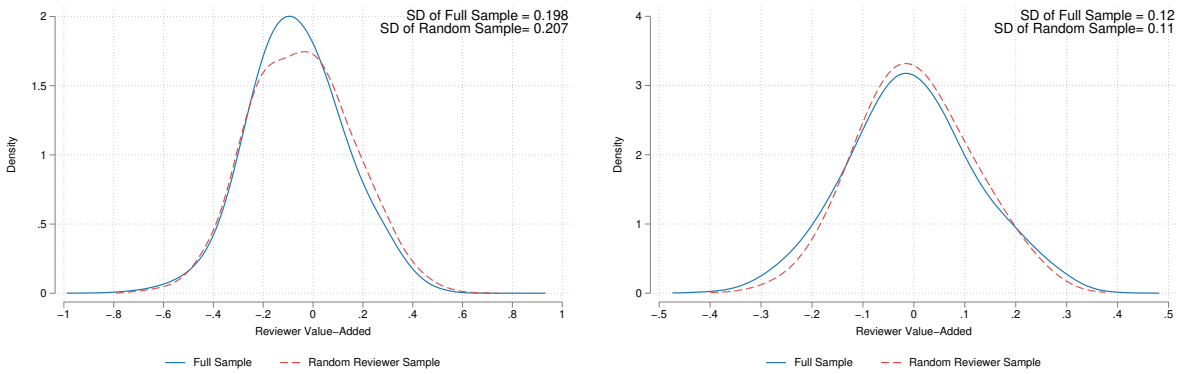


Notes: This figure compares reviewer VA estimates based on two different developer outcomes: code quantity (y-axis) and code quality (x-axis) in the full sample. Reviewer VA is estimated using the forecast-based estimator by Chetty, Friedman, and Rockoff (2014). The fitted line shows a positive and statistically significant relationship between the two measures, indicating that reviewers who increase developer output also tend to improve code quality.

Figure E3: Reviewer VA Distribution: Full Sample versus Random Reviewer Sample

(a) VA based on Code Quantity

(b) VA based on Code Quality



Notes: This figure compares the distributions of reviewer VA measured in the random and full samples, using code quantity and code quality, respectively.

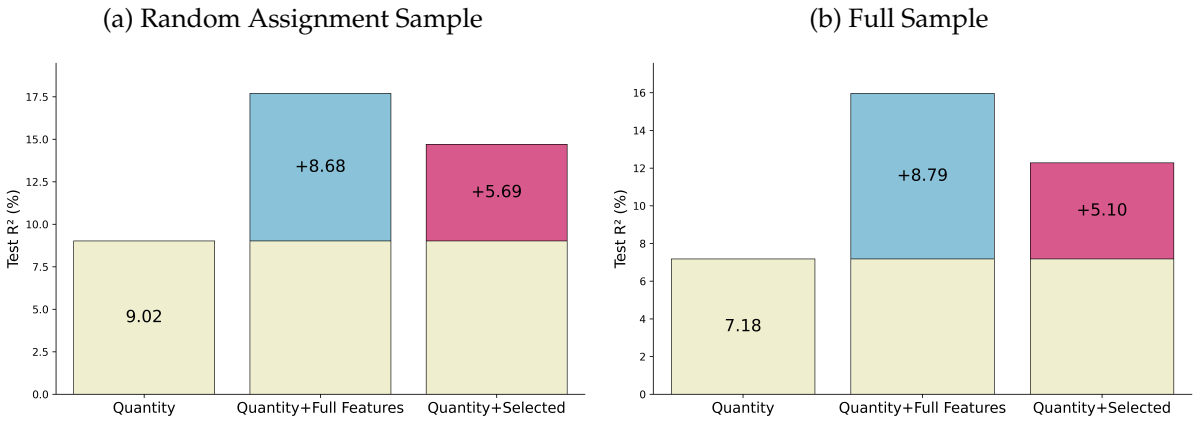
the forecast-based estimator of Chetty et al. (2014).

I compare four sets of predictors. The first is reviewer demographics, measured by gender and race. The second is feedback quantity, measured as the total number of messages reviewers send per week. The third is full feedback text, represented by 768-dimensional semantic embeddings. The fourth is feedback quantity combined with selected feedback features, defined as the weekly shares of toxic, positive, and constructive feedback per reviewer.

The results show that feedback explains substantial variation in reviewer value-added. In

the random sample, feedback quantity alone accounts for about 9 pp of the variance. Adding embeddings increases explanatory power by roughly 18 pp. Using only the selected features captures most of this improvement. The full sample yields similar patterns.

Figure E4: Predicting Reviewer Value-Added from Feedback: Developer Code Quality



Notes: This figure shows how different predictors explain variation in reviewer value-added (VA), measured by the reviewer's effect on developer code quality. Each bar reports out-of-sample R^2. *Demographics* includes reviewer gender and race. *Quantity* is the number of feedback messages written. *Quantity + Full Text* adds 768-dimensional text embeddings. *Quantity + Selected Feedback* adds the weekly shares of toxic, positive, and constructive feedback per reviewer. Values on the bars indicate the improvement in explanatory power relative to the quantity-only model.

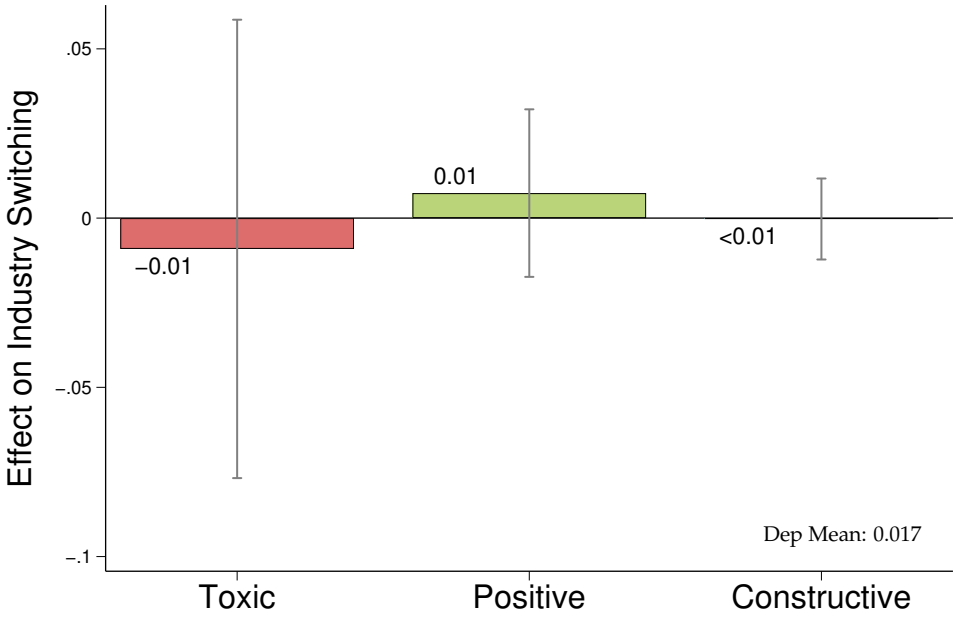
F Linking GitHub to LinkedIn Career Histories

The goal is to build a panel on developers' communications, activities, and employers by merging GitHub with Revelio Labs. I obtain names, employers, biographies, profile images, locations, and emails from the GitHub API. Because profiles are used for career search, developers tend to report accurate details (El-Komboz and Goldbeck, 2024). In the random reviewer assignment sample, about 70% of developers list a LinkedIn profile, personal website, or other social account on GitHub, and many reuse the same photo, which aids identification.

The matching process proceeds in three steps. First, I create direct matches for developers who link their LinkedIn URL on GitHub or vice versa. Second, for unmatched cases, I construct a match pool using names, locations, employment histories, and biographies. To account for naming variations, I match on common nicknames (for example, "Alex" for "Alexander" or "Alexandra"), abbreviated names (for example, "Tina L." for "Tina Liu"), and names with or without middle names (for example, "Tina Jin Liu" simplified to "Tina Liu"). The pool includes exact matches, first name with last initial matches, and matches involving middle names; developers with entirely different names or typos are excluded. Third, I incorporate additional characteristics such as position, firm, and geographic location to identify the best LinkedIn candidate for each GitHub developer. A research assistant reviews all candidates in the pool and selects the best match. I also tested an LLM-based approach (GPT-4o-mini and DeepSeek R1) to select matches, but its performance was inferior to manual review.

Employer switching is measured from LinkedIn employment spells. For each developer-week, I identify the LinkedIn employer active in that week and define an employer switch as the start of a subsequent LinkedIn position at a different employer within 365 days. I define an occupation switch analogously, as a subsequent position in a different SOC major group (the first two digits of the O*NET-SOC code) within 365 days; occupation measures the worker's role, unlike NAICS, which measures the employer's industry. I define a broad-industry switch as a subsequent position in a different one-digit NAICS sector group (the leading digit of the six-digit NAICS code) within 365 days. The one-digit grouping is coarse: leading digit 5, for example, spans Information (51), Finance (52), Real Estate (53), Professional, Scientific and Technical Services (54), Management (55), and Administrative Services (56), so moves within that block are not counted as industry switches. These measures capture transitions in the developer's reported LinkedIn career history; they do not require the LinkedIn employer to be the GitHub repository owner. A switch is coded only over a fully observed follow-up window, so a developer-week whose 365-day horizon extends past the end of the LinkedIn panel is treated as missing rather than as a non-switch. When a developer holds several overlapping LinkedIn positions in the same week, I collapse them to a single person-week so that a move to a *simultaneous* position is not counted as a switch. The employer-switch result is robust to keying the switch on Revelio's canonical firm identifier (`rcid`) rather than the employer name. Figure F1 reports the effects of feedback on broad-industry switches, and Figure 9b panel (b) reports occupation switches. Neither is significantly affected by any feedback type.

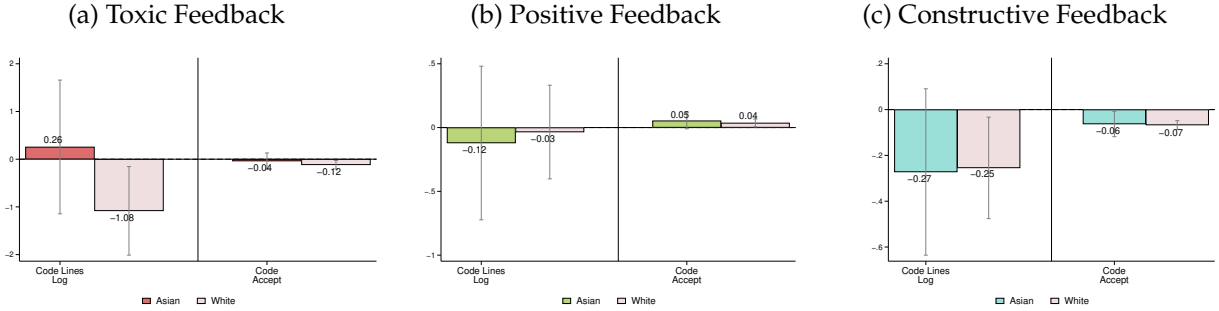
Figure F1: Effect of Feedback on Industry Switching



Notes: This figure reports IV estimates of the effects of toxic, positive, and constructive feedback on the probability that a developer switches broad industries within one year. Industry switches are measured from LinkedIn career histories. Feedback exposure is instrumented using leave-one-developer-out reviewer feedback styles in the random reviewer sample. Coefficients are reported in percentage points. Vertical lines report 90% confidence intervals.

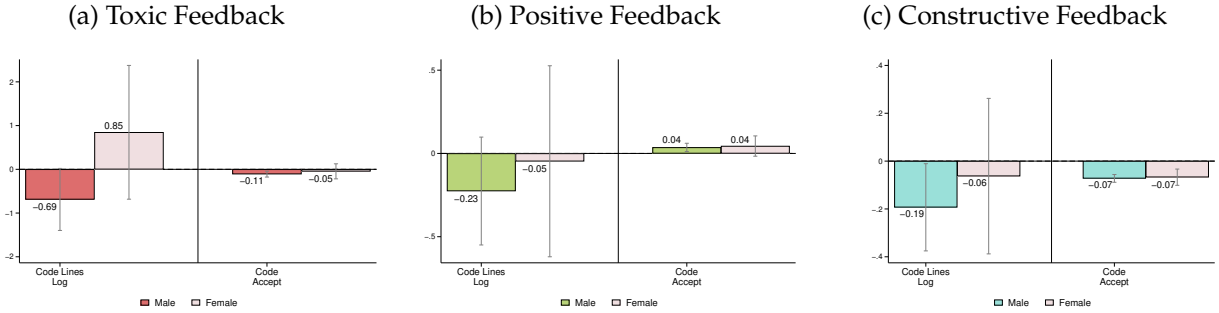
G Exploratory Heterogeneity and Additional Outcomes

Figure G1: Heterogeneous Effects of Feedback by Developer Race (Asian versus White)



Notes: This figure shows the heterogeneous effects of feedback on developer outcomes during the one to four weeks following feedback, separately by developer race. Dark bars denote Asian developers, and light bars denote White developers. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Figure G2: Heterogeneous Effects of Feedback by Developer Gender



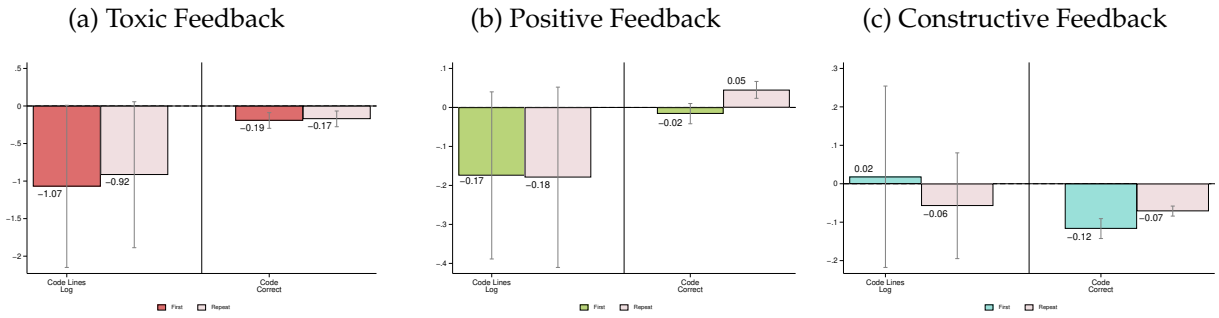
Notes: This figure shows the heterogeneous effects of feedback on developer outcomes in the 1–4 weeks following feedback, by developer gender. Panels (a)–(c) report estimates for toxic, positive, and constructive feedback, respectively. Dark bars represent male developers; light bars represent female developers. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Table G1: Assessing Bias in On-Platform Feedback Effects

	Interaction	s.e.(Interaction)	p-value
Toxic Feedback			
Code Quantity Ever	0.199	0.176	0.261
Code Quantity Log	−1.625	1.584	0.305
Code Quality	0.206	0.165	0.210
Case Quality	0.082	0.202	0.684
Non-Code Quantity Ever	0.033	0.101	0.744
Non-Code Quantity Log	0.557	0.637	0.382
Positive Feedback			
Code Quantity Ever	−0.004	0.067	0.949
Code Quantity Log	0.685	0.451	0.129
Code Quality	−0.058	0.052	0.263
Case Quality	−0.000	0.068	1.000
Non-Code Quantity Ever	0.014	0.037	0.714
Non-Code Quantity Log	0.113	0.202	0.576
Constructive Feedback			
Code Quantity Ever	0.033	0.034	0.326
Code Quantity Log	−0.369	0.236	0.118
Code Quality	0.019	0.026	0.463
Non-Code Quantity Ever	−0.014	0.018	0.439
Non-Code Quantity Log	0.251**	0.113	0.026
Case Quality	−0.012	0.036	0.737
Team × Year FE	✓	✓	✓
Decile t–1 activity	✓	✓	✓
Decile t–1 message	✓	✓	✓

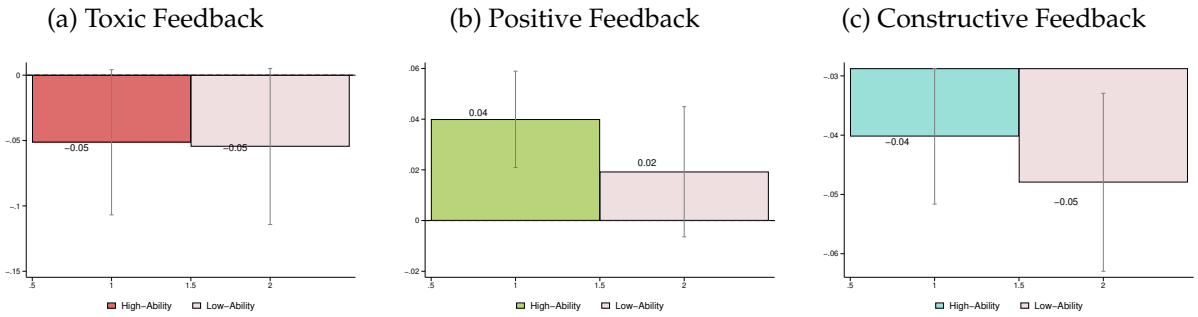
Notes: The "Interaction" column reports the coefficient on $I^{\theta}_{im,t} \times C_{im}$. The Co-located dummy C_{im} equals 1 if all team members are in the same country. See Appendix A.6 for details on how team location is constructed. Outcomes are measured 1–4 weeks after the developer received feedback. Standard errors are clustered at the reviewer level. * $p < .10$, ** $p < .05$, *** $p < .01$.

Figure G3: Heterogeneous Effects of Feedback by First vs. Repeat Exposure



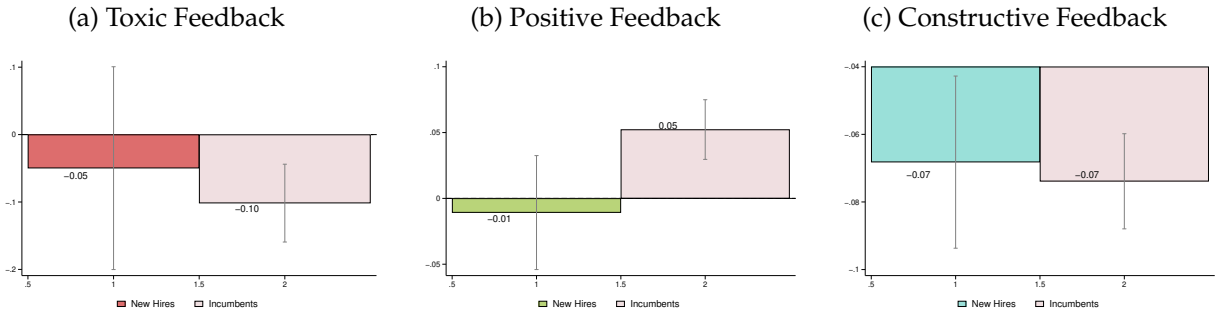
Notes: This figure shows the heterogeneous effects of reviewer feedback by whether the feedback was given on a developer's first time ("First") or later ("Later"). Dark bars represent first-time feedback; light bars represent later feedback. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Figure G4: Heterogeneous Effects of Feedback by Developer Ability



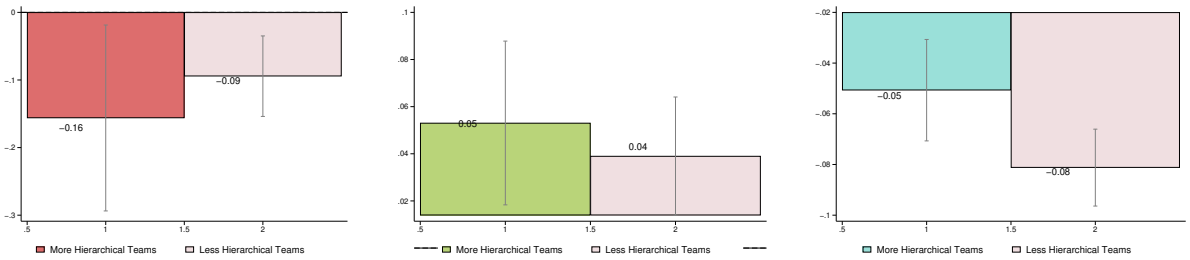
Notes: This figure shows how the effects of reviewer feedback vary by developer ability, defined using lagged code quality. Developers are split into high- and low-ability groups based on their past performance. Dark bars represent high-ability developers; light bars represent low-ability developers. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Figure G5: Heterogeneous Effects of Feedback by New Hires vs. Incumbents



Notes: This figure shows the heterogeneous effects of reviewer feedback by worker status as a new hire (within the first two months on a team) or as an incumbent (longer than two months). Dark bars represent new hires; light bars represent incumbents. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

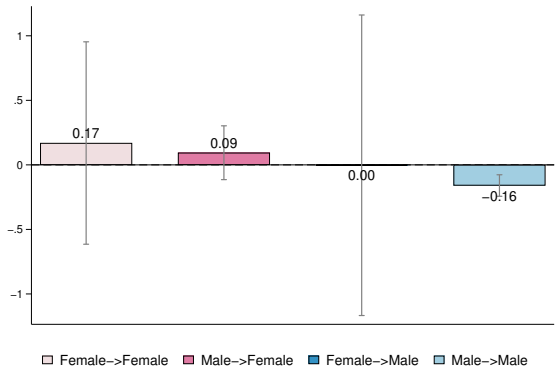
Figure G6: Heterogeneous Effects in More vs. Less Hierarchical Teams



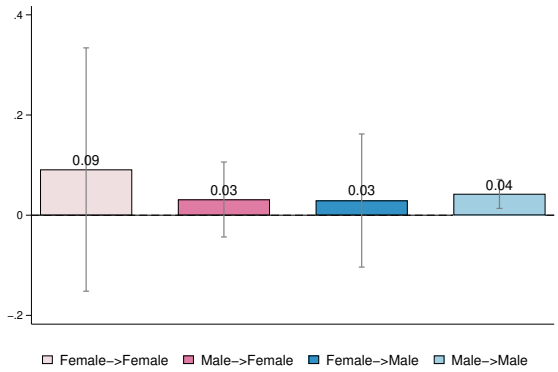
Notes: The figure shows heterogeneous effects by team hierarchy score (see Section 3.4). Dark bars represent hierarchical teams with a score greater than 0.5; light bars represent flat teams. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Figure G7: Gender Match Effects of Feedback on Developer Productivity

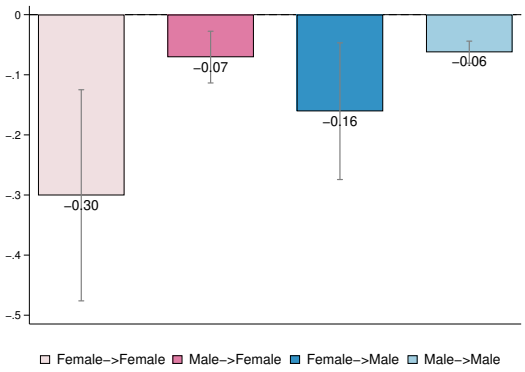
(a) Toxic Feedback



(b) Positive Feedback



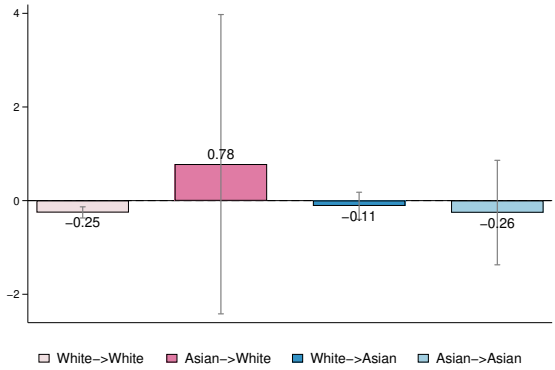
(c) Constructive Feedback



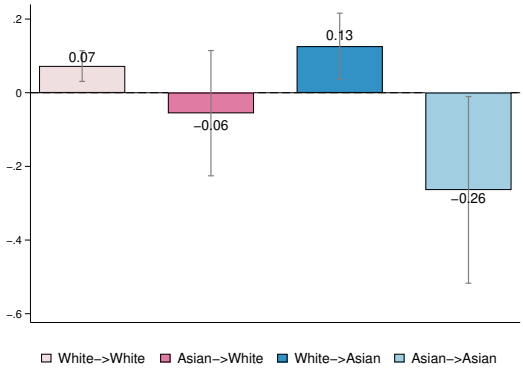
Notes: This figure shows the heterogeneous effects of demographic match between reviewer and developer. For example, "Female → Male" indicates feedback from a female reviewer to a male developer. Standard errors are clustered at the reviewer level. Vertical lines represent 90% confidence intervals.

Figure G8: Race Match Effects of Feedback on Developer Productivity

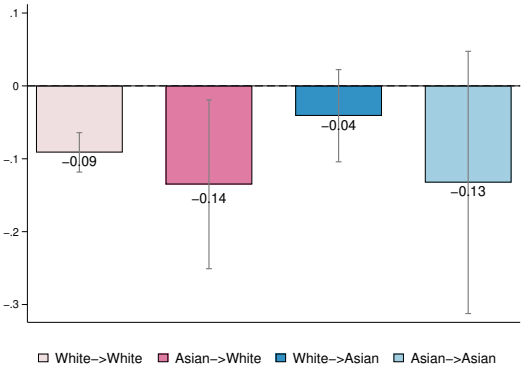
(a) Toxic Feedback



(b) Positive Feedback



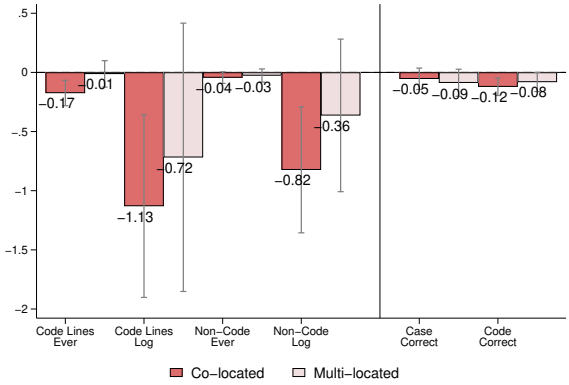
(c) Constructive Feedback



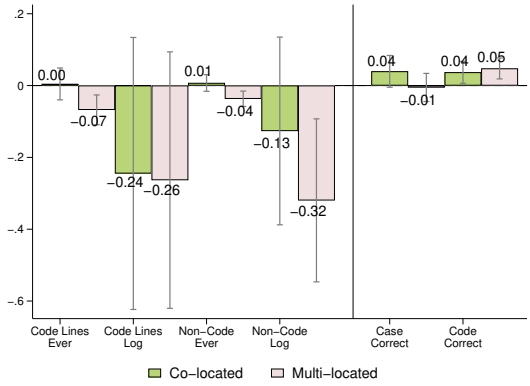
Notes: This figure shows the heterogeneous effects of demographic match between reviewer and developer. For example, "White → Asian" indicates feedback from a White reviewer to an Asian developer. Standard errors are clustered at the reviewer level. Vertical lines represent 90% confidence intervals.

Figure G9: Heterogeneous Effects of Feedback by Multi-location Team Structure

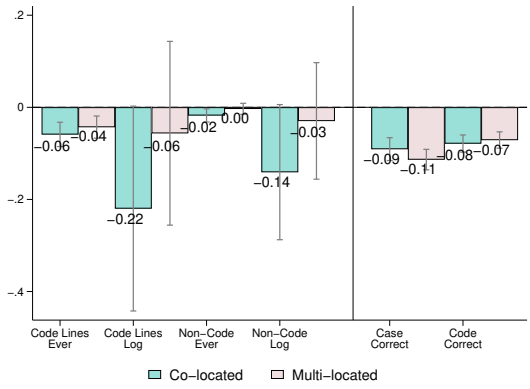
(a) Toxic Feedback



(b) Positive Feedback



(c) Constructive Feedback

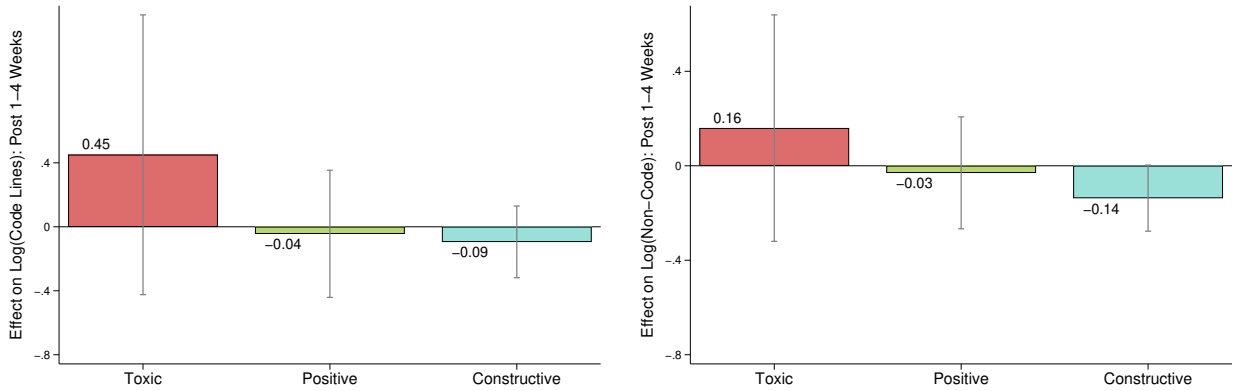


Notes: This figure splits teams by whether the share of members in the most represented country is above or below the sample median. This is a coarser, more balanced split than the co-location definition used in Table G1, where a team is co-located only if *all* members share a country. Dark bars: concentrated teams (above-median share). Light bars: dispersed teams (below-median share). I estimate the effect in each subsample. Baseline sample of cases for the random reviewer sample from 2017 to 2023. Controls include all variables listed in Table 5. Standard errors are clustered at the reviewer level. Vertical lines show 90% confidence intervals.

Figure G10: Effect of Feedback on Developer Outcomes in Other Teams

(a) Code Quantity

(b) Non-Code Quantity

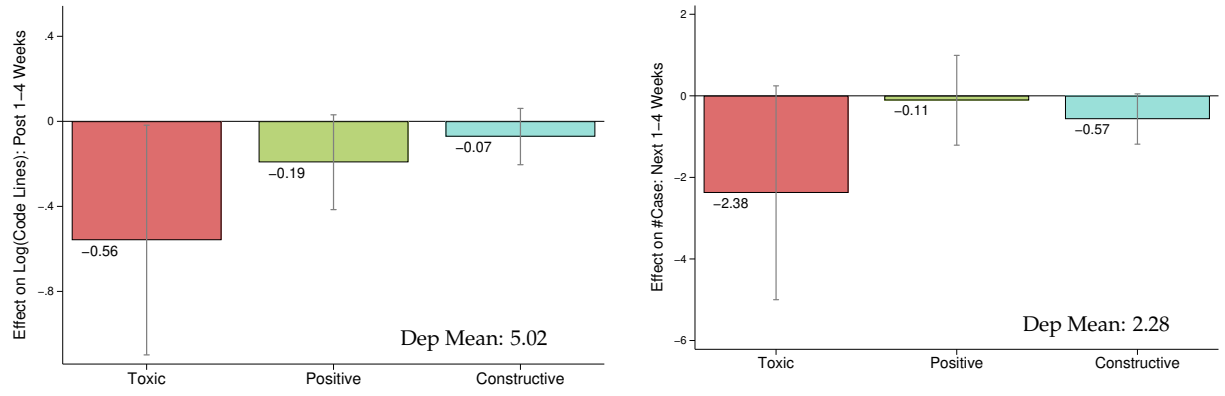


Notes: This figure shows the 2SLS estimates of receiving feedback in a focal team on a developer's productivity in *other* teams during the following 1–4 weeks. Standard errors are clustered at the reviewer level. Lines report 90% confidence intervals.

Figure G11: Effect of Feedback on Code Quantity and Number of Cases

(a) Code Quantity

(b) Number of Cases



Notes: This figure reports 2SLS estimates of the effect of a given feedback type on developer outcomes within the same team during weeks 1–4 after the feedback. The sample includes teams with random reviewer assignment, at least two reviewers per year, and reviewers with at least 10 cases per year. Standard errors are clustered by reviewer, and lines show 90% confidence intervals.